COLLÈGE DE PARIS SUPÉRIEUR

RÉPUBLIQUE TOGOLAISE

Information Technology

Tronc Commun

Travail – Liberté – Patrie

INF-1001

Introduction à la Programmation

FONDAMENTAUX DU LANGAGE PYTHON

De l'algorithmique à la programmation Python

Variables, conditions, boucles, fonctions, structures de données

Python IDE Débogage

Bachelor 1 (B1) | Semestre 1 | 4 ECTS

Enseignant : Dr. AFENYIVEH Dzo M. Serge

dzo.serge@gmail.com | d.afenyiveh@univkara.com

(+228) 91 75 98 33 | 98352331

Collège de Paris Supérieur | 2024–2025

Version 1.0 | Mise à jour : 10 janvier 2026

Sommaire

Sommaire i

SYLLABUS DU COURS 1

1 INTRODUCTION ET ENVIRONNEMENT PYTHON 7
- 1.1 Historique, philosophie et caractéristiques de Python 8
- 1.2 Installation et gestion des packages Python 12
- 1.3 Découverte du REPL et premières commandes : Votre premier laboratoire Python 17
- 1.4 Environnements de développement (IDEs) : Votre poste de travail Python complet 21
- 1.5 Structure d'un programme Python : Organiser votre premier code 28
- 1.6 Variables et types de données de base : Les fondations de l'information IT 31
- 1.7 Les Opérateurs en Python : Manipuler les données 40
- 1.8 Travaux Pratiques (TP) du Module 1 48
- 1.9 Quiz du Module 1 : Testez vos connaissances 54

2 STRUCTURES DE CONTRÔLE : LOGIQUE ET AUTOMATISATION ... 57
- 2.1 Expressions Booléennes : La base de la logique 58
- 2.2 Structures Conditionnelles : Prendre des décisions 60
- 2.3 La Boucle `for` : Itérer sur des séquences 66
- 2.4 La Boucle `while` : Répéter tant que... 70
- 2.5 Contrôle du Flux : `break`, `continue`, `pass` 73
- 2.6 Cas Pratiques IT 75
- 2.7 Travaux Pratiques du Module 2 77
- 2.8 Quiz du Module 2 80

3 FONCTIONS EN PYTHON : MODULARITÉ ET RÉUTILISABILITÉ 83
- 3.1 Définition et Appel de Fonctions 85
- 3.2 Retour de Valeurs avec `return` 87
- 3.3 Types de Paramètres — Flexibilité maximale 90

3.4 Portée des Variables — Pourquoi « x n'existe pas » ? 94
3.5 Fonctions Intégrées — Votre boîte à outils Python 96
3.6 Modules Standards — Vos super-pouvoirs 97
3.7 Fonctions Lambda — Pour les petites tâches rapides 99
3.8 Documentation des Fonctions — Pour vous et les autres 100
3.9 Travaux Pratiques du Module 3 102
3.10 Quiz du Module 3 105

4 STRUCTURES DE DONNÉES : STOCKER, ORGANISER ET MANIPULER L'INFORMATION 109
4.1 Les Listes : Collections ordonnées et modifiables 111
4.2 Les Dictionnaires : Paires clé-valeur pour un accès ultra-rapide 115
4.3 Les Tuples : Collections ordonnées et immuables 119
4.4 Les Ensembles : Collections non ordonnées et sans doublons 123
4.5 Applications Pratiques — Du quotidien au professionnel 128
4.6 Travaux Pratiques du Module 4 133
4.7 Quiz du Module 4 138

5 MANIPULATION DES FICHIERS : STOCKER ET RÉCUPÉRER L'INFORMATION 142
5.1 La mémoire volatile vs persistante 143
5.2 La fonction `open()` : La porte d'entrée 144
5.3 Le mot-clé `with` : La méthode recommandée 145
5.4 Lire un fichier 147
5.5 Écrire et Ajouter 149
5.6 Gestion des erreurs courantes 151
5.7 Travailler avec des formats courants 153
5.8 Travaux Pratiques du Module 5 157
5.9 Quiz du Module 5 162

6 INTRODUCTION AU DÉBOGAGE ET BONNES PRATIQUES 166
6.1 Pourquoi le débogage est essentiel ? 167
6.2 Les trois types d'erreurs en Python 168
6.3 Techniques de débogage de base 178
6.4 Gestion des exceptions avec try/except 186

6.5 Introduction à l'utilisation des bibliothèques 194
6.6 Documentation et structuration du code 202
6.7 Bonnes pratiques pour un code clair et lisible 206
6.8 Travaux Pratiques 211
6.9 Quiz du Chapitre 216
6.10 Récapitulatif du Chapitre 218

Table des matières détaillée **220**

Syllabus du Cours

Programmation Python

Introduction à la Programmation en Langage Python

Bachelor 1 (B1) – Tronc commun

Description du Cours

Ce cours initie les étudiants aux concepts fondamentaux de la programmation en utilisant **Python**, un langage polyvalent et accessible, largement utilisé en *science des données*, *développement web* et *automatisation*.

Python Scripts Débogage Mini-projets

Informations Générales

Intitulé	Introduction à la Programmation en Langage Python
# Code	B1COM0304
Niveau	Bachelor 1 (B1) – Tronc commun
Durée	24 heures (CM + TD + TP)
Crédits	2
Semestre	Semestre 1
Prérequis	› Introduction à l'Algorithmique et structure de données

Objectifs Pédagogiques

À l'issue de ce cours, l'étudiant doit être capable de :

- ✅ Comprendre la syntaxe et les concepts fondamentaux du langage Python.
- ✅ Écrire et exécuter des scripts Python simples pour résoudre des problèmes.
- ✅ Utiliser les structures de données principales : listes, dictionnaires, tuples et ensembles.
- ✅ Appliquer les structures de contrôle pour la logique de programmation.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

- ✅ Déboguer et tester des programmes Python.

Contenu Pédagogique

1. Introduction au langage Python

- Historique et applications du langage Python
- Installation de Python et prise en main de l'environnement de développement (IDE)
- Structure d'un programme Python
- Types de données de base et variables

2. Structures de contrôle

- Les conditions : `if`, `else`, `elif`
- Les boucles : `for`, `while`
- Utilisation des boucles pour itérer sur des structures de données

3. Fonctions en Python

- Définition et appel de fonctions
- Paramètres et retour de valeurs
- Portée des variables (locale et globale)
- Fonctions intégrées et modules standards (`math`, `random`, etc.)

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

4. Structures de données

- Listes et opérations associées (ajout, suppression, tri, etc.)
- Dictionnaires : création, accès et manipulation
- Tuples et ensembles : caractéristiques et utilisation
- Applications pratiques des structures de données

5. Introduction à la manipulation des fichiers

- Lecture et écriture dans les fichiers
- Gestion des erreurs lors de l'accès aux fichiers
- Utilisation de fichiers pour stocker et récupérer des données

6. Introduction au débogage et bonnes pratiques

- Techniques de débogage de base
- Gestion des exceptions (`try`, `except`)
- Introduction à l'utilisation des bibliothèques
- Documentation et structuration du code
- Bonnes pratiques pour écrire un code Python clair et lisible

Approches Pédagogiques

- Exercices de programmation à chaque séance pour pratiquer les concepts.
- Utilisation de notebooks interactifs (Jupyter/Colab) pour faciliter l'expérimentation.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

- Travaux dirigés orientés vers la résolution de problèmes pratiques avec Python.
- Mini-projets pour renforcer les compétences en fin de cours.

Modalités d'Évaluation

40%	30%	30%
Contrôle Continu	**Projet Final**	**Examen Final**
Quiz, TP notés	Application + Rapport	Écrit + Analyse

Références Bibliographiques

[1] **Zelle, J.** *Python Programming : An Introduction to Computer Science.* Franklin, Beedle & Associates, 2010.

[2] **Matthes, E.** *Python Crash Course.* No Starch Press, 2016.

[3] **Sweigart, A.** *Automate the Boring Stuff with Python.* No Starch Press, 2015.

[4] **Lutz, M.** *Learning Python.* O'Reilly Media, 2013.

[5] **Python Software Foundation.** *Documentation officielle Python 3.* https://docs.python.org/3/

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

> *« Programmer, c'est apprendre à décomposer un problème en étapes simples et vérifiables. »*

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Chapitre I

Introduction et environnement Python

◎ Objectifs d'apprentissage

À l'issue de ce chapitre, vous serez capable de :

- ✓Connaître l'historique, la philosophie et les principes fondamentaux de Python.
- ✓Comprendre la place centrale de Python dans les métiers de l'IT : développement web, data science, automatisation, etc.
- ✓Maîtriser l'installation de Python et la gestion des bibliothèques (`pip`, `conda`).
- ✓Utiliser la console interactive (REPL) pour des tests rapides et exécuter des scripts Python.
- ✓Interagir avec l'utilisateur via les fonctions `print()` et `input()`.
- ✓Choisir et configurer un environnement de développement (IDE) adapté à vos projets IT.

1.1 Historique, philosophie et caractéristiques de Python

Python est un langage de programmation **de haut niveau**, **interprété**, **orienté objet** et **multiparadigme**. Son objectif central est la **lisibilité** : un code plus facile à relire se maintient mieux et coûte moins cher à faire évoluer, ce qui est déterminant en **Information Technology**. Le langage a été initié par **Guido van Rossum** en 1989 au *CWI* (Centrum Wiskunde & Informatica, Pays-Bas) et rendu public en 1991.

La philosophie *"batteries included"* signifie que Python fournit dès l'installation une bibliothèque standard riche (fichiers, JSON, réseau, dates, tests, etc.). En IT, cela se traduit par un gain immédiat : moins de dépendances à ajouter pour produire un script utile.

Le Zen de Python : principes utiles en ingénierie logicielle

Le *Zen de Python* (Tim Peters) est accessible via `import this`. Il sert de guide pour écrire du code exploitable en équipe :

- **Beau est mieux que laid. Utilité IT :** La lisibilité réduit le temps de revue, de maintenance et le taux d'erreurs.
- **Explicite est mieux qu'implicite. Utilité IT :** Un comportement explicite diminue les bugs et facilite le débogage en production.
- **Simple est mieux que complexe. Utilité IT :** La complexité augmente les risques (pannes, vulnérabilités, coûts).
- **La lisibilité compte. Utilité IT :** L'onboarding et la collaboration (Agile/DevOps) deviennent plus rapides.

1.1 .1 Sous le capot : interprété vs compilé (utile pour Dev/-Debug)

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

En informatique, la manière dont un langage est exécuté est fondamentale pour la performance, le déploiement et le débogage. **Python est un langage interprété**. Contrairement aux langages compilés (comme C++, Java où le code source est transformé en un fichier binaire exécutable avant d'être lancé), Python exécute le code via un programme intermédiaire : l'interpréteur. Il lit, analyse et exécute chaque instruction ligne par ligne, ou en convertit le code source en un "bytecode" intermédiaire exécuté par une machine virtuelle Python.

Cette distinction a des implications directes sur votre travail en IT :

- **Cycle de développement (DevOps) :** Rapidité d'essai et de modification du code. Pas de longue phase de compilation.
- **Déploiement et Distribution :** Moins de fichiers binaires à gérer, mais nécessite un interpréteur Python sur la machine cible.
- **Débogage (Debugging) :** Facilite l'identification des erreurs, car elles sont souvent détectées à l'exécution de la ligne concernée.
- **Performance :** Généralement plus lent à l'exécution qu'un langage compilé, mais la productivité du développeur est accrue. Pour pallier cela, les bibliothèques critiques de Python sont souvent écrites en C/C++ et "liées" (bindées) à Python.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

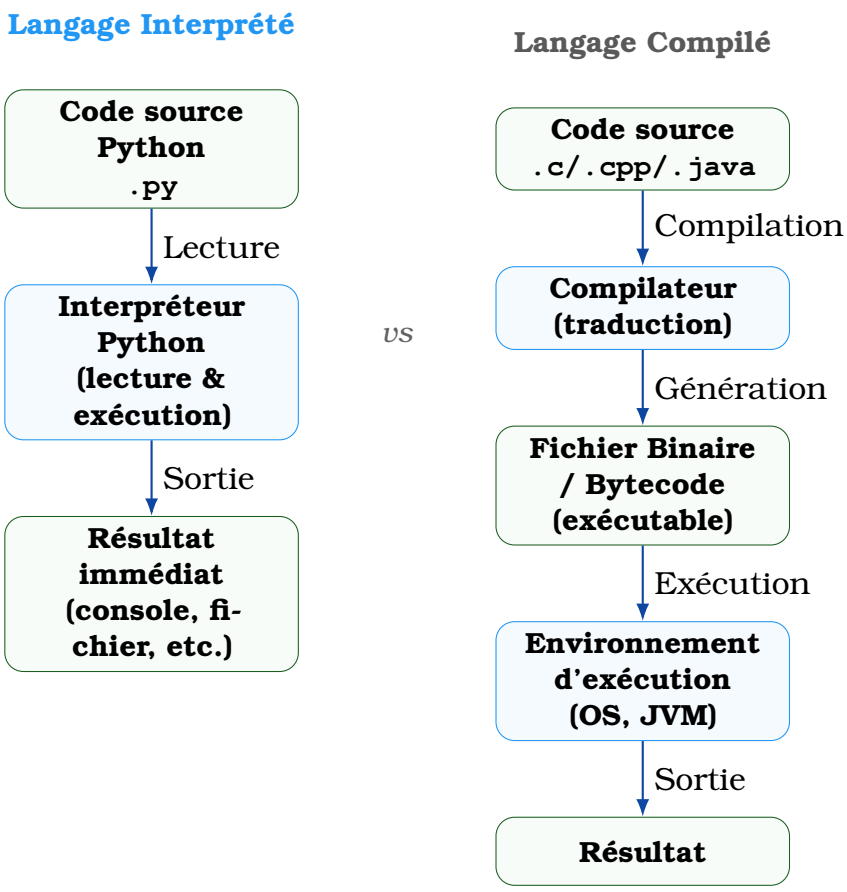


Figure 1.1 – *Comparaison des chaînes d'exécution : interprété (Python) vs compilé (ex : C++, Java).*

Utilité IT immédiate

Python est le choix privilégié pour les scripts d'administration système, les outils DevOps, les prototypes rapides d'applications web (APIs) ou les tests unitaires. Pour les tâches nécessitant des performances extrêmes (calculs intensifs, rendu graphique), on s'appuie sur des bibliothèques Python optimisées en C/C++ (comme NumPy) ou des architectures distribuées.

1.1.2 Carte conceptuelle : caractéristiques et bénéfices IT

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Python offre un ensemble de caractéristiques qui se traduisent par des avantages directs et concrets pour le développeur et l'ingénieur IT. Comprendre cette synergie est essentiel pour exploiter pleinement le langage.

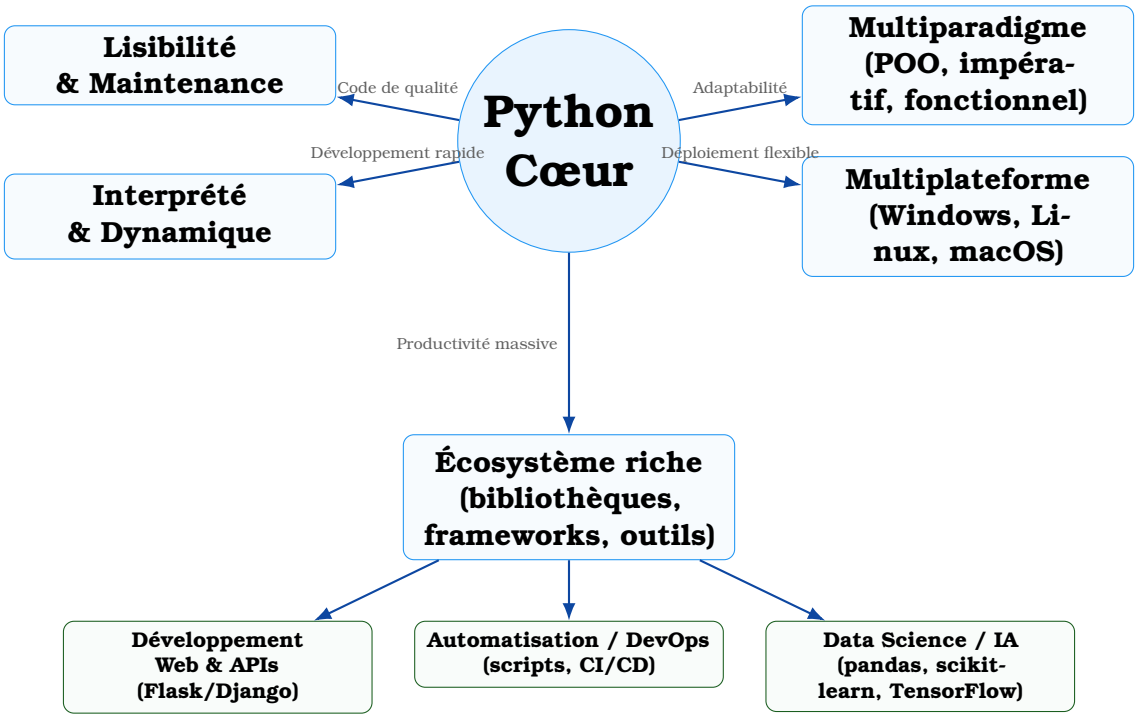


Figure 1.2 – *Carte conceptuelle des caractéristiques de Python et leurs bénéfices directs en IT.*

1.1.3 Chronologie : versions, maturité et rôle en IT

L'évolution de Python reflète sa capacité à s'adapter aux besoins croissants de l'industrie informatique. Comprendre cette chronologie permet d'apprécier sa robustesse actuelle.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

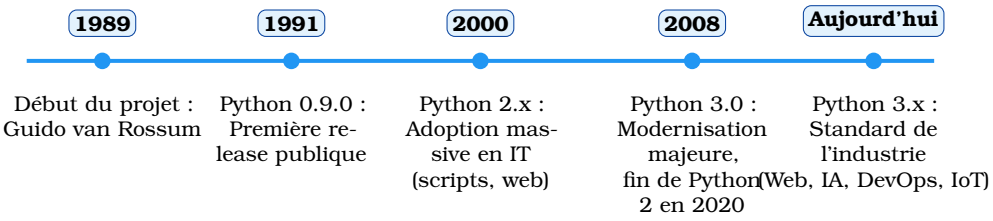


Figure 1.3 – *Chronologie simplifiée : Python, de son origine à son rôle de standard IT.*

1.2 Installation et gestion des packages Python

L'installation de Python et la gestion efficace de ses bibliothèques (aussi appelées "packages" ou "modules") sont des étapes fondamentales. Une configuration propre et bien organisée est la première pierre angulaire de tout projet IT réussi. Elle garantit la stabilité, la reproductibilité de vos développements et évite les "conflits de dépendances" qui peuvent rapidement devenir un casse-tête.

1.2.1 Téléchargement et installation de Python

Choix recommandé

Pour un étudiant en **Bachelor IT**, deux voies sont réalistes :

- **Anaconda/Miniconda** : recommandé si vous visez **Data/ML** (dépendances compilées, environnements conda).
- **Python officiel** : idéal pour **scripts, web, automatisation** (léger, standard).

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Option A — Anaconda / Miniconda

Profil : Data Science, ML, notebooks
Inclus : `conda`, Jupyter, libs scientifiques

Points forts :

- gestion d'environnements robuste
- dépendances natives mieux gérées
- démarrage rapide pour data/IA

Téléchargement : https://www.anaconda.com/download

Option B — Python officiel

Profil : IT général (scripts, web, automation)
Inclus : `pip` (PyPI), `venv`

Points forts :

- installation légère et standard
- excellent pour projets web/API
- workflow pro via `venv` + `pip`

Téléchargement : https://www.python.org/downloads/

Figure 1.4 – *Deux chemins d'installation : choisissez selon le type de projets IT visés.*

Table 1.1 – *Comparatif rapide : Anaconda/Miniconda vs Python officiel*

Critère	Anaconda/Miniconda	Python officiel
Gestion d'environnements	`conda` (intégré)	`venv` (standard)
Installation	plus lourde (Anaconda), légère (Miniconda)	légère
Dépendances compilées (Data/ML)	souvent plus simple	parfois plus délicat
Usage IT typique	data/IA, notebooks, calcul	scripts, web, automation, CLI
Reproductibilité	`environment.yml`	`requirements.txt`

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Étapes d'installation (quel que soit le choix)

1. Télécharger l'installeur (Anaconda/Miniconda ou Python officiel).
2. Lancer l'installation et suivre l'assistant.
3. **Étape critique (Windows) :** ajouter Python au **PATH** (ou enregistrer Python par défaut).
4. Vérifier dans un terminal : `python -version`.

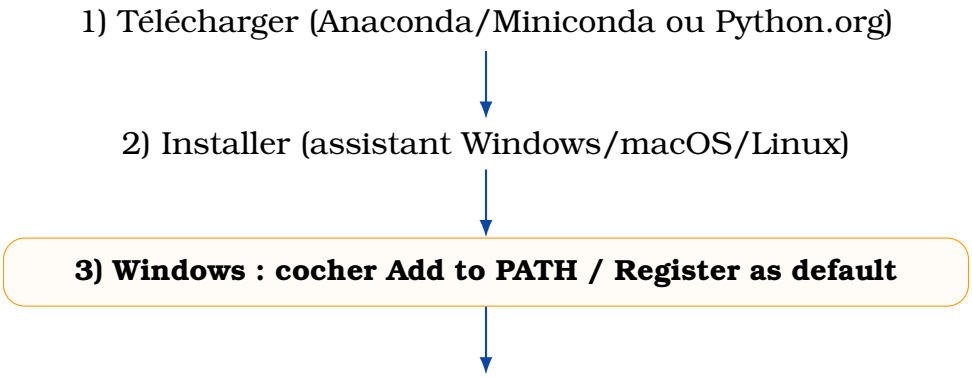


Figure 1.5 – *Workflow recommandé : installation + étape critique PATH + vérification.*

1.2.1.1 Vérification : confirmer que l'environnement est prêt

Après l'installation, vérifiez que les commandes sont accessibles depuis un terminal.

Terminal

```
python --version
python -c "import sys; print(sys.executable)"
pip --version
# Si Anaconda/Miniconda :
conda --version
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Interprétation rapide

- Si `python` ou `pip` est introuvable : problème de **PATH** ou d'environnement non activé.
- `sys.executable` indique le **Python réellement utilisé** (diagnostic n°1 en cas de confusion IDE/terminal).

1.2 .2 Environnements virtuels et packages : robustesse et reproductibilité

Environnement virtuel

Un **environnement virtuel** est un espace isolé contenant un Python et des bibliothèques propres à un projet. L'activation redirige `python` et `pip` vers cet espace, évitant de « polluer » l'installation globale.

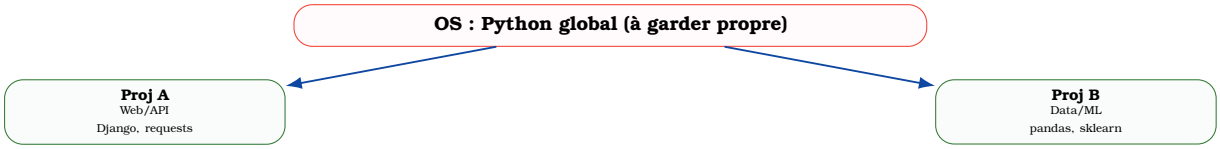


Figure 1.6 – *Un projet = un environnement.*

Bénéfices IT

- **Isolation** : pas de conflits de versions.
- **Reproductibilité** : recréation sur une autre machine.
- **Déploiement** : base propre CI/CD.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

1.2.2 .1 `pip` vs `conda` : choisir comme un ingénieur IT

Table 1.2 – *`pip` et `conda` : rôle et limites*

Outil	Points forts	Limites / vigilance
`pip`	Standard Python, énorme catalogue (PyPI)	gère surtout le Python ; dépendances natives parfois difficiles
`conda`	gère aussi binaires/dépendances natives ; environnements intégrés	plus « écosystème » conda ; mélange pip/conda à maîtriser

//

Terminal

```
python -m pip install requests
python -m pip install --upgrade requests
python -m pip uninstall requests
python -m pip list
python -m pip freeze > requirements.txt
python -m pip install -r requirements.txt
```

//

Terminal

```
conda create -n dev_web python=3.11
conda activate dev_web
conda install flask numpy
conda env export > environment.yml
conda env create -f environment.yml
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

⚠ Débogage : `ModuleNotFoundError`

Cette erreur signifie : « le module n'est pas installé dans **l'environnement** qui exécute votre script ».
Méthode en 3 étapes :

1. Identifier le Python courant : `python -c "import sys; print(sys.executable)"`.
2. Installer dans ce Python : `python -m pip install nom_du_module`.
3. Relancer (et redémarrer l'IDE si nécessaire).

1.3 Découverte du REPL et premières commandes : Votre premier laboratoire Python

Le REPL (Read-Eval-Print Loop), ou interpréteur interactif Python, est votre premier outil pour dialoguer directement avec Python. Imaginez-le comme une calculatrice super-puissante où vous pouvez taper des instructions Python et obtenir un résultat immédiat. C'est l'endroit parfait pour expérimenter et tester vos premières lignes de code.

1.3.1 La console interactive Python (REPL) : Testez vos idées instantanément

Le REPL fonctionne sur un cycle simple et rapide : il **lit** ce que vous tapez, l'**évalue** (l'exécute), **affiche** le résultat, puis **boucle** pour la prochaine instruction.

Pour lancer le REPL, ouvrez un terminal (ou l'invite de commande sur Windows) et tapez `python` (ou `python3` si `python` est une version plus ancienne). Vous verrez apparaître le prompt `>>>`.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Terminal

```
python
>>> print("Bonjour, Python !") # Un message simple
Bonjour, Python !

>>> 5 + 3 # Un calcul arithmétique de base
8

>>> # Un calcul informatique simple : combien de kilooctets
    dans un mégaoctet ?
>>> 1024 * 1024
1048576

>>> "Python est " + "facile !" # On peut "coller" des textes
'Python est facile !'

>>> exit() # Pour quitter le REPL (ou Ctrl+D sur
    Linux/macOS, Ctrl+Z puis Entrée sur Windows)
```

Pourquoi le REPL est utile pour commencer ?

Le REPL est parfait pour :

- **Apprendre vite :** Tester des commandes Python sans avoir à créer un fichier.
- **Expérimenter :** Voir immédiatement le résultat de vos instructions.
- **Découvrir :** Explorer de nouvelles fonctions ou opérateurs.

1.3.2 Fonctions d'interaction de base : `print()` et `input()`

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Pour qu'un programme soit utile, il doit pouvoir communiquer. Les fonctions `print()` et `input()` sont vos premières commandes pour que votre programme "parle" à l'utilisateur et "écoute" ce qu'il a à dire.

1.3.2.1 `print()` : Votre programme s'exprime

La fonction `print()` est la façon dont votre programme affiche du texte, des nombres, ou des résultats sur la console. C'est essentiel pour donner des informations à l'utilisateur ou pour vérifier ce que fait votre code.

In [1]:
```
## 1. Afficher un message de bienvenue
print("Bienvenue au cours de Python !")

# 2. Afficher le résultat d'un calcul
nombre_pommes = 10
nombre_oranges = 5
print("J'ai", nombre_pommes + nombre_oranges, "fruits.")

# 3. Utiliser les f-strings pour un message clair (méthode moderne)
# Les f-strings (f"...") sont pratiques pour insérer des variables directement dans le texte.
nom_utilisateur = "Alice"
print(f"Bonjour, {nom_utilisateur} ! Ravi de te voir.")
```

Out [1]:
```
Bienvenue au cours de Python !
J'ai 15 fruits !
Bonjour, Alice ! Ravi de te voir.
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

L'astuce des f-strings

Les **f-strings** (chaînes de caractères formatées, préfixées par `f` comme `f"Mon texte ma_variable"`) sont la méthode la plus simple et moderne pour afficher des messages combinant du texte et des variables. Adoptez-les dès maintenant !

1.3.2 .2 `input()` : Votre programme écoute l'utilisateur

La fonction `input()` permet à votre programme de demander une information à l'utilisateur et d'attendre sa réponse. La réponse est ensuite stockée dans une variable.

In [2]:
```
## 1. Demander le nom de l'utilisateur
prenom = input("Quel est votre prénom ? ")
print(f"Bonjour, {prenom} !")

# 2. Demander une information numérique (attention au type !)
annee_naissance_str = input("En quelle année êtes-vous né(e) ? ")
# La fonction input() renvoie TOUJOURS du texte (une "chaîne de caractères").
# Pour faire des calculs, il faut convertir ce texte en nombre.
annee_naissance_int = int(annee_naissance_str)  # Convertit le texte en nombre entier
age_approx = 2024 - annee_naissance_int         # Exemple de calcul simple
print(f"Vous avez environ {age_approx} ans.")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
Out[2]: Quel est votre prénom ? Léa
        Bonjour, Léa !
        En quelle année êtes-vous né(e) ? 2003
        Vous avez environ 21 ans.
```

⚠ Attention au type de données avec `input()`

`input()` renvoie toujours une **chaîne de caractères** (du texte). Si vous attendez un nombre pour faire des calculs, vous devez le **convertir explicitement** en utilisant `int()` (pour les nombres entiers) ou `float()` (pour les nombres à virgule). Si l'utilisateur tape du texte non numérique pour une conversion, votre programme produira une erreur (`ValueError`). Nous verrons comment gérer ces erreurs plus tard.

1.4 Environnements de développement (IDEs) : Votre poste de travail Python complet

Bien que le REPL soit excellent pour l'expérimentation rapide, l'écriture et la gestion de projets Python plus complexes (applications web, scripts d'automatisation avancés, systèmes d'analyse de données) nécessitent un outil plus sophistiqué : un **IDE** (Integrated Development Environment - Environnement de Développement Intégré).

Un IDE regroupe en une seule interface des outils essentiels pour toutes les phases du développement logiciel, augmentant considérablement votre productivité et la qualité de votre code.

1.4.1 Fonctionnalités clés d'un IDE et leur utilité en IT

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Un bon IDE Python offre un ensemble de fonctionnalités qui transforment votre expérience de codage :

- **Éditeur de code intelligent :**
  - **Coloration syntaxique :** Met en évidence les mots-clés, chaînes, commentaires. Facilite la lecture du code.
  - **Autocomplétion (IntelliSense) :** Suggère les noms de variables, fonctions, classes et méthodes. Réduit les erreurs de frappe et accélère l'écriture du code.
  - **Vérification d'erreurs en temps réel (Linting) :** Souligne les erreurs de syntaxe ou les avertissements de style pendant que vous écrivez. Permet de corriger les problèmes avant même d'exécuter le code.
  - **Refactoring :** Outils pour renommer des variables/fonctions à travers tout le projet, extraire du code en fonctions, etc. Maintient la qualité du code et la structure du projet.

  Utilité IT : Accélère le développement, réduit les bugs, et améliore la maintenabilité du code, des points cruciaux pour la livraison rapide et la stabilité des applications.
- **Débogueur intégré : Utilité IT :** Permet d'exécuter le code pas à pas, de définir des points d'arrêt (*breakpoints*) pour inspecter l'état des variables à un moment précis, et de détecter les erreurs logiques. Indispensable pour trouver et corriger des bugs complexes dans des systèmes IT.
- **Gestion de projet et de fichiers : Utilité IT :** Organise vos fichiers, dossiers, dépendances et environnements virtuels. Simplifie la navigation dans de grands projets et l'intégration avec les systèmes de contrôle de version (Git).
- **Console interactive intégrée : Utilité IT :** Combinaison des avantages du REPL avec la gestion du projet. Permet de tester des extraits de code dans le contexte de votre application.
- **Intégration de Terminal : Utilité IT :** Accès direct à la ligne de commande pour exécuter des scripts Python, gérer les paquets avec `pip`/`conda` ou utiliser Git, sans quitter l'IDE.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

1.4 .2 Les IDEs Python populaires pour les étudiants en IT

Plusieurs IDEs sont populaires et adaptés au développement Python. Votre choix dépendra de vos préférences, du type de projet et des ressources de votre machine.

1.4.2 .1 PyCharm : L'IDE de référence pour le développement Python professionnel

PyCharm

Développé par JetBrains, PyCharm est un IDE puissant et très apprécié, particulièrement adapté aux projets Python de grande envergure et aux équipes professionnelles. Il existe en version gratuite (Community) et payante (Professional, avec des fonctionnalités avancées pour le web, la data science, etc.).

Points forts pour l'IT :

- **Analyse de code intelligente :** Détecte les erreurs, les problèmes de style (PEP 8) et les vulnérabilités potentielles.
- **Débogueur avancé :** Un des meilleurs débogueurs du marché pour Python, essentiel pour les applications complexes.
- **Intégration poussée :** Support natif pour les frameworks web (Django, Flask), les bases de données, Docker, Git.
- **Environnements virtuels :** Gestion fluide de `venv` et `conda`.

Installation : Téléchargez la version Community sur https://www.jetbrains.com/pycharm/download/. L'assistant d'installation est intuitif et vous guidera pour configurer votre interpréteur Python.

1.4.2 .2 VS Code (Visual Studio Code) : La flexibilité au service du développeur Python

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

VS Code

Visual Studio Code de Microsoft est un éditeur de code open source, léger et multiplateforme. Sa force réside dans son écosystème d'extensions très riche qui lui permet de se transformer en un IDE complet pour Python (et bien d'autres langages). Il est extrêmement populaire pour sa rapidité, sa flexibilité et sa grande personnalisation.

Points forts pour l'IT :

- **Léger et rapide :** Démarrage instantané, faible consommation de ressources.
- **Extensions puissantes :** L'extension officielle "Python" de Microsoft (avec Pylance pour l'autocomplétion) offre une expérience IDE complète.
- **Polyvalence :** Idéal pour les scripts d'automatisation, le développement web (front-end/back-end), la gestion de configuration (DevOps), et l'analyse de logs.
- **Intégration Git :** Support natif et intuitif pour le contrôle de version.

Installation :

1. Téléchargez VS Code sur `https://code.visualstudio.com/`.
2. Installez l'extension officielle "Python" de Microsoft via le gestionnaire d'extensions intégré (accessible via Ctrl+Shift+X).

1.4.2 .3 Jupyter Notebook / JupyterLab : L'environnement interactif pour la Data Science et l'analyse

Jupyter Notebook est un environnement de développement web interactif qui permet de créer des documents appelés "notebooks". Ces notebooks sont des documents "vivants" qui mélangent :

- Du code exécutable (Python, R, Julia).

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

- Du texte formaté (Markdown), des équations (LaTeX).
- Des visualisations (graphiques, tableaux) et des médias.

JupyterLab est une version plus avancée et modulable de Jupyter Notebook, offrant une interface plus complète avec des onglets, un explorateur de fichiers et la possibilité d'ouvrir différents types de documents.

Points forts pour l'IT (Data Science Recherche) :

- **Interactivité par cellule :** Exécutez le code cellule par cellule, parfait pour l'exploration de données, l'expérimentation et le prototypage rapide d'algorithmes.
- **Affichage Riche :** Affiche directement les graphiques (Matplotlib, Seaborn), les tableaux (Pandas) et les résultats de calculs inline, idéal pour l'analyse exploratoire des données.
- **Documentation Intégrée et Reproductible :** Combinez du code avec des explications textuelles en Markdown, créant des rapports techniques interactifs et reproductibles.
- **Partage Facile :** Les fichiers `.ipynb` sont faciles à partager et peuvent être ouverts et exécutés par d'autres utilisateurs via Jupyter ou des plateformes comme Google Colab.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

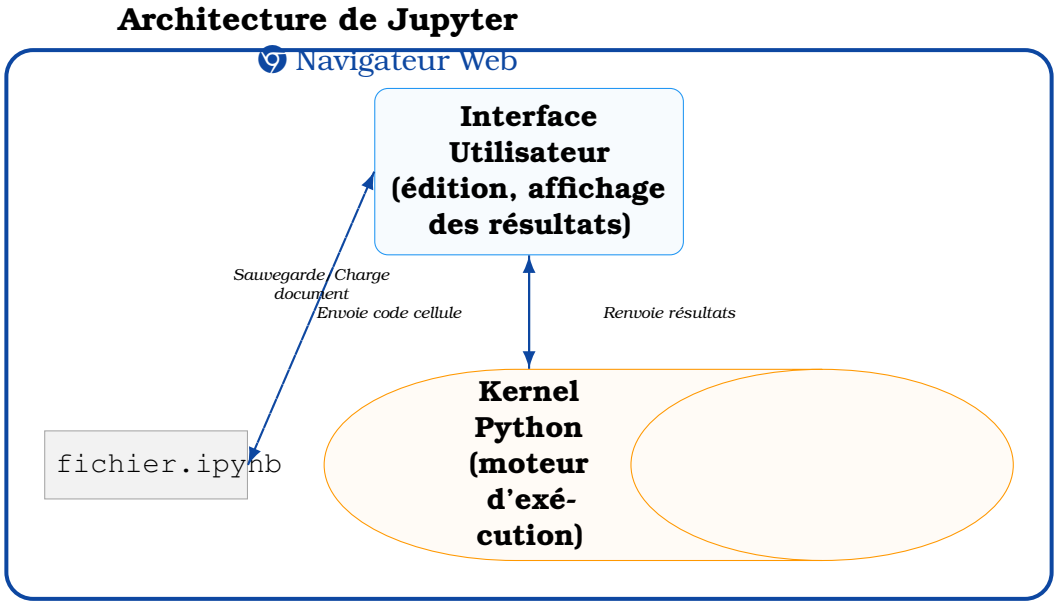


Figure 1.7 – *Fonctionnement interactif de Jupyter Notebook/Lab, idéal pour l'analyse de données et la recherche IT.*

Installation et Lancement :

- **Si Anaconda est installé :** Jupyter Notebook et JupyterLab sont généralement déjà inclus. Lancez-les depuis le "Anaconda Navigator" ou via votre terminal :

```
Terminal
jupyter notebook
# ou pour JupyterLab
jupyter lab
```

- **Si Python officiel est installé (avec pip) :**

```
Terminal
pip install notebook
# ou pour l'interface plus riche de JupyterLab
pip install jupyterlab
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Puis lancez avec `jupyter notebook` ou `jupyter lab`.

Table 1.3 – *Comparatif des environnements de développement Python pour les professionnels de l'IT*

Environnement	Caractéristiques Principales	Scénarios d'Utilisation Idéaux en IT
REPL (Console)	Exécution immédiate, ligne par ligne, retour instantané.	Tests rapides de syntaxe/fonctions, exploration d'APIs, debugging de snippets.
Jupyter Notebook / JupyterLab	Interactif, mélange code/texte/graphiques, exécution par cellule.	Analyse exploratoire de données, rapports d'audit interactifs, prototypage ML/IA, tutoriels.
PyCharm	IDE complet : débogueur avancé, refactoring, gestion de projet, intégration Git.	Développement d'applications web (Django/Flask), microservices, backend complexes, grands projets d'équipe.
VS Code	Éditeur léger et extensible : extensions riches, support multi-langages, terminal intégré.	Scripts d'automatisation, développement web (front/back), gestion de configuration (DevOps), analyse de logs.
Spyder	Orienté scientifique : explorateur de variables, console IPython, éditeur Matplotlib.	Data Science traditionnelle, analyse statistique, simulations (ingénierie, finance), bioinformatique.
Google Colab	Notebooks basés sur le cloud : GPU/TPU gratuits, partage facile, pas d'installation locale.	Projets Deep Learning collaboratifs, prototypage rapide d'IA, apprentissage à distance sans infrastructure matérielle.

1.5 Structure d'un programme Python : Organiser votre premier code

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Écrire du code ne se résume pas à taper des instructions ; il s'agit aussi de les organiser de manière logique pour qu'elles soient comprises par la machine ET par les humains. En IT, un programme bien structuré est la clé de la **maintenabilité** et de la **collaboration**.

1.5.1 L'Indentation : La règle d'or de Python

En Python, l'**indentation** n'est pas juste pour la beauté du code, c'est une **règle de syntaxe** ! Elle définit les blocs de code. Imaginez-la comme les paragraphes d'un texte : chaque nouveau paragraphe (bloc de code) commence avec une indentation.

IndentationError : L'erreur fréquente du débutant

Une `IndentationError` survient si votre code n'est pas indenté correctement (par exemple, un mélange d'espaces et de tabulations, ou une indentation incohérente). L'usage recommandé est de **4 espaces** par niveau d'indentation. Votre IDE configurera cela automatiquement.

In [3]:

```
## Un code simple pour illustrer l'indentation
serveur_actif = True

if serveur_actif:  # Si le serveur est actif...
    print("Le serveur est opérationnel.")  # ... alors exécute cette ligne (indentée)
    print("Vérification en cours...")      # ... et celle-ci (même indentation)

print("Fin de la vérification.")           # Cette ligne s'exécute toujours, elle n'est pas indentée
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
Out[3]:
Le serveur est opérationnel.
Vérification en cours...
Fin de la vérification.
```

Utilité IT de l'indentation : L'indentation obligatoire rend le code Python naturellement lisible, réduisant les erreurs de logique et facilitant la compréhension mutuelle au sein d'une équipe IT.

1.5 .2 Les commentaires : Votre code parle aux humains

Les commentaires sont des notes dans votre code destinées aux humains (vous, et vos collègues !). L'interpréteur Python les ignore. Ils sont essentiels pour expliquer ce que fait une partie complexe du code, pourquoi un choix a été fait, ou pour laisser une note importante.

```
In [4]:
## Ceci est un commentaire sur une seule ligne.
# Il explique la ligne de code suivante.

# Déclaration d'une variable pour stocker le nom de
    l'application
nom_application = "GestionRH"

# Affichage d'un message de démarrage de l'application
print(f"Démarrage de l'application : {nom_application}")

# Les commentaires peuvent aussi être placés après une
    instruction
version = "1.0.0"  # Numéro de version actuel
print(f"Version : {version}")
```

```
Out[4]:
Démarrage de l'application : GestionRH
Version : 1.0.0
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

L'art du commentaire efficace en IT

- **Expliquez le "pourquoi", pas le "quoi" :** Le code explique souvent le "quoi". Le commentaire doit expliquer le "pourquoi" (ex : "Pourquoi utiliser cette technique plutôt qu'une autre ?").
- **Soyez concis et pertinent :** Des commentaires trop longs ou évidents sont contre-productifs.

Utilité IT : Les commentaires sont cruciaux pour la **maintenabilité** du code et le **partage des connaissances** dans les équipes IT. Ils aident les nouveaux membres à comprendre rapidement le projet.

1.5.3 Point d'entrée d'un script : Votre programme commence ici

Pour des scripts simples écrits dans un seul fichier, Python exécute le code de haut en bas. Cependant, une bonne pratique existe pour définir explicitement le "point de départ" de votre script. Ce n'est pas une obligation pour les scripts très simples, mais c'est une convention professionnelle que vous verrez souvent.

mon_premier_script.py

```
# mon_premier_script.py

# 1. Zone de définition (fonctions, variables globales, etc.)
message_bienvenue = "Bienvenue dans votre premier script Python !"

def afficher_infos_systeme():
    # Cette fonction sera implémentée plus tard pour afficher des infos système
    return "Informations système fictives."
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
9
10 # 2. Point d'entrée principal du script
11 # Ce bloc s'exécute UNIQUEMENT si le script est lancé
      directement
12 if __name__ == "__main__":
13     print(message_bienvenue)
14     infos = afficher_infos_systeme()
15     print(infos)
16     print("Script terminé.")
```

Utilité IT de ce bloc (`if __name__ == "__main__":`) :

- **Clarté structurelle :** Il indique clairement où commence l'exécution principale de votre script.
- **Flexibilité future :** Sans rentrer dans les détails techniques complexes de "modules" pour le moment, sachez que ce bloc permet à votre script d'être potentiellement réutilisé par d'autres programmes sans exécuter sa logique principale.

1.6 Variables et types de données de base : Les fondations de l'information IT

En programmation, manipuler l'information est au cœur de tout ce que vous ferez en IT. Pour cela, vous aurez besoin de deux concepts fondamentaux : les **variables** pour stocker cette information, et les **types de données** pour définir sa nature. Comprendre ces concepts est la première étape pour écrire des programmes qui gèrent des données de manière efficace et sans erreur.

1.6.1 Variables : Conteneurs pour vos données

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Imaginez une variable comme une **boîte nommée** dans la mémoire de votre ordinateur. Vous pouvez y ranger une valeur, et cette valeur peut changer au cours de l'exécution de votre programme.

Variable en programmation

Une **variable** est un emplacement nommé dans la mémoire de l'ordinateur qui permet de stocker une valeur. Son contenu peut être modifié pendant l'exécution d'un programme.

1.6.1 .1 Déclaration et affectation en Python

En Python, la déclaration (créer la boîte) et l'affectation (y mettre une valeur) se font en une seule étape simple, en utilisant l'opérateur `=`. Python est un langage à **typage dynamique** : vous n'avez pas besoin de spécifier le type de la variable (texte, nombre, etc.) à l'avance ; Python le déduit de la valeur que vous lui donnez.

In [5]:

```
## Affectation de différentes valeurs à des variables
nombre_utilisateurs_actifs = 150  # Un nombre entier
nom_application = "Gestion de Stock"  # Une chaîne de
    caractères
est_service_demarre = True  # Une valeur booléenne
    (vrai/faux)

# Affichage des valeurs des variables
print("Utilisateurs actifs:", nombre_utilisateurs_actifs)
print("Application courante:", nom_application)
print("Service démarré:", est_service_demarre)

# La valeur d'une variable peut être modifiée à tout moment
nombre_utilisateurs_actifs = 155
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
13 print("Nouvel affichage des utilisateurs:",
       nombre_utilisateurs_actifs)
```

```
Out[5]:
Utilisateurs actifs: 150
Application courante: Gestion de Stock
Service démarré: True
Nouvel affichage des utilisateurs: 155
```

Règles et bonnes pratiques pour les noms de variables en IT

Choisir des noms de variables **clairs et descriptifs** est crucial pour la lisibilité et la maintenabilité de votre code, surtout quand vous travaillez en équipe sur des projets IT :

- **Descriptifs :** `nom_utilisateur` est bien plus explicite que `n` ou `x`.
- **Minuscules avec underscores (`snake_case`) :** C'est la convention Python (ex : `nombre_elements`, `chemin_fichier_log`).
- **Pas de caractères spéciaux ou d'espaces :** Utilisez _ à la place des espaces.
- **Ne pas commencer par un chiffre :** `1_variable` est invalide, mais `variable_1` est valide.
- **Éviter les mots-clés réservés de Python :** (ex : `print`, `if`, `for`, `class`).

1.6.2 Types de données de base : La nature de l'information

Le **type de données** d'une variable indique la catégorie de valeur qu'elle contient (nombre, texte, vrai/faux, etc.). Bien que Python détermine les types automatiquement, comprendre leur fonctionnement est essentiel pour :

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

- Choisir le bon type pour stocker vos données efficacement.
- Effectuer les opérations correctes sur ces données.
- Éviter les erreurs de type dans vos programmes.

Vous pouvez connaître le type de n'importe quelle variable ou valeur en utilisant la fonction `type()`.

1.6.2.1 Nombres : `int` (entiers) et `float` (flottants)

Les nombres sont le pain quotidien de tout programme IT, que ce soit pour compter des ressources, calculer des performances ou manipuler des données financières.

- **Entiers (`int`) :** Représentent des nombres entiers, positifs ou négatifs, sans partie décimale. **Usages IT :** Compteurs (nombre de requêtes), identifiants (ID utilisateur), ports réseau (80, 443), adresses IP (parties).
- **Flottants (`float`) :** Représentent des nombres réels, avec une partie décimale. **Usages IT :** Mesures précises (température CPU : 65.5°C), calculs de latence réseau (12.345 ms), taux et pourcentages (taux d'erreur : 0.0125).

In [6]:
```
## Exemples de nombres entiers et flottants
nombre_machines_virtuelles = 50
version_os = 10
print(f"Type de 'nombre_machines_virtuelles':
    {type(nombre_machines_virtuelles)}")
print(f"Type de 'version_os': {type(version_os)}")

latence_moyenne_ms = 12.34
taux_utilisation_cpu = 0.855  # 85.5 %
print(f"Type de 'latence_moyenne_ms':
    {type(latence_moyenne_ms)}")
print(f"Type de 'taux_utilisation_cpu':
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
        {type(taux_utilisation_cpu)}")
```

Out[6]:
```
Type de 'nombre_machines_virtuelles': <class 'int'>
Type de 'version_os': <class 'int'>
Type de 'latence_moyenne_ms': <class 'float'>
Type de 'taux_utilisation_cpu': <class 'float'>
```

⚠ Attention à la précision des flottants en IT

Les nombres flottants peuvent parfois avoir de très petites imprécisions dues à la manière dont les ordinateurs stockent les nombres à virgule. Pour la plupart des usages, c'est transparent, mais pour des calculs financiers critiques ou des comparaisons d'égalité strictes, soyez vigilant. Pour une précision décimale exacte, Python offre le module `decimal`.

1.6.2 .2 Chaînes de caractères (`str`) : Manipuler le texte et les informations structurées

Le texte est omniprésent en IT. Les chaînes de caractères (`str`) sont utilisées pour stocker et manipuler toutes sortes d'informations textuelles.

- **Déclaration :** Les chaînes sont délimitées par des guillemets simples (`'`) ou doubles (`""`).
- **Usages IT :** Noms de fichiers, messages de logs, adresses IP, URLs, noms d'utilisateurs, mots de passe (avant hachage !), requêtes SQL.

In [7]:
```
## Déclaration de chaînes de caractères
nom_fichier_log = "access_2024-03-15.log"
message_alerte = "Alerte sécurité: Tentative de connexion
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
    suspecte."
adresse_ip_source = "192.168.1.10"
print(f"Type de 'nom_fichier_log': {type(nom_fichier_log)}")
print(f"Adresse IP source: {adresse_ip_source}")

# Concaténation de chaînes (assembler plusieurs chaînes)
chemin_complet_log = "/var/log/apache/" + nom_fichier_log
print(f"Chemin complet du log: {chemin_complet_log}")

# Longueur d'une chaîne avec len()
print(f"La longueur du message d'alerte est de
    {len(message_alerte)} caractères.")
```

Out[7]:
```
Type de 'nom_fichier_log': <class 'str'>
Adresse IP source: 192.168.1.10
Chemin complet du log: /var/log/apache/access_2024-03-15.log
La longueur du message d'alerte est de 53 caractères.
```

Chaînes et sécurité IT

La manipulation de chaînes provenant d'utilisateurs est une source majeure de vulnérabilités. **Validez et nettoyez toujours les entrées utilisateur** (ex : pour éviter les injections SQL ou les attaques XSS). Ne stockez jamais de mots de passe en clair.

1.6.2 .3 Booléens (`bool`) : La logique Vrai/Faux

Les booléens sont le fondement de la prise de décision en informatique. Une variable booléenne ne peut avoir que deux valeurs : `True` (vrai) ou `False` (faux).

› **Usages IT :** Représenter l'état d'un système (serveur actif ?), le résultat d'une

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

condition (utilisateur authentifié ?), l'activation d'une fonctionnalité (mode maintenance activé ?).

In [8]:
```
## Exemples de variables booléennes
serveur_principal_actif = True
sauvegarde_terminee_avec_erreurs = False
print(f"Type de 'serveur_principal_actif':
    {type(serveur_principal_actif)}")
print(f"Sauvegarde réussie sans erreur: {not
    sauvegarde_terminee_avec_erreurs}")  # Utilisation de
    'not'

# Les comparaisons renvoient des booléens
charge_cpu = 95
seuil_critique = 90
alerte_cpu = charge_cpu > seuil_critique  # La condition
    est-elle vraie ?
print(f"Alerte CPU déclenchée: {alerte_cpu}")
```

Out[8]:
```
Type de 'serveur_principal_actif': <class 'bool'>
Sauvegarde réussie sans erreur: True
Alerte CPU déclenchée: True
```

Logique binaire et informatique

L'informatique est basée sur la logique binaire (0 ou 1, True ou False). Comprendre les booléens est la première étape pour écrire des programmes capables de prendre des décisions complexes basées sur l'état de votre système IT.

1.6.3 Conversion de types (Type Casting) : Adapter les données

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

En IT, les données que vous recevez ne sont pas toujours dans le format idéal. La **conversion de type** (ou *type casting*) est l'opération qui consiste à transformer une valeur d'un type de données vers un autre. C'est indispensable pour manipuler des données provenant de saisies utilisateur, de fichiers ou de bases de données.

In [9]:

```python
## 1. Conversion d'une chaîne de caractères en nombre (le plus fréquent)
port_config_str = "443"  # Port reçu d'un fichier de configuration (c'est du texte)
port_config_int = int(port_config_str)  # On le convertit en entier pour pouvoir faire des calculs ou des comparaisons
print(f"Port en entier : {port_config_int}, type : {type(port_config_int)}")

charge_cpu_str = "85.7"  # Charge CPU lue d'un capteur (texte)
charge_cpu_float = float(charge_cpu_str)  # On le convertit en flottant
print(f"Charge CPU en flottant : {charge_cpu_float}, type : {type(charge_cpu_float)}")

# 2. Conversion d'un nombre en chaîne (pour affichage ou concaténation)
id_transaction = 987654
log_message = "Transaction ID: " + str(id_transaction) + " effectuée avec succès."
print(log_message)

# 3. Conversion en booléen
# int en bool : 0 devient False, tout autre nombre devient True
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```python
# str en bool : chaîne vide ("") devient False, toute autre chaîne devient True
bool_zero = bool(0)
bool_non_zero = bool(50)
bool_empty_str = bool("")
bool_non_empty_str = bool("OK")
print(f"bool(0): {bool_zero}, bool(50): {bool_non_zero}, bool(''): {bool_empty_str}, bool('OK'): {bool_non_empty_str}")
```

Out[9]:
```
Port en entier : 443, type : <class 'int'>
Charge CPU en flottant : 85.7, type : <class 'float'>
Transaction ID: 987654 effectuée avec succès.
bool(0): False, bool(50): True, bool(''): False, bool('OK'): True
```

⚠ Risque de `ValueError` lors de la conversion : L'importance du c

Une conversion de type échoue si la valeur ne peut pas être transformée dans le type cible. Par exemple, `int("bonjour")` provoquera une `ValueError` car "bonjour" n'est pas un nombre entier valide. En IT, c'est un point critique, surtout lors de la lecture de saisies utilisateur ou de données provenant de sources externes. La gestion de ces erreurs (via les blocs `try-except`) est essentielle pour rendre vos programmes robustes et tolérants aux pannes, et sera abordée dans un chapitre ultérieur.

1.7 Les Opérateurs en Python : Manipuler les données

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Les opérateurs sont les outils qui permettent de **calculer**, **comparer** et **combiner** les données dans vos programmes. Maîtriser les opérateurs de base est essentiel pour écrire des programmes fonctionnels.

Opérateur

Un **opérateur** est un symbole qui effectue une opération sur une ou plusieurs valeurs appelées **opérandes** et produit un résultat.
Exemple : Dans `5 + 3`, le `+` est l'opérateur, `5` et `3` sont les opérandes, et `8` est le résultat.

1.7.1 Opérateurs Arithmétiques

Les opérateurs arithmétiques effectuent des **calculs mathématiques** sur des nombres.

Table 1.4 – *Opérateurs arithmétiques en Python*

Op.	Nom	Exemple	Résultat	Description
`+`	Addition	`7 + 3`	`10`	Somme
`-`	Soustraction	`7 - 3`	`4`	Différence
`*`	Multiplication	`7 * 3`	`21`	Produit
`/`	Division	`7 / 2`	`3.5`	Résultat décimal
`//`	Division entière	`7 // 2`	`3`	Partie entière
`%`	Modulo	`7 % 3`	`1`	Reste de la division
`**`	Puissance	`2 ** 3`	`8`	$2^3 = 8$

In [10]:
```
## Opérateurs arithmétiques de base
a, b = 17, 5

print(f"a = {a}, b = {b}")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```python
print(f"Addition       : a + b  = {a + b}")
print(f"Soustraction   : a - b  = {a - b}")
print(f"Multiplication: a * b  = {a * b}")
print(f"Division       : a / b  = {a / b}")
print(f"Div. entière   : a // b = {a // b}")
print(f"Modulo         : a % b  = {a % b}")
print(f"Puissance      : a ** 2 = {a ** 2}")
```

Out[10]:
```
a = 17, b = 5
Addition : a + b = 22
Soustraction : a - b = 12
Multiplication: a * b = 85
Division : a / b = 3.4
Div. entière : a // b = 3
Modulo : a
Puissance : a ** 2 = 289
```

Division entière et modulo

Ces deux opérateurs sont très utiles en programmation :

- `//` donne le **quotient** (nombre de fois que le diviseur "entre" dans le dividende)
- `%` donne le **reste** de la division

Exemple : $17 \div 5 = 3$ reste 2 donc `17 // 5 = 3` et `17 % 5 = 2`
Application : Tester si un nombre est pair : `nombre % 2 == 0`

1.7.2 Opérateurs de Comparaison

Les opérateurs de comparaison comparent deux valeurs et retournent un **booléen** (`True` ou `False`). Ils sont essentiels pour les conditions.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Table 1.5 – *Opérateurs de comparaison en Python*

Op.	Signification	Exemple	Résultat
==	Égal à	`5 == 5`	`True`
!=	Différent de	`5 != 3`	`True`
<	Inférieur à	`3 < 5`	`True`
>	Supérieur à	`5 > 3`	`True`
<=	Inférieur ou égal	`5 <= 5`	`True`
>=	Supérieur ou égal	`5 >= 3`	`True`

In [11]:
```python
## Comparaisons de base
x, y = 10, 20

print(f"x = {x}, y = {y}")
print(f"x == y : {x == y}")    # Égal ?
print(f"x != y : {x != y}")    # Différent ?
print(f"x < y  : {x < y}")     # Inférieur ?
print(f"x > y  : {x > y}")     # Supérieur ?
print(f"x <= y : {x <= y}")    # Inférieur ou égal ?
```

Out[11]:
```
x = 10, y = 20
x == y : False
x != y : True
x < y : True
x > y : False
x <= y : True
```

⚠ Ne confondez pas = et ==

- = est l'**affectation** : `x = 5` (donne la valeur 5 à x)
- == est la **comparaison** : `x == 5` (vérifie si x vaut 5)

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

C'est une erreur très fréquente chez les débutants !

Comparaisons chaînées

Python permet d'écrire des comparaisons comme en mathématiques :

```
In [12]:
1 age = 25
2 # Vérifier si age est entre 18 et 65
3 print(18 <= age <= 65)  # Plus lisible que : age >= 18
      and age <= 65
```

```
Out[12]:
True
```

1.7.3 Opérateurs d'Affectation

Les opérateurs d'affectation servent à **assigner** des valeurs aux variables. Les opérateurs **composés** combinent une opération avec l'affectation.

Table 1.6 – *Opérateurs d'affectation courants*

Op.	Description	Exemple	Équivalent à
`=`	Affectation simple	`x = 5`	–
`+=`	Ajouter et affecter	`x += 3`	`x = x + 3`
`-=`	Soustraire et affecter	`x -= 3`	`x = x - 3`
`*=`	Multiplier et affecter	`x *= 2`	`x = x * 2`
`/=`	Diviser et affecter	`x /= 2`	`x = x / 2`

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [13]:
```python
## Opérateurs d'affectation composés
compteur = 10
print(f"Valeur initiale : {compteur}")

compteur += 5    # Ajoute 5
print(f"Après += 5 : {compteur}")

compteur *= 2    # Multiplie par 2
print(f"Après *= 2 : {compteur}")

compteur -= 10   # Soustrait 10
print(f"Après -= 10 : {compteur}")
```

Out [13]:
```
Valeur initiale : 10
Après += 5 : 15
Après *= 2 : 30
Après -= 10 : 20
```

Astuce Python : échange de variables

Python permet d'échanger deux variables sans variable temporaire :

In [14]:
```python
a, b = 5, 10
print(f"Avant : a = {a}, b = {b}")

a, b = b, a  # Échange en une ligne !
print(f"Après : a = {a}, b = {b}")
```

Out [14]:
```
Avant : a = 5, b = 10
Après : a = 10, b = 5
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

1.7.4 Opérateurs Logiques

Les opérateurs logiques combinent des expressions booléennes. Ils sont indispensables pour créer des **conditions complexes**.

Table 1.7 – *Opérateurs logiques en Python*

Op.	Description	Exemple	Résultat
`and`	ET : vrai si les deux sont vrais	`True and False`	`False`
`or`	OU : vrai si au moins un est vrai	`True or False`	`True`
`not`	NON : inverse la valeur	`not True`	`False`

A	B	Résultat
V	V	V
V	F	F
F	V	F
F	F	F

A	B	Résultat
V	V	V
V	F	V
F	V	V
F	F	F

not

A	Résultat
V	F
F	V

Figure 1.8 – *Tables de vérité des opérateurs logiques (V = Vrai, F = Faux)*

In [15]:

```python
## Opérateurs logiques
age = 25
a_permis = True

# Peut conduire si majeur ET a le permis
peut_conduire = age >= 18 and a_permis
print(f"Peut conduire : {peut_conduire}")

# Réduction si étudiant OU senior
est_etudiant = True
est_senior = False
a_reduction = est_etudiant or est_senior
print(f"A une réduction : {a_reduction}")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
14
15 # Accès refusé si NON authentifié
16 est_authentifie = False
17 acces_refuse = not est_authentifie
18 print(f"Accès refusé : {acces_refuse}")
```

```
Out[15]:
Peut conduire : True
A une réduction : True
Accès refusé : True
```

1.7.5 Opérateurs d'Appartenance

Les opérateurs `in` et `not in` vérifient si une valeur est **présente dans une séquence** (liste, chaîne, etc.).

Table 1.8 – *Opérateurs d'appartenance*

Op.	Description	Exemple	Résultat
`in`	Est présent dans	`"a" in "chat"`	`True`
`not in`	N'est pas présent	`"z" not in "chat"`	`True`

```
In [16]:
1 ## Vérifier la présence dans une liste
2 fruits = ["pomme", "banane", "orange"]
3 print(f"'pomme' dans fruits : {'pomme' in fruits}")
4 print(f"'kiwi' dans fruits : {'kiwi' in fruits}")
5
6 # Vérifier la présence dans une chaîne
7 message = "Bonjour Python"
8 print(f"'Python' dans message : {'Python' in message}")
9 print(f"'Java' dans message : {'Java' in message}")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Out[16]:
```
'pomme' dans fruits : True
'kiwi' dans fruits : False
'Python' dans message : True
'Java' dans message : False
```

1.7.6 Priorité des Opérateurs

Quand une expression contient plusieurs opérateurs, Python les évalue selon un **ordre de priorité** (comme en mathématiques).

Table 1.9 – *Priorité des opérateurs (du plus au moins prioritaire)*

Priorité	Opérateurs	Description
1 (haute)	`()`	Parenthèses
2	`**`	Puissance
3	`* / // %`	Multiplication, divisions, modulo
4	`+ -`	Addition, soustraction
5	`< > <= >= == !=`	Comparaisons
6	`not`	NON logique
7	`and`	ET logique
8 (basse)	`or`	OU logique

In [17]:
```
## Priorité des opérateurs
resultat = 2 + 3 * 4       # * avant +
print(f"2 + 3 * 4 = {resultat}")

resultat = (2 + 3) * 4     # Parenthèses d'abord
print(f"(2 + 3) * 4 = {resultat}")

resultat = 2 ** 3 * 2      # ** avant *
print(f"2 ** 3 * 2 = {resultat}")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
Out[17]:
    2 + 3 * 4 = 14
    (2 + 3) * 4 = 20
    2 ** 3 * 2 = 16
```

Conseil pratique

En cas de doute sur la priorité, utilisez des **parenthèses** ! Elles rendent votre code plus clair et évitent les erreurs.

```
# Difficile à lire
resultat = a + b * c / d - e

# Plus clair
resultat = a + ((b * c) / d) - e
```

1.8 Travaux Pratiques (TP) du Module 1

Ces travaux pratiques vous permettent d'appliquer les connaissances acquises dans ce module : installation, environnement, variables, types de données et opérateurs.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Travaux Pratiques : Mettre les mains dans le code

TP 1 – Vérification d'environnement et découverte du REPL

Objectif : Vérifier que votre environnement Python est fonctionnel et découvrir l'interpréteur interactif.

Partie A – Vérification de l'installation

1. Ouvrez un terminal (Invite de commandes sur Windows, Terminal sur macOS/Linux).
2. Exécutez les commandes suivantes et notez les résultats :
   - `python -version` (ou `python3 -version`)
   - `pip -version`
3. Si Python n'est pas reconnu, vérifiez qu'il est bien ajouté au PATH.

Partie B – Exploration du REPL

1. Lancez l'interpréteur Python en tapant `python` (ou `python3`).
2. Dans le REPL, effectuez les opérations suivantes :
   - Affichez `"Bonjour, Python !"`
   - Calculez `2025 - 1991` (âge de Python)
   - Calculez `2 ** 10` (valeur de 1 Ko en octets)
   - Testez `import this` pour afficher le Zen de Python
3. Quittez le REPL avec `exit()` ou `Ctrl+D`.

Livrable : Capture d'écran montrant les commandes et résultats.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

TP 2 – Premier script : Variables et interactions utilisateur

Objectif : Créer un script Python structuré utilisant les variables, les types de données et les interactions utilisateur.

Instructions :

1. Ouvrez votre IDE.
2. Créez un fichier nommé `fiche_identite.py`.
3. Écrivez un programme qui :
   - Affiche un message de bienvenue
   - Demande le prénom de l'utilisateur (`input()`)
   - Demande son année de naissance
   - Demande sa ville
   - Calcule son âge approximatif (2024 - année)
   - Affiche une fiche récapitulative avec toutes les informations
4. Utilisez des **f-strings** pour le formatage.
5. Ajoutez des **commentaires** pour expliquer votre code.

Exemple de sortie attendue :

```
=== Bienvenue dans le générateur de fiche d'identité ===
Votre prénom : Avoundoupou
Votre année de naissance : 2005
Votre ville : Lomé
=== Votre fiche ===
Prénom : Mipoupou
Âge approximatif : 20 ans
Ville : Lomé
```

Bonus : Affichez le type de chaque variable avec `type()`.
Livrable : Le fichier `fiche_identite.py` et une capture de son exécution.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

TP 3 – Environnements virtuels et gestion des packages

Objectif : Maîtriser la création d'environnements virtuels et l'installation de packages.

Instructions :

1. Ouvrez un terminal et placez-vous dans un dossier de travail.
2. **Créez un environnement virtuel :**
   - Avec `venv` : `python -m venv mon_env`
   - Ou avec `conda` : `conda create -n mon_env python=3.11`
3. **Activez l'environnement :**
   - Windows : `mon_env\Scripts\activate`
   - macOS/Linux : `source mon_env/bin/activate`
   - Conda : `conda activate mon_env`
4. **Vérifiez l'activation :**
   - `python -c "import sys; print(sys.executable)"`
5. **Installez des packages :**
   - `pip install requests`
   - `pip list` pour voir les packages installés
6. **Exportez les dépendances :**
   - `pip freeze > requirements.txt`
7. **Désactivez l'environnement :**
   - `deactivate` (ou `conda deactivate`)

Livrable : Capture d'écran de toutes les étapes + le fichier `requirements.txt`.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

TP 4 – Calculatrice IT : Opérateurs et conversions

Objectif : Utiliser les opérateurs Python pour des calculs pratiques en IT.

Instructions :
Créez un fichier `calculatrice_it.py` qui réalise les calculs suivants :

Partie A – Calculs de stockage

1. Demandez une taille de fichier en Go à l'utilisateur.
2. Convertissez et affichez cette taille en :
   - Mégaoctets (Mo) : $\times 1024$
   - Kilooctets (Ko) : $\times 1024^2$
   - Octets : $\times 1024^3$
3. Utilisez l'opérateur `**` pour les puissances.

Partie B – Calculs avec modulo

1. Demandez un nombre de secondes à l'utilisateur.
2. Convertissez en heures, minutes et secondes.
3. **Indice :** Utilisez `//` et `%`
4. Exemple : 3725 secondes = 1h 2min 5s

Partie C – Vérifications avec opérateurs logiques

1. Demandez un pourcentage d'utilisation CPU.
2. Affichez :
   - `True` si usage $\geq 80\%$ (alerte)
   - `True` si usage entre 50% et 80% (attention)
   - `True` si usage $< 50\%$ (normal)

Livrable : Le fichier `calculatrice_it.py` et des captures d'exécution.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

★ TP 5 – Projet intégrateur : Analyseur de système

Objectif : Combiner tous les concepts du module dans un projet complet.
Contexte : Créer un outil de diagnostic système simplifié.

Créez le fichier `diagnostic_systeme.py` avec les éléments suivants :

1. **En-tête :** Commentaires (nom, date) + titre affiché
2. **Collecte via `input()` :** Nom machine, CPU (%), Mémoire (%), Disque total (Go), Disque utilisé (Go)
3. **Calculs :** Espace disponible (Go, Mo, Ko), pourcentage utilisation disque
4. **Alertes (opérateurs logiques) :**
   - `alerte_cpu` : True si CPU $\geq$ 90%
   - `alerte_memoire` : True si mémoire $\geq$ 85%
   - `alerte_disque` : True si disque $\geq$ 90%
   - `systeme_critique` : True si $\geq$ 2 alertes
5. **Rapport :** Affichage formaté avec f-strings + état (Sain/Attention/Critique)

Exemple de sortie attendue :

```
=== DIAGNOSTIC SYSTÈME v1.0 ===
Machine : SERVEUR-01
CPU : 75% | Mémoire : 82% | Disque : 65%
Espace disponible : 175 Go (179200 Mo)
Alertes : CPU=False | Mém=False | Disque=False
»> ÉTAT : SAIN «<
```

Bonus : Utilisez `if __name__ == "__main__":`
Livrable : Fichier Python + capture d'écran d'exécution.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

1.9 Quiz du Module 1 : Testez vos connaissances

Vrai/Faux **QCM** **Question ouverte**

1 VRAI/FAUX

Python est un langage compilé, ce qui signifie que le code source est entièrement transformé en code machine avant l'exécution.

☐ **Vrai** ☐ **Faux**

2 QCM

Selon le Zen de Python, quel principe est fondamental pour écrire du bon code ?

A. Le code doit être le plus court possible, même si c'est illisible

B. La lisibilité compte et l'explicite est mieux que l'implicite

C. Les performances sont plus importantes que la clarté

D. Il faut toujours utiliser les fonctionnalités les plus avancées

3 VRAI/FAUX

La fonction `input()` retourne **toujours** une chaîne de caractères (`str`), même si l'utilisateur saisit un nombre.

☐ **Vrai** ☐ **Faux**

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

4 QCM

Quel est l'objectif principal d'un environnement virtuel en Python ?

A. Accélérer l'exécution du code Python

B. Protéger le code source contre les virus

C. Isoler les dépendances d'un projet pour éviter les conflits

D. Permettre l'exécution de code C++ dans Python

5 QCM

Quelle est le type de la variable `x` après l'instruction `x = 3.14` ?

A. `int` **B.** `float` **C.** `str` **D.** `double`

6 VRAI/FAUX

En Python, l'opérateur `/` effectue toujours une division **entière** (sans décimales).

☐ **Vrai** ☐ **Faux**

7 QCM

Quelle est la valeur de l'expression `17 % 5` ?

A. 3 **B.** 2 **C.** 3.4 **D.** 12

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

8 OUVERTE

Citez trois domaines d'application où Python est largement utilisé dans le secteur IT.

1. Developpement de site web
2. Developpement des jeux vidéos
3. Cybersécurité

9 QCM

Quelle est la valeur de l'expression `True and False or True` ?

A. `True` **B.** `False` **C.** Erreur **D.** `None`

10 OUVERTE

Vous obtenez l'erreur : `ModuleNotFoundError: No module named 'requests'`
Expliquez en 2-3 étapes comment résoudre ce problème.

Étape 1 : _____

Étape 2 : _____

Étape 3 : _____

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

CHAPITRE II

STRUCTURES DE CONTRÔLE : LOGIQUE ET AUTOMATISATION

◎ Objectifs d'apprentissage

À l'issue de ce module, vous serez capable de :

- ✓ Comprendre et utiliser les **expressions booléennes** pour la prise de décision.
- ✓ Implémenter des **branchements conditionnels** (`if`, `elif`, `else`).
- ✓ Automatiser des tâches avec les **boucles** `for` et `while`.
- ✓ Contrôler le flux d'exécution avec `break`, `continue` et `pass`.
- ✓ Imbriquer des structures de contrôle pour résoudre des problèmes complexes.

Les structures de contrôle sont le **cœur de la logique** de tout programme. Elles permettent de :

- **Prendre des décisions** : exécuter du code selon des conditions
- **Répéter des actions** : automatiser des tâches répétitives
- **Contrôler le flux** : gérer l'ordre d'exécution des instructions

Les 3 types de flux d'exécution

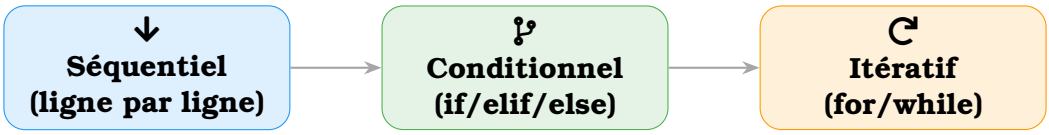


Figure 2.1 – *Les trois paradigmes de flux d'exécution en programmation*

2.1 Expressions Booléennes : La base de la logique

Avant d'aborder les structures conditionnelles, il est essentiel de maîtriser les **expressions booléennes** — des expressions qui s'évaluent à `True` ou `False`.

Expression booléenne

Une **expression booléenne** est une expression qui, une fois évaluée, produit une valeur de type `bool` : soit `True` (vrai), soit `False` (faux).
Ces expressions sont la base de toute **prise de décision** en programmation.

2.1.1 Construction d'expressions booléennes

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Table 2.1 – *Opérateurs pour construire des expressions booléennes*

Catégorie	Opérateurs	Exemple	Résultat
Comparaison	`==, !=, <, >, <=, >=`	`5 > 3`	`True`
Logique	`and, or, not`	`True and False`	`False`
Appartenance	`in, not in`	`"a" in "chat"`	`True`
Identité	`is, is not`	`x is None`	`True/False`

In [1]:

```python
## Exemples d'expressions booléennes
age = 25
salaire = 35000
est_etudiant = True

# Comparaisons simples
print(f"age >= 18 : {age >= 18}")
print(f"salaire > 30000 : {salaire > 30000}")

# Combinaisons avec opérateurs logiques
majeur_et_employe = age >= 18 and salaire > 0
print(f"Majeur ET employé : {majeur_et_employe}")

# Comparaisons chaînées
dans_tranche = 18 <= age <= 65
print(f"Entre 18 et 65 ans : {dans_tranche}")
```

Out[1]:

```
age >= 18 : True
salaire > 30000 : True
Majeur ET employé : True
Entre 18 et 65 ans : True
```

2.1.2 Priorité des opérateurs logiques

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Ordre d'évaluation

Les opérateurs logiques ont un ordre de priorité :

1. `not` (le plus prioritaire)
2. `and`
3. `or` (le moins prioritaire)

Conseil : Utilisez des parenthèses pour clarifier vos intentions !

In [2]:
```
## Priorité des opérateurs
resultat1 = True or False and False        # and avant or
resultat2 = (True or False) and False      # Parenthèses changent l'ordre

print(f"True or False and False = {resultat1}")
print(f"(True or False) and False = {resultat2}")
```

Out[2]:
```
True or False and False = True
(True or False) and False = False
```

2.2 Structures Conditionnelles : Prendre des décisions

Les structures conditionnelles permettent d'exécuter du code **uniquement si** certaines conditions sont remplies.

2.2.1 L'instruction `if` simple

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Instruction `if`

L'instruction `if` exécute un bloc de code **si et seulement si** la condition est vraie (`True`).

Syntaxe :

```
if condition:
    # Code exécuté si condition est True
    instruction1
    instruction2
```

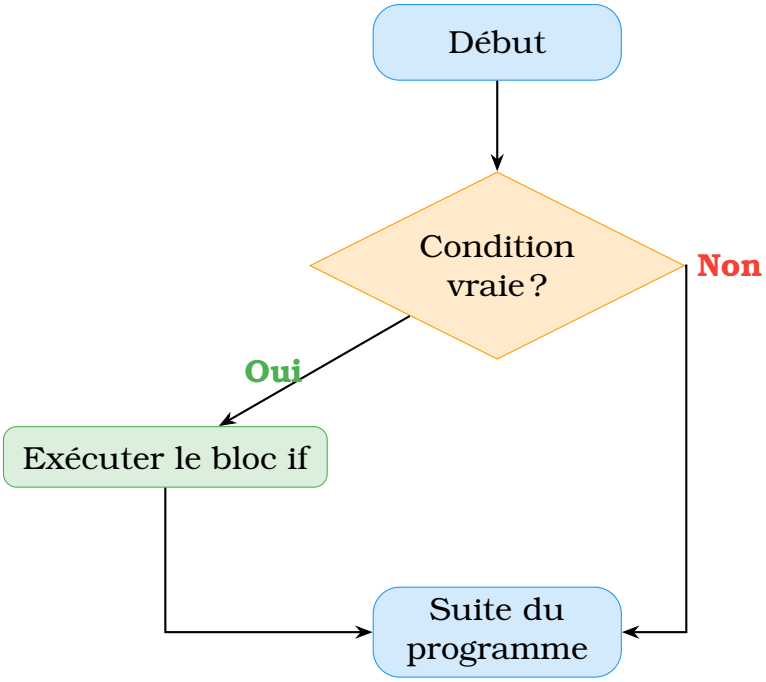


Figure 2.2 – *Organigramme de l'instruction `if` simple*

In [3]:

```
## Exemple : Vérification de l'âge
age = 20

if age >= 18:
    print("Vous êtes majeur.")
    print("Accès autorisé.")

```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
print("Fin du programme.")   # Toujours exécuté
```

Out[3]:
```
Vous êtes majeur.
Accès autorisé.
Fin du programme.
```

⚠ L'indentation est obligatoire !

En Python, le bloc de code sous le `if` **doit être indenté** (4 espaces recommandés). L'indentation définit ce qui appartient au bloc conditionnel.

```
if condition:
    print("Dans le if")     # Indenté = fait partie du if
print("Hors du if")         # Non indenté = toujours exécuté
```

2.2 .2 L'instruction `if-else`

Instruction `if-else`

L'instruction `else` définit un bloc de code à exécuter **si la condition est fausse**.

Syntaxe :

```
if condition:
    # Code si True
else:
    # Code si False
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

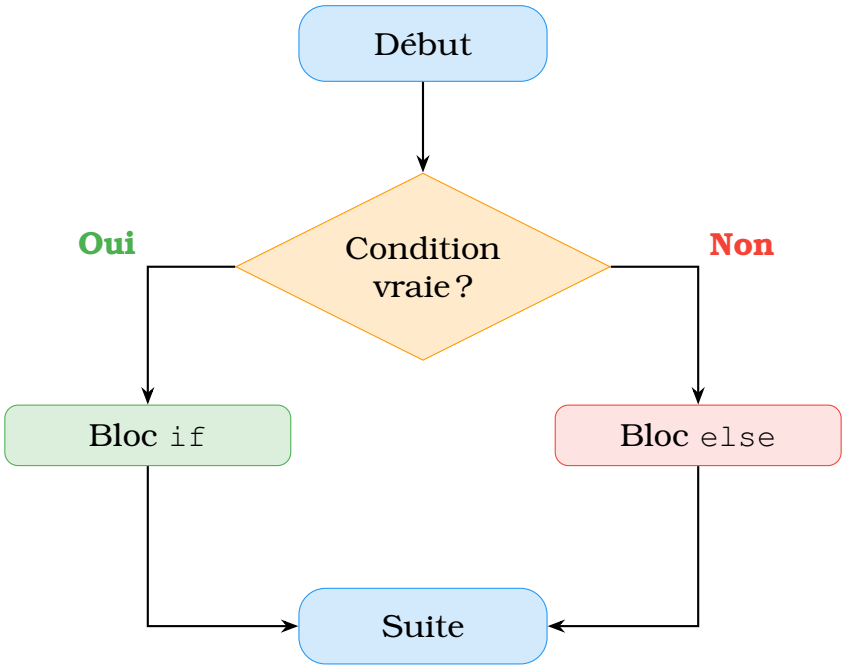


Figure 2.3 – *Organigramme de l'instruction* `if-else`

```
In [4]:
## Exemple : Pair ou impair
nombre = 7

if nombre % 2 == 0:
    print(f"{nombre} est pair.")
else:
    print(f"{nombre} est impair.")
```

```
Out[4]:
7 est impair.
```

2.2.3 L'instruction `if-elif-else`

Pour gérer **plusieurs conditions mutuellement exclusives**, on utilise `elif` (contraction de "else if").

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Instruction `elif`

`elif` permet de tester plusieurs conditions en séquence. La première condition vraie déclenche l'exécution de son bloc, puis on sort de la structure.

Syntaxe :

```
if condition1:
    # Code si condition1 est True
elif condition2:
    # Code si condition2 est True
elif condition3:
    # Code si condition3 est True
else:
    # Code si aucune condition n'est True
```

In [5]:

```
## Exemple : Classification de notes
note = 75

if note >= 90:
    mention = "Excellent"
elif note >= 80:
    mention = "Très bien"
elif note >= 70:
    mention = "Bien"
elif note >= 60:
    mention = "Assez bien"
elif note >= 50:
    mention = "Passable"
else:
    mention = "Insuffisant"

print(f"Note : {note}/100 - Mention : {mention}")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
Out[5]:
Note : 75/100 - Mention : Bien
```

Ordre des conditions

Avec `elif`, l'ordre des conditions est **important** ! Python évalue les conditions de haut en bas et s'arrête à la première qui est vraie.
Conseil : Placez les conditions les plus spécifiques en premier.

2.2.4 Conditions imbriquées

On peut **imbriquer** des structures conditionnelles pour des logiques plus complexes.

```
In [6]:
## Exemple : Système de tarification
age = 25
est_etudiant = True

if age < 18:
    tarif = "Junior"
    prix = 5
elif age >= 65:
    tarif = "Senior"
    prix = 7
else:
    # Adulte : vérifier si étudiant
    if est_etudiant:
        tarif = "Étudiant"
        prix = 8
    else:
        tarif = "Adulte"
        prix = 12
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
19
20 print(f"Tarif : {tarif} - Prix : {prix}€")
```

Out[6]:
```
Tarif : Étudiant - Prix : 8€
```

2.2.5 L'opérateur ternaire

Pour des conditions simples, Python offre une syntaxe condensée sur une ligne.

In [7]:
```
1 ## Opérateur ternaire
2 age = 20
3 statut = "Majeur" if age >= 18 else "Mineur"
4 print(statut)
5
6 # Équivalent à :
7 # if age >= 18:
8 #     statut = "Majeur"
9 # else:
10 #     statut = "Mineur"
```

Out[7]:
```
Majeur
```

2.3 La Boucle `for` : Itérer sur des séquences

La boucle `for` permet de **parcourir** les éléments d'une séquence (liste, chaîne, range, etc.) et d'exécuter un bloc de code pour chaque élément.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Boucle `for`

La boucle `for` itère sur chaque élément d'un **itérable** (objet parcourable).
Syntaxe :

```
for variable in iterable:
    # Code exécuté pour chaque élément
    instruction
```

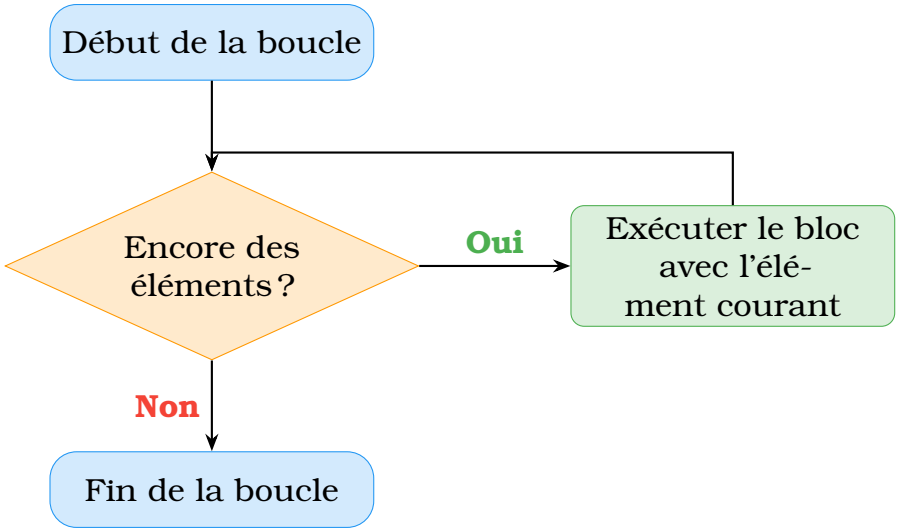


Figure 2.4 – *Organigramme de la boucle* `for`

2.3.1 Parcourir une liste

```
In [8]:
## Parcourir une liste de fruits
fruits = ["pomme", "banane", "orange", "kiwi"]

for fruit in fruits:
    print(f"J'aime les {fruit}s !")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Out [8] :
```
J'aime les pommes !
J'aime les bananes !
J'aime les oranges !
J'aime les kiwis !
```

2.3.2 La fonction `range()`

La fonction `range()` génère une séquence de nombres, très utile pour les boucles avec compteur.

Table 2.2 – *Variantes de* `range()`

Syntaxe	Description	Exemple
`range(n)`	0 à n-1	`range(5)` → 0,1,2,3,4
`range(a, b)`	a à b-1	`range(2,6)` → 2,3,4,5
`range(a, b, pas)`	a à b-1, par pas	`range(0,10,2)` → 0,2,4,6,8

In [9] :
```
## Utilisation de range()
print("Comptage de 0 à 4 :")
for i in range(5):
    print(i, end=" ")

print("\n\nComptage de 1 à 5 :")
for i in range(1, 6):
    print(i, end=" ")

print("\n\nNombres pairs de 0 à 10 :")
for i in range(0, 11, 2):
    print(i, end=" ")

print("\n\nCompte à rebours :")
for i in range(5, 0, -1):
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
    print(i, end=" ")
print("Décollage !")
```

```
Out[9]:
Comptage de 0 à 4 : 0 1 2 3 4
Comptage de 1 à 5 : 1 2 3 4 5
Nombres pairs de 0 à 10 : 0 2 4 6 8 10
Compte à rebours : 5 4 3 2 1 Décollage !
```

2.3.3 Parcourir avec index : `enumerate()`

Pour obtenir à la fois l'index et la valeur, utilisez `enumerate()`.

```
In [10]:
## enumerate() : index + valeur
langages = ["Python", "Java", "C++", "JavaScript"]

for index, langage in enumerate(langages):
    print(f"{index + 1}. {langage}")
```

```
Out[10]:
1. Python
2. Java
3. C++
4. JavaScript
```

2.3.4 Parcourir une chaîne de caractères

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [11]:
```
## Parcourir chaque caractère d'une chaîne
mot = "Python"

for lettre in mot:
    print(lettre, end="-")
```

Out[11]:
```
P-y-t-h-o-n-
```

2.3.5 Boucles `for` imbriquées

In [12]:
```
## Table de multiplication
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i} x {j} = {i*j}", end="  |  ")
    print()  # Nouvelle ligne
```

Out[12]:
```
1 x 1 = 1  |  1 x 2 = 2  |  1 x 3 = 3  |
2 x 1 = 2  |  2 x 2 = 4  |  2 x 3 = 6  |
3 x 1 = 3  |  3 x 2 = 6  |  3 x 3 = 9  |
```

2.4 La Boucle `while` : Répéter tant que...

La boucle `while` exécute un bloc de code **tant qu'une condition est vraie**.

Boucle `while`

La boucle `while` répète un bloc de code tant que la condition reste `True`.
Syntaxe :

```
while condition:
    # Code répété tant que condition est True
    instruction
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
# Modifier quelque chose pour éviter boucle infinie !
```

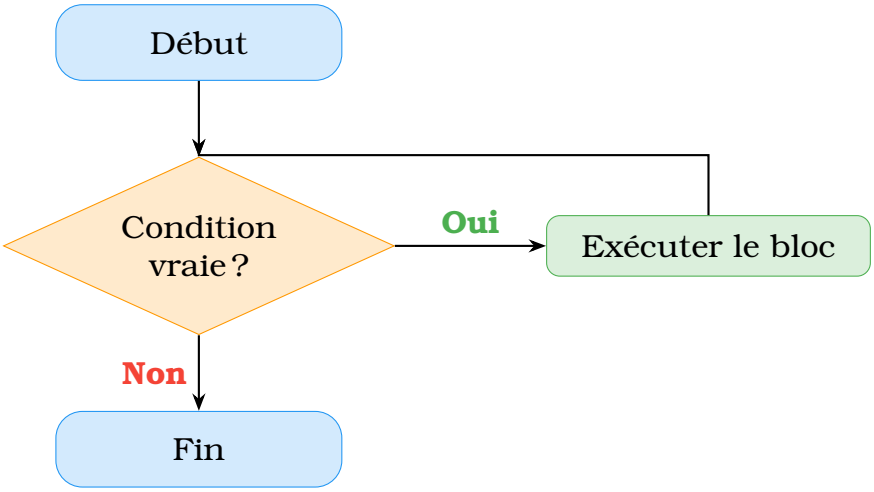


Figure 2.5 – *Organigramme de la boucle* `while`

In [13]:
```
## Exemple simple : compteur
compteur = 1

while compteur <= 5:
    print(f"Tour n{compteur}")
    compteur += 1  # IMPORTANT : modifier le compteur !

print("Boucle terminée.")
```

Out[13]:
```
Tour n°1
Tour n°2
Tour n°3
Tour n°4
Tour n°5
Boucle terminée.
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

⚠ Attention aux boucles infinies !

Si la condition ne devient jamais `False`, la boucle continue indéfiniment !
Toujours s'assurer que :

- La condition peut devenir fausse
- Une variable est modifiée dans la boucle

Pour arrêter une boucle infinie : `Ctrl+C`

2.4.1 Boucle `while` avec saisie utilisateur

In [14]:

```
## Demander un mot de passe jusqu'à réussite
mot_de_passe_correct = "python123"
tentatives = 0
max_tentatives = 3

while tentatives < max_tentatives:
    saisie = input("Mot de passe : ")
    tentatives += 1

    if saisie == mot_de_passe_correct:
        print("Accès autorisé !")
        break  # Sortir de la boucle
    else:
        restantes = max_tentatives - tentatives
        if restantes > 0:
            print(f"Incorrect. {restantes} tentative(s) restante(s).")
        else:
            print("Trop de tentatives. Compte bloqué.")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

2.4.2 Différence entre `for` et `while`

Table 2.3 – *Quand utiliser `for` ou `while` ?*

Critère	**`for`**	**`while`**
Nombre d'itérations	Connu à l'avance	Inconnu à l'avance
Usage typique	Parcourir une séquence	Attendre une condition
Exemple	Liste, range, chaîne	Saisie utilisateur, jeux
Risque de boucle infinie	Faible	Élevé si mal conçu

2.5 Contrôle du Flux : `break`, `continue`, `pass`

Python offre trois instructions pour contrôler le comportement des boucles.

2.5.1 L'instruction `break`

`break` **termine immédiatement** la boucle en cours.

```
In [15]:
## Trouver le premier nombre divisible par 7
for nombre in range(1, 100):
    if nombre % 7 == 0:
        print(f"Premier multiple de 7 : {nombre}")
        break  # Sortie immédiate
```

```
Out[15]:
Premier multiple de 7 : 7
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

2.5.2 L'instruction `continue`

`continue` **passe à l'itération suivante**, en ignorant le reste du bloc.

In [16]:
```
## Afficher uniquement les nombres impairs
for i in range(1, 11):
    if i % 2 == 0:
        continue  # Sauter les pairs
    print(i, end=" ")
```

Out[16]:
```
1 3 5 7 9
```

2.5.3 L'instruction `pass`

`pass` est une instruction **vide** — elle ne fait rien. Utile comme placeholder.

In [17]:
```
## Structure à compléter plus tard
for i in range(5):
    if i == 3:
        pass  # TODO: traitement spécial
    else:
        print(i)
```

Out[17]:
```
0
1
2
4
```

2.5.4 Schéma récapitulatif

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

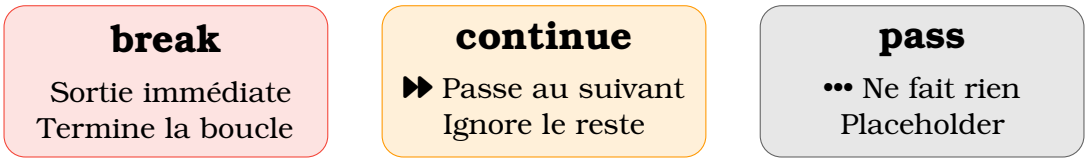


Figure 2.6 – *Les trois instructions de contrôle de flux*

2.6 Cas Pratiques IT

2.6.1 Validation de données

In [18]:
```
## Validation d'une adresse email simple
email = "utilisateur@domaine.com"

if "@" in email and "." in email:
    parties = email.split("@")
    if len(parties) == 2 and len(parties[0]) > 0 and
    len(parties[1]) > 3:
        print(f"Email valide : {email}")
    else:
        print("Format d'email incorrect")
else:
    print("Email invalide : manque @ ou .")
```

Out[18]:
```
Email valide : utilisateur@domaine.com
```

2.6.2 Calcul de statistiques

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [19]:
```python
## Calculer la moyenne, min et max d'une liste
notes = [85, 92, 78, 95, 88, 76, 90]

somme = 0
minimum = notes[0]
maximum = notes[0]

for note in notes:
    somme += note
    if note < minimum:
        minimum = note
    if note > maximum:
        maximum = note

moyenne = somme / len(notes)

print(f"Notes : {notes}")
print(f"Moyenne : {moyenne:.2f}")
print(f"Minimum : {minimum}")
print(f"Maximum : {maximum}")
```

Out[19]:
```
Notes : [85, 92, 78, 95, 88, 76, 90]
Moyenne : 86.29
Minimum : 76
Maximum : 95
```

2.6.3 Recherche dans une liste

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [20]:

```python
## Rechercher un utilisateur par ID
utilisateurs = [
    {"id": 101, "nom": "Alice", "role": "admin"},
    {"id": 102, "nom": "Bob", "role": "user"},
    {"id": 103, "nom": "Charlie", "role": "user"},
]

id_recherche = 102
trouve = False

for user in utilisateurs:
    if user["id"] == id_recherche:
        print(f"Utilisateur trouvé : {user['nom']} ({user['role']})")
        trouve = True
        break

if not trouve:
    print(f"Aucun utilisateur avec l'ID {id_recherche}")
```

Out[20]:

```
Utilisateur trouvé : Bob (user)
```

2.7 Travaux Pratiques du Module 2

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Travaux Pratiques : Structures de Contrôle

TP 1 – Calculatrice de mentions

Créez un programme qui :

1. Demande une note sur 100
2. Affiche la mention correspondante :
   - 90+ : Excellent | 80-89 : Très bien | 70-79 : Bien
   - 60-69 : Assez bien | 50-59 : Passable | <50 : Échec
3. Indique si l'étudiant est admis (note $\geq$ 50)

TP 2 – Table de multiplication

Créez un programme qui :

1. Demande un nombre à l'utilisateur
2. Affiche sa table de multiplication de 1 à 10
3. Utilise une boucle `for` avec `range()`

Bonus : Permettre à l'utilisateur de choisir la plage (ex : de 5 à 15).

Exemple de sortie attendue :

```
=== Bienvenue dans le générateur de fiche d'identité ===
Votre prénom : Avoundoupou
Votre année de naissance : 2005
Votre ville : Lomé
=== Votre fiche ===
Prénom : Mipoupou
Âge approximatif : 20 ans
Ville : Lomé
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

TP 3 – Jeu du nombre mystère

Créez un jeu où :

1. Le programme choisit un nombre entre 1 et 100 (utilisez `random.randint()`)
2. L'utilisateur doit deviner le nombre
3. À chaque tentative, indiquer "Plus grand" ou "Plus petit"
4. Compter le nombre de tentatives
5. Utiliser une boucle `while`

TP 4 – Analyse de texte

Créez un programme qui analyse une phrase :

1. Demande une phrase à l'utilisateur
2. Compte le nombre de voyelles et de consonnes
3. Affiche le nombre de mots
4. Indique si la phrase contient des chiffres

★ TP 5 – Générateur de mot de passe

Créez un générateur de mot de passe :

1. Demande la longueur souhaitée (minimum 8)
2. Demande si inclure : majuscules, chiffres, caractères spéciaux
3. Génère un mot de passe aléatoire selon les critères
4. Vérifie la robustesse (faible/moyen/fort)

Modules utiles : `random`, `string`

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

2.8 Quiz du Module 2

1 **[Vrai/Faux]** L'instruction `elif` peut être utilisée sans `if` préalable.

☐ Vrai ☐ Faux

2 **[QCM]** Que produit `range(2, 8, 2)` ?

A. 2, 4, 6, 8

B. 2, 4, 6

C. 2, 3, 4, 5, 6, 7

D. 0, 2, 4, 6

3 **[Vrai/Faux]** Une boucle `while True:` s'exécute indéfiniment sauf si elle contient un `break`.

☐ Vrai ☐ Faux

4 **[QCM]** Quelle instruction permet de passer à l'itération suivante sans exécuter le reste du bloc ?

A. `break`

B. `continue`

C. `pass`

D. `next`

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

5 **[QCM]** Combien de fois s'exécute cette boucle ? `for i in range(5): print(i)`

- **A.** 4 fois
- **B.** 5 fois
- **C.** 6 fois
- **D.** Erreur

6 **[Vrai/Faux]** L'instruction `pass` termine la boucle en cours.

☐ Vrai ☐ Faux

7 **[QCM]** Quelle est la valeur de `x` après : `x = 10 if 5 > 3 else 20` ?

- **A.** 10
- **B.** 20
- **C.** 5
- **D.** Erreur

8 **[Question ouverte]** Quelle est la différence principale entre `for` et `while` ?

For est utilisé quand le nbre d'itération est connu et while quand le nbre d'itération est inconnu

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

9 **[QCM]** Que fait `enumerate()` ?

A. Compte les éléments

B. Retourne index et valeur

C. Trie la liste

D. Inverse la liste

10 **[Question ouverte]** Écrivez une boucle `for` qui affiche les nombres de 1 à 5.

Récapitulatif : Structures de Contrôle

Conditions

- `if` : Si condition vraie
- `elif` : Sinon si...
- `else` : Sinon (défaut)
- Ternaire : `a if cond else b`

Boucle `for`

- Parcourt un itérable
- `range()` pour compteur
- `enumerate()` pour index
- Nombre d'itérations connu

Boucle `while`

- Tant que condition vraie
- Nombre d'itérations inconnu
- Attention boucles infinies !

Contrôle de flux

- `break` : Sortie immédiate
- `continue` : Itération suivante
- `pass` : Ne fait rien

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

CHAPITRE III

FONCTIONS EN PYTHON : MODULARITÉ ET RÉUTILISABILITÉ

◎ Objectifs d'apprentissage

À l'issue de ce module, vous serez capable de :

- ✓ **Définir et appeler** des fonctions — comme créer une recette de cuisine réutilisable.
- ✓ Maîtriser les **paramètres** — comme personnaliser une pizza (taille, ingrédients).
- ✓ Utiliser `return` — pour obtenir un résultat utilisable (ex : calculer votre IMC).
- ✓ Comprendre la **portée des variables** — pourquoi une variable « disparaît » après une fonction.
- ✓ Exploiter les **modules standards** — comme utiliser une calculatrice scientifique intégrée.
- ✓ Documenter avec des **docstrings** — pour que vos collègues comprennent votre code.

Fonction = Recette de cuisine

Une fonction, c'est comme une **recette** :

- **Ingrédients** = paramètres (farine, œufs...)
- **Étapes** = corps de la fonction (mélanger, cuire...)
- **Résultat** = valeur retournée (un gâteau !)

→ Une fois la recette écrite, vous pouvez la réutiliser autant de fois que vous voulez !

Les fonctions sont des **blocs de code réutilisables** qui permettent de :

- **Éviter la répétition** : écrire une fois, utiliser partout (ex : calculer la TVA sur 100 produits)
- **Organiser le code** : diviser un gros problème en petites tâches (ex : inscription → valider email → créer compte → envoyer confirmation)
- **Faciliter la maintenance** : corriger un bug à un seul endroit
- **Améliorer la lisibilité** : `calculer_imc(poids, taille)` est plus clair que 5 lignes de formule

Une fonction : boîte noire utile

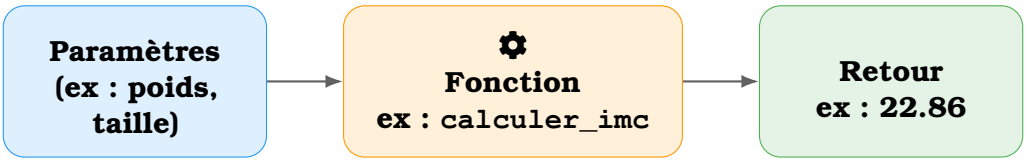


Figure 3.1 – *Vous donnez des données → la fonction travaille → vous récupérez un résultat*

3.1 Définition et Appel de Fonctions

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

3.1.1 Syntaxe de base — décortiquée

Fonction en Python — vue simple

Une fonction est un **petit programme dans le programme**. Elle commence par `def`, a un nom, des paramètres (optionnels), et se termine par `:`.

Structure :

```
def nom_fonction(param1, param2):
    """Description courte (docstring)"""
    # Instructions indentées
    resultat = param1 + param2
    return resultat
```

In [1]:

```
def saluer():
    """Affiche un message de bienvenue."""
    print("Bonjour et bienvenue !")

# Appel de la fonction (exécution)
saluer()
saluer()  # On peut répéter l'appel
```

Out[1]:

```
Bonjour et bienvenue !
Bonjour et bienvenue !
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Pourquoi l'indentation ?

En Python, l'indentation (décalage vers la droite) n'est pas optionnelle : → Elle indique **ce qui appartient à la fonction**. → Utilisez **4 espaces** (pas de tabulations) — votre IDE le fait automatiquement.

3.1.2 Fonctions avec paramètres — cas concret : saluer quelqu'un

In [2]:
```
def saluer_personne(nom):
    """Salue une personne par son prénom."""
    print(f"Bonjour, {nom} !")

# Appels avec différents arguments
saluer_personne("Alice")
saluer_personne("Bob")
saluer_personne("Charlie")
```

Out[2]:
```
Bonjour, Alice !
Bonjour, Bob !
Bonjour, Charlie !
```

In [3]:
```
def calculer_age(annee_naissance):
    """Calcule l'âge approximatif en 2024."""
    age = 2024 - annee_naissance
    return age

# Utilisation
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
age_alice = calculer_age(2000)
print(f"Alice a {age_alice} ans.")

age_bob = calculer_age(1995)
print(f"Bob a {age_bob} ans.")
```

Out [3]:
```
Alice a 24 ans.
Bob a 29 ans.
```

Paramètre vs Argument — différence clé

- **Paramètre** : variable dans la définition (`nom` dans `def saluer_personne(nom)`)
- **Argument** : valeur réelle passée à l'appel (`"Alice"` dans `saluer_personne("Alice")`)

→ C'est comme un emplacement (`nom`) et ce qu'on y met (`"Alice"`).

3.2 Retour de Valeurs avec `return`

`return` — la clé de la réutilisabilité

`return` permet à une fonction de **donner un résultat** que vous pouvez :

- Stocker dans une variable
- Afficher avec `print()`
- Utiliser dans un calcul

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

› Passer à une autre fonction

Sans `return`, la fonction retourne `None` (« rien »).

In [4]:
```python
def additionner(a, b):
    """Retourne la somme de deux nombres."""
    return a + b

# Stocker le résultat
somme = additionner(10, 5)
print(f"10 + 5 = {somme}")

# Utiliser directement
print(f"Résultat immédiat : {additionner(7, 3)}")

# Réutiliser dans un calcul
total = additionner(100, additionner(20, 5))
print(f"100 + (20 + 5) = {total}")
```

Out[4]:
```
10 + 5 = 15
Résultat immédiat : 10
100 + (20 + 5) = 125
```

3.2.1 Retourner plusieurs valeurs — cas concret : division euclidienne

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [5]:

```python
def diviser(a, b):
    """Retourne quotient et reste de a divisé par b."""
    quotient = a // b    # Division entière
    reste = a % b        # Modulo (reste)
    return quotient, reste

# Récupérer les deux valeurs
q, r = diviser(17, 5)
print(f"17  5 = {q} reste {r}")

# Ou récupérer le tuple complet
resultat = diviser(20, 6)
print(f"20  6  quotient = {resultat[0]}, reste =
    {resultat[1]}")
```

Out [5]:

```
17 ÷ 5 = 3 reste 2
20 ÷ 6 → quotient = 3, reste = 2
```

3.2 .2 Différence cruciale : `return` vs `print`

Erreur fréquente des débutants

- `print()` affiche à l'écran → utile pour le débogage
- `return` renvoie une valeur → indispensable pour la réutilisation

→ Une fonction sans `return` ne peut pas être utilisée dans des calculs !

In [6]:

```python
## Fonction avec print (mauvaise pratique pour les calculs)
def carre_print(x):
    print(x * x)

# Fonction avec return (bonne pratique)
def carre_return(x):
    return x * x

```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```python
# Test
print("=== Avec print ===")
resultat1 = carre_print(4)    # Affiche 16, mais resultat1 = None
print(f"Résultat stocké : {resultat1}")

print("\n=== Avec return ===")
resultat2 = carre_return(4)   # Ne affiche rien, mais resultat2 = 16
print(f"Résultat stocké : {resultat2}")
print(f"Double du carré : {resultat2 * 2}")
```

Out[6]:
```
=== Avec print ===
16
Résultat stocké : None
=== Avec return ===
Résultat stocké : 16
Double du carré : 32
```

3.3 Types de Paramètres — Flexibilité maximale

Python permet de créer des fonctions très flexibles — comme un formulaire web avec champs obligatoires et optionnels.

3.3.1 Paramètres par défaut — cas concret : message personnalisé

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [7]:

```python
def envoyer_message(destinataire, message="Bonjour !"):
    """Envoie un message à un destinataire."""
    print(f" À {destinataire} : {message}")

# Appels
envoyer_message("Alice")                        # Utilise le message par défaut
envoyer_message("Bob", "Salut, ça va ?")        # Remplace le message
envoyer_message("Charlie", message="Coucou !")  # Argument nommé
```

Out[7]:

```
→ À Alice : Bonjour !
→ À Bob : Salut, ça va ?
→ À Charlie : Coucou !
```

⚠ Ordre des paramètres

Les paramètres **sans valeur par défaut** doivent toujours venir **avant** ceux avec valeur par défaut :

```
def bon_appel(prenom, nom, age=18):       # OK
def mauvais_appel(age=18, prenom, nom):   # ERREUR ! SyntaxError
```

3.3.2 Arguments nommés — clarté et flexibilité

In [8]:

```python
def creer_produit(nom, prix, categorie="Autre", en_stock=True):
    """Crée une fiche produit."""
    print(f"Produit : {nom}")
    print(f"Prix : {prix} €")
    print(f"Catégorie : {categorie}")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
    print(f"En stock : {'Oui' if en_stock else 'Non'}")
    print("-" * 30)

# Appels variés
creer_produit("Laptop", 999, "Électronique")
creer_produit("Cahier", 2.5, en_stock=False)
creer_produit(prix=15, nom="Stylo", categorie="Fournitures")
```

Out[8]:
```
Produit : Laptop
Prix : 999 €
Catégorie : Électronique
En stock : Oui --------------------
Produit : Cahier
Prix : 2.5 €
Catégorie : Autre
En stock : Non --------------------
Produit : Stylo
Prix : 15 €
Catégorie : Fournitures
En stock : Oui --------------------
```

3.3.3 Nombre variable d'arguments — cas concret : calculer la moyenne

In [9]:
```
def calculer_moyenne(*notes):
    """Calcule la moyenne d'un nombre variable de notes."""
    if len(notes) == 0:
        return 0
    return sum(notes) / len(notes)

```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
# Appels avec différents nombres de notes
print(f"Moyenne de 12 et 15 : {calculer_moyenne(12, 15):.2f}")
print(f"Moyenne de 10, 14, 16 : {calculer_moyenne(10, 14, 16):.2f}")
print(f"Moyenne vide : {calculer_moyenne():.2f}")
```

Out[9]:
```
Moyenne de 12 et 15 : 13.50
Moyenne de 10, 14, 16 : 13.33
Moyenne vide : 0.00
```

3.3.4 Arguments nommés variables — cas concret : profil utilisateur

In [10]:
```
def creer_profil(nom, **details):
    """Crée un profil avec des informations variables."""
    print(f"Profil de {nom} :")
    for cle, valeur in details.items():
        print(f"   {cle.capitalize()} : {valeur}")
    print()

# Appels flexibles
creer_profil("Alice", age=25, ville="Paris", metier="Développeuse")
creer_profil("Bob", hobbies=["Lecture", "Voyage"], actif=True)
```

Out[10]:
```
Profil de Alice :
• Age : 25
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
    • Ville : Paris
    • Metier : Développeuse
  Profil de Bob :
    • Hobbies : ['Lecture', 'Voyage']
    • Actif : True
```

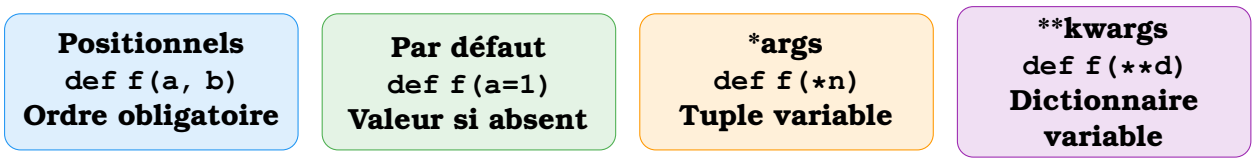


Figure 3.2 – *Les 4 types de paramètres — choisissez selon vos besoins*

3.4 Portée des Variables — Pourquoi « x n'existe pas » ?

Portée (Scope) — vue intuitive

La portée d'une variable, c'est sa « zone de validité » :

- **Locale** : variable créée dans une fonction → n'existe qu'à l'intérieur
- **Globale** : variable créée en dehors des fonctions → accessible partout

→ Comme une conversation privée (locale) vs une annonce publique (globale).

3.4.1 Variables locales — cas concret : calcul temporaire

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [11]:

```python
def calculer_remise(prix, pourcentage):
    """Calcule le prix après remise."""
    remise = prix * pourcentage / 100  # Variable LOCALE
    prix_final = prix - remise
    return prix_final

prix_initial = 100
prix_apres = calculer_remise(prix_initial, 20)
print(f"Prix initial : {prix_initial} €")
print(f"Prix après remise : {prix_apres} €")

# print(remise)  # ERREUR ! 'remise' n'existe pas ici
```

Out[11]:

```
Prix initial : 100 €
Prix après remise : 80.0 €
```

3.4.2 Variables globales — à utiliser avec précaution

In [12]:

```python
## Variable GLOBALE (définie en dehors des fonctions)
compteur_visites = 0

def visiter_page():
    global compteur_visites  # Autorise la modification de la globale
    compteur_visites += 1
    print(f"Nombre de visites : {compteur_visites}")

# Simuler des visites
visiter_page()
visiter_page()
visiter_page()
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
Out[12]:
    Nombre de visites : 1
    Nombre de visites : 2
    Nombre de visites : 3
```

⚠ Évitez les globales quand possible

Les variables globales rendent le code difficile à comprendre et à tester.
→ Préférez toujours : **paramètres en entrée + return en sortie**.

3.5 Fonctions Intégrées — Votre boîte à outils Python

Python inclut des dizaines de fonctions prêtes à l'emploi — comme les touches de raccourci de votre clavier.

Table 3.1 – *Fonctions intégrées utiles au quotidien*

Fonction	Cas d'usage	Exemple
`len()`	Compter des éléments	`len(["a", "b"])` → 2
`sum()`	Totaliser des nombres	`sum([10, 20, 30])` → 60
`max()/min()`	Trouver extrêmes	`max([5, 1, 9])` → 9
`round()`	Arrondir un prix	`round(19.996, 2)` → 20.00
`sorted()`	Trier une liste	`sorted(["c", "a", "b"])` → ["a", "b", "c"]

```
In [13]:
1 ## Cas concret : analyse de ventes
2 ventes_journalieres = [120, 95, 150, 80, 200, 110, 130]
3
4 print(f"Ventes de la semaine : {ventes_journalieres}")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```python
print(f"Total : {sum(ventes_journalieres)} €")
print(f"Moyenne : {sum(ventes_journalieres) /
    len(ventes_journalieres):.1f} €/jour")
print(f"Meilleure journée : {max(ventes_journalieres)} €")
print(f"Pire journée : {min(ventes_journalieres)} €")
print(f"Jours triés (croissant) :
    {sorted(ventes_journalieres)}")
```

Out[13]:
```
Ventes de la semaine : [120, 95, 150, 80, 200, 110, 130]
Total : 885 €
Moyenne : 126.4 €/jour
Meilleure journée : 200 €
Pire journée : 80 €
Jours triés (croissant) : [80, 95, 110, 120, 130, 150, 200]
```

3.6 Modules Standards — Vos super-pouvoirs

Un **module** est un fichier Python contenant des fonctions utiles. Python en inclut des centaines — voici les plus pratiques.

3.6.1 Le module `math` — pour les calculs précis

In [14]:
```python
import math

# Cas concret : calculer la distance entre deux points
def distance(x1, y1, x2, y2):
    """Calcule la distance euclidienne."""
    return math.sqrt((x2 - x1)**2 + (y2 - y1)**2)
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
7
8  # Exemple : distance Paris-Lyon (approximative)
9  paris = (2.35, 48.86)   # (longitude, latitude)
10 lyon = (4.84, 45.76)
11 dist = distance(paris[0], paris[1], lyon[0], lyon[1])
12 print(f"Distance Paris-Lyon : {dist:.2f} degrés")
13
14 # Autres fonctions utiles
15 print(f"25 = {math.sqrt(25)}")
16 print(f"Arrondi supérieur de 4.1 = {math.ceil(4.1)}")
17 print(f"Arrondi inférieur de 4.9 = {math.floor(4.9)}")
```

Out[14]:
```
Distance Paris-Lyon : 5.23 degrés
25 = 5.0
Arrondi supérieur de 4.1 = 5
Arrondi inférieur de 4.9 = 4
```

3.6.2 Le module `random` — pour l'aléatoire contrôlé

In [15]:
```
1  import random
2
3  # Cas concret : tirage au sort
4  participants = ["Alice", "Bob", "Charlie", "Diana", "Eve"]
5
6  # Tirer un gagnant
7  gagnant = random.choice(participants)
8  print(f"Le gagnant est : {gagnant}")
9
10 # Mélanger l'ordre de passage
11 random.shuffle(participants)
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```python
print(f"Ordre de passage : {participants}")

# Générer un code promo aléatoire
lettres = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
code = ''.join(random.choices(lettres, k=6))
print(f"Code promo : {code}")
```

```
Out[15]:
Le gagnant est : Charlie
Ordre de passage : ['Eve', 'Bob', 'Alice', 'Diana',
'Charlie']
Code promo : XKQRTM
```

3.7 Fonctions Lambda — Pour les petites tâches rapides

Fonction Lambda — vue pratique

Une fonction lambda est une **fonction courte sur une ligne**, sans nom. **Syntaxe :** `lambda paramètres: expression` → Idéale pour les tris personnalisés, les filtres, les opérations simples.

```
In [16]:
## Cas concret : trier des étudiants par note
etudiants = [
    {"nom": "Alice", "note": 85},
    {"nom": "Bob", "note": 92},
    {"nom": "Charlie", "note": 78}
]

```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```python
# Trier par note (croissant)
tries = sorted(etudiants, key=lambda x: x["note"])
print("Par note croissante :")
for e in tries:
    print(f"   {e['nom']} : {e['note']}/100")

# Trier par nom (alphabétique)
tries_nom = sorted(etudiants, key=lambda x: x["nom"])
print("\nPar nom :")
for e in tries_nom:
    print(f"   {e['nom']} : {e['note']}/100")
```

Out[16]:
```
Par note croissante :
• Charlie : 78/100
• Alice : 85/100
• Bob : 92/100
Par nom :
• Alice : 85/100
• Bob : 92/100
• Charlie : 78/100
```

3.8 Documentation des Fonctions — Pour vous et les autres

In [17]:
```python
def calculer_imc(poids, taille):
    """
    Calcule l'Indice de Masse Corporelle (IMC).

    Paramètres:
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
        poids (float): Poids en kilogrammes (ex: 70.5)
        taille (float): Taille en mètres (ex: 1.75)

    Retourne:
        float: L'IMC calculé (ex: 22.86)

    Exemples:
        >>> calculer_imc(70, 1.75)
        22.857142857142858
        >>> calculer_imc(85, 1.80)
        26.234567901234566
    """
    return poids / (taille ** 2)

# Accéder à la documentation
print("=== Documentation de calculer_imc ===")
print(calculer_imc.__doc__)

# Utilisation pratique
poids = float(input("Votre poids (kg) : ") or "70")
taille = float(input("Votre taille (m) : ") or "1.75")
imc = calculer_imc(poids, taille)
print(f"\nVotre IMC : {imc:.2f}")
```

Out[17]:

```
=== Documentation de calculer_imc ===
Calcule l'Indice de Masse Corporelle (IMC).
Paramètres:
poids (float): Poids en kilogrammes (ex: 70.5)
taille (float): Taille en mètres (ex: 1.75)
Retourne:
float: L'IMC calculé (ex: 22.86)
Exemples:
>> calculer_imc(70, 1.75)
22.857142857142858
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
>> calculer_imc(85, 1.80)
26.234567901234566
Votre poids (kg) : 70
Votre taille (m) : 1.75
Votre IMC : 22.86
```

Bonnes pratiques de documentation

- Soyez **clair et concis** — pas de jargon inutile
- Donnez des **exemples concrets** — comme ci-dessus
- Utilisez `help(nom_fonction)` dans Python pour voir la doc
- Documentez les **erreurs possibles** (ex : division par zéro)

3.9 Travaux Pratiques du Module 3

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Travaux Pratiques : Fonctions

TP 1 – Fonctions de calcul basiques

Créez les fonctions suivantes :

1. `aire_rectangle(longueur, largeur)` : retourne l'aire
2. `perimetre_rectangle(longueur, largeur)` : retourne le périmètre
3. `aire_cercle(rayon)` : retourne l'aire (πr^2)
4. `convertir_celsius_fahrenheit(celsius)` : retourne la température en °F

Testez chaque fonction avec plusieurs valeurs.

TP 2 – Fonctions de validation

Créez des fonctions de validation qui retournent `True` ou `False` :

1. `est_pair(n)` : vérifie si un nombre est pair
2. `est_positif(n)` : vérifie si un nombre est positif
3. `est_majeur(age)` : vérifie si l'âge est ≥ 18
4. `est_email_valide(email)` : vérifie si l'email contient "@" et "."

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

TP 3 – Fonctions avec listes

Créez des fonctions de traitement de listes :

1. `calculer_moyenne(notes)` : retourne la moyenne d'une liste
2. `trouver_min_max(nombres)` : retourne (minimum, maximum)
3. `compter_occurrences(liste, element)` : compte les apparitions
4. `filtrer_positifs(nombres)` : retourne une liste sans les négatifs

Bonus : N'utilisez pas les fonctions intégrées `min()`, `max()`, `sum()`.

TP 4 – Générateur de mot de passe

Créez une fonction `generer_mot_de_passe(longueur=12, majuscules=True, chiffres=True, speciaux=True)` qui :

1. Génère un mot de passe aléatoire
2. Respecte les options (avec ou sans majuscules, chiffres, caractères spéciaux)
3. Retourne le mot de passe généré

Modules : `random`, `string`

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

★ TP 5 – Mini-calculatrice modulaire

Créez une calculatrice avec les fonctions :

1. `additionner(a, b)`, `soustraire(a, b)`, `multiplier(a, b)`, `diviser(a, b)`
2. `calculer(operation, a, b)` : appelle la bonne fonction selon l'opération (+, -, *, /)
3. `menu()` : affiche le menu et gère les saisies utilisateur

Le programme doit tourner en boucle jusqu'à ce que l'utilisateur choisisse de quitter.

3.10 Quiz du Module 3

1 **[Vrai/Faux]** Une fonction sans `return` retourne automatiquement `0`.

☐ Vrai ☐ Faux

2 **[QCM]** Quelle syntaxe définit correctement une fonction ?

A. `function ma_fonction():`

B. `def ma_fonction():`

C. `func ma_fonction():`

D. `define ma_fonction():`

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

3 **[Vrai/Faux]** `*args` permet de passer un nombre variable d'arguments nommés.

☐ Vrai ☐ Faux

4 **[QCM]** Que retourne `math.sqrt(25)` ?

A. 5

B. 5.0

C. 25

D. Erreur

5 **[QCM]** Comment importer uniquement la fonction `sqrt` du module `math` ?

A. `import sqrt from math`

B. `from math import sqrt`

C. `import math.sqrt`

D. `include math.sqrt`

6 **[Vrai/Faux]** Une variable définie dans une fonction est accessible en dehors de cette fonction.

☐ Vrai ☐ Faux

7 **[QCM]** Quelle est la syntaxe correcte d'une fonction lambda ?

A. `lambda: x * 2`

B. `lambda x: x * 2`

C. `lambda(x): x * 2`

D. `def lambda x: x * 2`

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

8 **[Question ouverte]** Quelle est la différence entre `print()` et `return` ?

9 **[QCM]** Que signifie LEGB dans le contexte de la portée des variables ?

A. Local, External, Global, Built-in

B. Local, Enclosing, Global, Built-in

C. Loop, Enclosing, Global, Base

D. Local, Enclosing, General, Built-in

10 **[Question ouverte]** Écrivez une fonction `double(x)` qui retourne le double de x.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Récapitulatif : Fonctions en Python

Définition

- `def nom(params):`
- Corps indenté
- Docstring recommandée

Paramètres

- Positionnels : `f(a, b)`
- Par défaut : `f(a=1)`
- Variables : `*args`, `**kwargs`

Retour

- `return valeur`
- Multiples : `return a, b`
- Sans return → `None`

Portée

- Locale → Englobante → Globale → Built-in
- `global` pour modifier une globale

Modules courants

- `math` : calculs mathématiques
- `random` : aléatoire
- `import module as alias`

Lambda

- `lambda x: expression`
- Fonctions courtes anonymes

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

CHAPITRE IV

STRUCTURES DE DONNÉES : STOCKER, ORGANISER ET MANIPULER L'INFORMATION

◎ Objectifs d'apprentissage

À l'issue de ce module, vous serez capable de :

- ✓ **Maîtriser les Listes** — comme gérer une file d'attente flexible ou un panier d'achats modifiable.
- ✓ **Utiliser les Dictionnaires** — comme un carnet d'adresses ultra-rapide (retrouver un numéro grâce à un nom).
- ✓ **Comprendre les Tuples** — comme des coordonnées GPS ou un code de sécurité (des données fixes et intouchables).
- ✓ **Exploiter les Ensembles (Sets)** — comme trier des cartes uniques pour éliminer les doublons instantanément.
- ✓ **Choisir la bonne structure** — pour optimiser la vitesse et l'organisation de vos programmes.

Pourquoi les structures de données ?

Votre cerveau organise l'information naturellement :

- Une **liste de courses** → `list` (ordre important, modifications fréquentes)
- Un **annuaire téléphonique** → `dict` (recherche rapide par nom)
- Un **code PIN** (immuable) → `tuple` (valeurs fixes, séquentielles)
- Un **ensemble de cartes à jouer uniques** → `set` (pas de doublons)

→ Les structures de données sont des **modèles mentaux codés**, pour que l'ordinateur comprenne et optimise votre logique.

Les structures de données permettent de :

- **Stocker** des informations de manière organisée et efficace en mémoire
- **Accéder** rapidement aux données (recherche, tri, filtrage)
- **Manipuler** efficacement les données (ajout, suppression, transformation)
- **Modéliser** des problèmes réels (utilisateur, panier, réseau social, itinéraire)

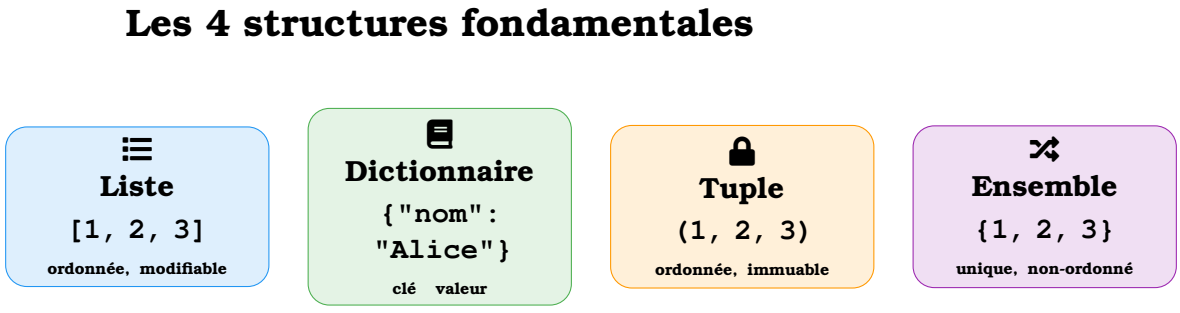


Figure 4.1 – *Chaque structure a un rôle spécifique — choisissez la bonne pour votre besoin*

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

4.1 Les Listes : Collections ordonnées et modifiables

Liste — vue théorique et scientifique

Une **liste** est une séquence **ordonnée**, **modifiable** et **indexée** d'éléments.
En informatique, c'est un **tableau dynamique** (array) qui :

- Se redimensionne automatiquement quand il est plein
- Garantit un accès en O(1) par index
- Garantit un ajout en O(1) à la fin (amorti)

Comparable à un cahier où les pages (éléments) sont numérotées et peuvent être modifiées ou remplacées.

4.1.1 Création et accès — cas concret : liste de courses

In [1]:
```
## Création d'une liste de courses
courses = ["lait", "œufs", "pain", "pommes"]

# Accès par index (commence à 0)
print(f"Premier article : {courses[0]}")
print(f"Dernier article : {courses[-1]}")  # Index négatif = depuis la fin

# Slicing (tranches) - extrait une partie de la liste
print(f"Les 2 premiers articles : {courses[0:2]}")  # De 0 à
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
    2 (exclu)

# Modification d'un élément
courses[1] = "yaourts"  # Remplacer "œufs" par "yaourts"
print(f"Liste mise à jour : {courses}")

# Longueur de la liste
print(f"Nombre d'articles : {len(courses)}")
```

Out[1]:
```
Premier article : lait
Dernier article : pommes
Les 2 premiers articles : ['lait', 'œufs']
Liste mise à jour : ['lait', 'yaourts', 'pain', 'pommes']
Nombre d'articles : 4
```

Analogie quotidienne

Une liste Python est comme un **tableau d'affichage** où :

- Chaque papier a un numéro précis (index)
- Vous pouvez changer un papier sans affecter les autres
- L'ordre est important
- Le tableau s'agrandit automatiquement quand nécessaire

Exemple : Liste de tâches dans votre agenda, où l'ordre est crucial.

4.1.2 Opérations courantes — gestion de tâches journalières

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [2]:

```
taches = ["Répondre aux emails", "Préparer la réunion"]

# Ajouter des éléments
taches.append("Appeler le client")       # À la fin
taches.insert(1, "Faire le café")        # À la position 1
print("Tâches à faire :")
for i, tache in enumerate(taches, 1):    # Commence la
    numérotation à 1
     print(f"{i}. {tache}")

# Supprimer des éléments
taches.remove("Faire le café")            # Par valeur
tache_terminee = taches.pop()             # Dernier élément
    (et le récupère)
print(f"\nTâche terminée : {tache_terminee}")
print(f"Tâches restantes : {taches}")

# Extension avec une autre liste
nouvelles_taches = ["Réserver restaurant", "Préparer
    présentation"]
taches.extend(nouvelles_taches)           # Ajoute tous les é
    léments
print(f"\nPlan de la journée : {taches}")
```

Out[2]:

```
Tâches à faire : 1. Répondre aux emails 2. Faire le café 3.
Préparer la réunion 4. Appeler le client
Tâche terminée : Appeler le client Tâches restantes :
['Répondre aux emails', 'Préparer la réunion']
Plan de la journée : ['Répondre aux emails', 'Préparer la
réunion', 'Réserver restaurant', 'Préparer présentation']
```

4.1.3 Tris et recherches — classement de notes d'étudiants

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [3]:

```
notes = [85, 92, 78, 96, 88, 76]
print(f"Notes originales : {notes}")

# Tri croissant (modifie la liste)
notes.sort()
print(f"Triées (croissant) : {notes}")

# Tri décroissant
notes.sort(reverse=True)
print(f"Triées (décroissant) : {notes}")

# Version non-destructive du tri
notes_originales = [85, 92, 78, 96, 88, 76]
triees = sorted(notes_originales)  # Ne modifie pas l'original
print(f"\nOriginal préservé : {notes_originales}")
print(f"Nouvelle liste triée : {triees}")

# Statistiques
meilleure = max(notes)
pire = min(notes)
moyenne = sum(notes) / len(notes)

print(f"\nMeilleure note : {meilleure}")
print(f"Pire note : {pire}")
print(f"Moyenne : {moyenne:.2f}")
```

Out[3]:

```
Notes originales : [85, 92, 78, 96, 88, 76] Triées
(croissant) : [76, 78, 85, 88, 92, 96] Triées (décroissant)
: [96, 92, 88, 85, 78, 76]
Original préservé : [85, 92, 78, 96, 88, 76] Nouvelle liste
triée : [76, 78, 85, 88, 92, 96]
Meilleure note : 96 Pire note : 76 Moyenne : 85.83
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

IT Pro : Performance des listes

- `append()` : $O(1)$ (très rapide)
- `insert(0, x)` : $O(n)$ (lent pour grandes listes)
- `x in list` : $O(n)$ (linéaire - lent pour grandes listes)
- `sort()` : $O(n \log n)$ (rapide mais pas instantané)

Si vous faites souvent des recherches, envisagez plutôt un `dict` ou `set` pour des recherches en $O(1)$.

4.2 Les Dictionnaires : Paires clé-valeur pour un accès ultra-rapide

Dictionnaire — vue scientifique

Un **dictionnaire** est une **table de hachage** (hash table) qui associe des **clés uniques** à des **valeurs**.
En informatique, il fonctionne par :

- **Fonction de hachage** : transforme chaque clé en position mémoire
- **Résolution de collisions** : gère quand deux clés ont le même hash

Avantage principal : accès, insertion et suppression en $O(1)$ en moyenne.

4.2.1 Création et accès — cas concret : profil utilisateur

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [4]:

```python
## Création d'un dictionnaire représentant un profil utilisateur
utilisateur = {
    "nom": "Alice",
    "age": 28,
    "ville": "Paris",
    "email": "alice@example.com",
    "langues": ["Français", "Anglais", "Espagnol"]
}

# Accès aux données par clé
print(f"Profil de {utilisateur['nom']}")
print(f"ge : {utilisateur['age']} ans")
print(f"Compétences linguistiques : {', '.join(utilisateur['langues'])}")

# Méthode get() avec valeur par défaut (évite KeyError)
telephone = utilisateur.get("telephone", "Non renseigné")
print(f"Téléphone : {telephone}")

# Ajout et modification de données
utilisateur["profession"] = "Développeuse Python"  # Nouvelle clé
utilisateur["age"] = 29                            # Modification
utilisateur["langues"].append("Allemand")          # Modification de liste imbriquée

print(f"\nProfil mis à jour :")
# Parcours clé-valeur avec .items()
for cle, valeur in utilisateur.items():
    if isinstance(valeur, list):
        print(f"   {cle.capitalize()} : {', '.join(valeur)}")
    else:
        print(f"   {cle.capitalize()} : {valeur}")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
Out[4]:
Profil de Alice Âge : 28 ans Compétences linguistiques :
Français, Anglais, Espagnol Téléphone : Non renseigné
Profil mis à jour : • Nom : Alice • Age : 29 • Ville : Paris
• Email : alice@example.com • Langues : Français, Anglais,
Espagnol, Allemand • Profession : Développeuse Python
```

Analogie quotidienne

Un dictionnaire Python est comme un **annuaire téléphonique** où :

- Vous cherchez directement par nom (clé), sans parcourir toutes les pages
- Chaque nom est unique (pas deux "Dupont Jean" identiques)
- À chaque nom correspond une ou plusieurs informations (valeurs)

Exemples quotidiens : carnet d'adresses, inventaire avec code-barres, menu avec prix.

4.2 .2 Méthodes utiles — gestion d'inventaire de magasin

```
In [5]:
## Inventaire de magasin avec stock et prix
inventaire = {
    "pommes": {"stock": 50, "prix": 1.20, "unité": "kg"},
    "bananes": {"stock": 30, "prix": 1.80, "unité": "kg"},
    "oranges": {"stock": 40, "prix": 2.10, "unité": "kg"},
    "fraises": {"stock": 0, "prix": 3.50, "unité":
   "barquette"}
}

# Affichage de l'inventaire avec formatage monétaire
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
print("INVENTAIRE DU MAGASIN:")
print("======================")
for produit, info in inventaire.items():
    statut = "EN STOCK" if info["stock"] > 0 else "RUPTURE "
    print(f"{produit.capitalize()} ({info['unité']}) -
    {info['prix']:.2f}€ - {statut}")

# Vérifier si un produit existe
produit = "kiwis"
if produit in inventaire:
    print(f"\n{produit.capitalize()} en stock :
    {inventaire[produit]['stock']}
    {inventaire[produit]['unité']}")
else:
    print(f"\n{produit.capitalize()} : non référencé dans
    l'inventaire !")

# Valeur totale de l'inventaire
valeur_totale = sum(info["stock"] * info["prix"] for info in
    inventaire.values())
print(f"\nValeur totale de l'inventaire :
    {valeur_totale:.2f}€")

# Opérations sur l'inventaire
print("\nOPÉRATIONS:")
# Vente de 5 kg de pommes
if inventaire["pommes"]["stock"] >= 5:
    inventaire["pommes"]["stock"] -= 5
    print(f" Vente: 5 kg de pommes à
    {inventaire['pommes']['prix']*5:.2f}€")
else:
    print(" Stock insuffisant de pommes")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
Out[5]: INVENTAIRE DU MAGASIN: ==================== Pommes (kg) -
    1.20€ - EN STOCK Bananes (kg) - 1.80€ - EN STOCK Oranges (kg)
    - 2.10€ - EN STOCK Fraises (barquette) - 3.50€ - RUPTURE
    Kiwis : non référencé dans l'inventaire !
    Valeur totale de l'inventaire : 189.00€
    OPÉRATIONS:  Vente: 5 kg de pommes à 6.00€
```

⚠ Pièges fréquents avec les dictionnaires

- `mon_dict["clé_absente"]` lève une `KeyError`
- Solution : `mon_dict.get("clé_absente", valeur_défaut)`
- Les clés doivent être hashables (immuables) :
  - OK : chaînes, nombres, tuples (contenant uniquement des hashables)
  - Erreur : listes, dictionnaires, ensembles

4.3 Les Tuples : Collections ordonnées et immuables

Tuple — vue théorique

Un **tuple** est une séquence **ordonnée** et **immuable** (non modifiable) d'éléments.
Caractéristiques informatiques :

- Structure plus légère qu'une liste en mémoire
- Hashable si tous ses éléments sont hashables (utilisable comme clé de dictionnaire)

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

> Multirôle : retours multiples de fonctions, associativité clé-valeur

En Python, un tuple est comme une liste "constante" - une fois créé, son contenu ne peut pas changer.

4.3.1 Création et utilisation — cas concret : coordonnées GPS

In [6]:

```python
## Coordonnées GPS sous forme de tuples (latitude, longitude)
paris = (48.8566, 2.3522)
lyon = (45.7640, 4.8357)
marseille = (43.2965, 5.3698)

print(f"Paris : {paris}")
print(f"Lyon : {lyon}")

# Accès par index
print(f"\nLatitude de Paris : {paris[0]}")
print(f"Longitude de Lyon : {lyon[1]}")

# Déballage (unpacking) - très utile et lisible
lat_paris, lon_paris = paris
print(f"\nParis  Latitude: {lat_paris}, Longitude:
    {lon_paris}")

# Tentative de modification (génère une erreur)
try:
    paris[0] = 49.0
except TypeError as e:
    print(f"\nErreur: {e}")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```python

# Tuple dans une liste (données fixes dans une collection modifiable)
villes = [
    ("Paris", 48.8566, 2.3522),
    ("Lyon", 45.7640, 4.8357),
    ("Marseille", 43.2965, 5.3698)
]

print("\nListe des villes :")
for nom, lat, lon in villes:
    print(f"   {nom} : ({lat}, {lon})")
```

Out[6]:
```
Paris : (48.8566, 2.3522) Lyon : (45.764, 4.8357)
Latitude de Paris : 48.8566 Longitude de Lyon : 4.8357
Paris → Latitude: 48.8566°, Longitude: 2.3522°
Erreur: 'tuple' object does not support item assignment
Liste des villes : • Paris : (48.8566, 2.3522) • Lyon :
(45.764, 4.8357) • Marseille : (43.2965, 5.3698)
```

Quand utiliser un tuple ?

- Coordonnées, dates, couleurs RGB
- Clés de dictionnaire (car immuables)
- Retour multiple de fonction (`return x, y`)
- Données qui ne doivent pas changer (sécurité)

4.3.2 Cas d'utilisation pratiques — tuples dans le monde réel

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [7]:

```python
## 1. Tuples comme retour multiple d'une fonction
def statistiques(nombres):
    """Calcule plusieurs statistiques d'une série"""
    return (min(nombres), max(nombres), sum(nombres) / len(nombres))

notes = [15, 12, 18, 9, 14, 16]
min_note, max_note, moyenne = statistiques(notes)
print(f"Notes - Min: {min_note}, Max: {max_note}, Moyenne: {moyenne:.1f}")

# 2. Échange de variables avec tuple unpacking
a, b = 10, 20
print(f"\nAvant échange: a = {a}, b = {b}")
a, b = b, a  # Échange élégant avec tuple unpacking
print(f"Après échange: a = {a}, b = {b}")

# 3. Tuples pour données fixes (couleurs RGB)
# Définition de couleurs comme tuples (R, G, B)
ROUGE = (255, 0, 0)
VERT = (0, 255, 0)
BLEU = (0, 0, 255)

# Fonction qui utilise des tuples pour les couleurs
def melanger_couleurs(couleur1, couleur2, ratio=0.5):
    """Mélange deux couleurs selon un ratio"""
    r1, g1, b1 = couleur1
    r2, g2, b2 = couleur2
    r = int(r1 * (1 - ratio) + r2 * ratio)
    g = int(g1 * (1 - ratio) + g2 * ratio)
    b = int(b1 * (1 - ratio) + b2 * ratio)
    return (r, g, b)

violet = melanger_couleurs(ROUGE, BLEU, 0.5)
print(f"\nMélange rouge+bleu (50\%): {violet}  Violet")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
Out[7]:
Notes - Min: 9, Max: 18, Moyenne: 14.0
Avant échange: a = 10, b = 20 Après échange: a = 20, b = 10
Mélange rouge+bleu (50%): (127, 0, 127) → Violet
```

4.4 Les Ensembles : Collections non ordonnées et sans doublons

Ensemble — vue mathématique

Un **ensemble** (set) est une collection **non ordonnée** d'éléments **uniques** et **hashables**.
Caractéristiques techniques :

- Implémenté comme une table de hachage sans valeurs (clés uniquement)
- Opérations de recherche, ajout, suppression en $O(1)$ (temps constant)
- Implémente les opérations mathématiques sur les ensembles (union, intersection...)

L'équivalent mathématique des ensembles de la théorie des ensembles.

4.4.1 Création et opérations — comparaison de playlists musicales

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [8]:

```python
## Playlists de trois utilisateurs (sous forme d'ensembles)
alice = {"Bohemian Rhapsody", "Hotel California", "Imagine", "Yesterday", "Billie Jean"}
bob = {"Imagine", "Stairway to Heaven", "Billie Jean", "Thriller", "Yesterday"}
charlie = {"Bohemian Rhapsody", "Stairway to Heaven", "Sweet Child O'Mine", "November Rain"}

# Affichage des playlists (triées pour la lisibilité)
print(f"Playlist d'Alice: {sorted(alice)}")
print(f"Playlist de Bob: {sorted(bob)}")
print(f"Playlist de Charlie: {sorted(charlie)}")

# 1. Opérations ensemblistes classiques
chansons_communes_ab = alice & bob  # Intersection
print(f"\nChansons communes à Alice et Bob: {sorted(chansons_communes_ab)}")

toutes_les_chansons = alice | bob | charlie  # Union
print(f"Nombre total de chansons uniques: {len(toutes_les_chansons)}")

uniques_alice = alice - (bob | charlie)  # Différence
print(f"Chansons qu'Alice est la seule à avoir: {sorted(uniques_alice)}")

# 2. Autres opérations ensemblistes
difference_symetrique = alice ^ bob  # XOR - dans l'un OU l'autre, mais pas les deux
print(f"\nChansons qu'Alice OU Bob ont (pas les deux): {sorted(difference_symetrique)}")

# 3. Opérations de modification
print("\nOpérations sur la playlist de Charlie:")
charlie.add("Sweet Dreams")  # Ajout
print(f"Après ajout: {len(charlie)} chansons")

charlie.remove("November Rain")  # Suppression (erreur si absent)
print(f"Après suppression: {len(charlie)} chansons")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
32
33 charlie.discard("Chanson Inexistante")  # Suppression sans
      erreur
34 print(f"Après discard (élément absent): {len(charlie)}
      chansons")
```

Out[8] :

```
Playlist d'Alice: ['Billie Jean', 'Bohemian Rhapsody', 'Hotel
California', 'Imagine', 'Yesterday']
Playlist de Bob: ['Billie Jean', 'Imagine', 'Stairway to
Heaven', 'Thriller', 'Yesterday']
Playlist de Charlie: ['Bohemian Rhapsody', 'November Rain',
'Stairway to Heaven', 'Sweet Child ÓMine']
Chansons communes à Alice et Bob: ['Billie Jean', 'Imagine',
'Yesterday']
Nombre total de chansons uniques: 9
Chansons qu'Alice est la seule à avoir: ['Hotel California']
Chansons qu'Alice OU Bob ont (pas les deux): ['Bohemian
Rhapsody', 'Hotel California', 'Stairway to Heaven',
'Thriller']
Opérations sur la playlist de Charlie:
Après ajout: 5 chansons
Après suppression: 4 chansons
Après discard (élément absent): 4 chansons
```

Analogie quotidienne

Un ensemble Python est comme :

- Une **collection de timbres** où chaque timbre doit être unique
- Un **club privé** où chaque membre a un numéro d'adhérent unique
- Une **boîte à outils** où vous n'avez besoin que d'un exemplaire de chaque outil

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

En mathématiques, les ensembles correspondent aux ensembles de la théorie des ensembles :

- $A \cup B$ (union) `A | B`
- $A \cap B$ (intersection) `A & B`
- $A - B$ (différence) `A - B`
- $A \triangle B$ (différence symétrique) `A ^ B`

4.4 .2 Performance — recherche dans une grande collection

In [9]:

```python
import time
import random

# Génération de données de test
taille = 100000  # 100,000 éléments
valeurs = [f"valeur_{i}" for i in range(taille)]
valeur_recherchee = f"valeur_{random.randint(0, taille-1)}"

# Création des structures
liste_valeurs = valeurs.copy()
ensemble_valeurs = set(valeurs)
dict_valeurs = {val: None for val in valeurs}  # Les valeurs sont les clés

# 1. Mesure du temps de recherche dans une liste
debut = time.time()
trouve_liste = valeur_recherchee in liste_valeurs
temps_liste = time.time() - debut

```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```python
# 2. Mesure du temps de recherche dans un ensemble
debut = time.time()
trouve_ensemble = valeur_recherchee in ensemble_valeurs
temps_ensemble = time.time() - debut

# 3. Mesure du temps de recherche dans un dictionnaire (par clé)
debut = time.time()
trouve_dict = valeur_recherchee in dict_valeurs
temps_dict = time.time() - debut

print(f"Recherche dans {taille:,} éléments:".replace(",", " "))
print(f"Liste:      {temps_liste*1000:.3f} ms")
print(f"Ensemble:   {temps_ensemble*1000:.6f} ms")
print(f"Dictionnaire: {temps_dict*1000:.6f} ms")

if temps_liste > 0:  # Éviter division par zéro
    print(f"\nGain de performance Set vs List: {temps_liste/temps_ensemble:.0f}x")
    print(f"Gain de performance Dict vs List: {temps_liste/temps_dict:.0f}x")
```

Out[9]:
```
Recherche dans 100 000 éléments:
Liste: 18.420 ms
Ensemble: 0.009537 ms
Dictionnaire: 0.009775 ms
Gain de performance Set vs List: 1931x
Gain de performance Dict vs List: 1884x
```

4.5 Applications Pratiques — Du quotidien au professionnel

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

4.5.1 Panier d'achat e-commerce — combinaison listes et dictionnaires

In [10]:

```
## Base de données produits (dictionnaire)
produits = {
    "LAP001": {"nom": "Laptop Pro X", "prix": 999, "stock": 10},
    "SOU002": {"nom": "Souris sans fil", "prix": 25.99, "stock": 50},
    "ECR004": {"nom": "Écran 27 pouces", "prix": 249.50, "stock": 8}
}

# Panier d'achat (liste de dictionnaires)
panier = []

# Fonction pour ajouter un article
def ajouter_au_panier(code_produit, quantite=1):
    if code_produit not in produits:
        print(f" Produit {code_produit} inconnu")
        return False

    if produits[code_produit]["stock"] < quantite:
        print(f" Stock insuffisant pour {produits[code_produit]['nom']}")
        return False

    # Vérifier si le produit est déjà dans le panier
    for article in panier:
        if article["code"] == code_produit:
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
24              article["quantite"] += quantite
25              print(f" Quantité mise à jour:
    {article['quantite']}")
26              return True
27
28      # Sinon, ajouter comme nouvel article
29      panier.append({
30          "code": code_produit,
31          "nom": produits[code_produit]["nom"],
32          "prix": produits[code_produit]["prix"],
33          "quantite": quantite
34      })
35
36      print(f" Ajouté au panier: {quantite}x
    {produits[code_produit]['nom']}")
37      return True
38
39  # Simulation d'achat
40  ajouter_au_panier("LAP001")
41  ajouter_au_panier("SOU002", 2)
42  ajouter_au_panier("SOU002")  # Augmente la quantité
43
44  # Afficher le contenu du panier
45  print("\n CONTENU DU PANIER:")
46  print("=" * 50)
47  for article in panier:
48      total_ligne = article["prix"] * article["quantite"]
49      print(f"{article['nom']}  {article['quantite']} =
    {total_ligne:.2f}€")
50
51  # Calculer le total
52  total = sum(article["prix"] * article["quantite"] for
    article in panier)
53  print("=" * 50)
54  print(f"TOTAL: {total:.2f}€")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Out[10]:
```
Ajouté au panier: 1x Laptop Pro X
Ajouté au panier: 2x Souris sans fil
Quantité mise à jour: 3
CONTENU DU PANIER:
==================================================
Laptop Pro X × 1 = 999.00€
Souris sans fil × 3 = 77.97€
==================================================
TOTAL: 1076.97€
```

4.5.2 Analyse de texte — comptage de mots avec dictionnaires et ensembles

In [11]:
```python
import re
from collections import Counter

# Texte d'exemple (extrait de "Le Petit Prince")
texte = """
Lorsque j'avais six ans j'ai vu, une fois, une magnifique
    image, dans un livre
sur la Forêt Vierge qui s'appelait "Histoires Vécues". Ça
    représentait un serpent
boa qui avalait un fauve. Voilà la copie du dessin.
"""

# Nettoyage et découpage du texte
def nettoyer_texte(texte):
    # Convertir en minuscules
    texte = texte.lower()
    # Remplacer les caractères de ponctuation par des espaces
    texte = re.sub(r'[^\w\s]', ' ', texte)
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
    # Diviser en mots
    mots = texte.split()
    return mots

mots = nettoyer_texte(texte)

# 1. Analyse de fréquence avec Counter
compteur = Counter(mots)
mots_frequents = compteur.most_common(5)

print("TOP 5 DES MOTS LES PLUS FRÉQUENTS:")
print("=" * 40)
for mot, frequence in mots_frequents:
    print(f"{mot:<15} {frequence:>5} occurrences")

# 2. Analyse avec ensemble - mots uniques
mots_uniques = set(mots)
print(f"\nNombre de mots uniques: {len(mots_uniques)} sur
    {len(mots)} mots au total")
print(f"Diversité lexicale:
    {len(mots_uniques)/len(mots):.2%}")
```

Out[11]:

```
TOP 5 DES MOTS LES PLUS FRÉQUENTS:
========================================
un → 2 occurrences
une → 2 occurrences
j → 2 occurrences
ai → 1 occurrences
vu → 1 occurrences
Nombre de mots uniques: 26 sur 29 mots au total
Diversité lexicale: 89.66%
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Synthèse : Les 4 règles d'or

1. **Liste** quand vous avez besoin d'une **séquence modifiable ordonnée**
   - Cas réels : historique des actions, étapes d'un processus, résultats d'analyse
   - Forces : ordre maintenu, indexation, facilité d'ajout/suppression
2. **Dictionnaire** quand vous avez besoin d'un **accès par clé ultra-rapide**
   - Cas réels : profils utilisateurs, configurations, traductions, inventaires
   - Forces : recherche O(1), flexible, supporte des valeurs complexes
3. **Tuple** quand vous avez des **données fixes et immuables**
   - Cas réels : coordonnées géographiques, configurations, valeurs RGB, retours multiples
   - Forces : performance, hashable (utilisable comme clé), garantie d'intégrité
4. **Ensemble** quand vous faites des **opérations ensemblistes ou d'unicité**
   - Cas réels : déduplications, appartenances, intersections entre collections
   - Forces : recherche O(1), élimination automatique des doublons

Conseil d'expert : Avant d'optimiser, mesurez les performances avec le module `timeit` — l'intuition est souvent trompeuse en informatique !

4.6 Travaux Pratiques du Module 4

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Travaux Pratiques : Structures de Données

TP 1 – Système de gestion de bibliothèque

Créez un système de gestion de livres avec :

- Une **liste** de livres (chaque livre est un dictionnaire)
- Fonctions pour : ajouter, supprimer, rechercher par titre/auteur
- Affichage du catalogue trié par titre

Structure d'un livre :

In [12]:
```
livre = {
    "titre": "Python pour les débutants",
    "auteur": "Alice Dupont",
    "isbn": "978-3-16-148410-0",
    "disponible": True
}
```

Bonus : Compter les livres disponibles/indisponibles avec un dictionnaire.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

TP 2 – Analyse de résultats de sondage

Vous avez les réponses à un sondage sur les langages préférés :

In [13]:
```
reponses = [
    "Python", "JavaScript", "Python", "Java",
    "Python", "C++", "JavaScript", "Python", "Go"
]
```

Écrivez un programme qui :

- Compte les occurrences avec un dictionnaire
- Trouve le langage le plus populaire
- Calcule le pourcentage de chaque langage
- Affiche les résultats triés par popularité

Exemple de sortie attendue :

Out[13]:
```
Résultats du sondage (9 réponses) :
• Python : 4 votes (44.44%)
• JavaScript : 2 votes (22.22%)
• Java : 1 vote (11.11%)
• C++ : 1 vote (11.11%)
• Go : 1 vote (11.11%)
Langage préféré : Python
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

TP 3 – Comparateur de playlists musicales

Créez un outil qui compare deux playlists :

- Utilisez des **ensembles** pour stocker les titres
- Calculez : musiques communes, uniques à chaque utilisateur
- Proposez des recommandations (musiques de l'autre non présentes)

Données de test :

In [14]:
```
alice = {"Imagine", "Bohemian Rhapsody", "Hotel
    California", "Smells Like Teen Spirit"}
bob = {"Bohemian Rhapsody", "Smells Like Teen
    Spirit", "Sweet Child O' Mine", "Wonderwall"}
```

Sortie attendue :

Out[14]:
```
=== Comparaison des playlists ===
Musiques communes (2) :
• Bohemian Rhapsody
• Smells Like Teen Spirit
Uniques à Alice (2) :
• Hotel California
• Imagine
Uniques à Bob (2) :
• Sweet Child O' Mine
• Wonderwall
Recommandations pour Alice :
• Sweet Child O' Mine
• Wonderwall
Recommandations pour Bob :
• Hotel California
• Imagine
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

TP 4 – Gestionnaire de configuration sécurisée

Créez un système de configuration immuable :

- Stockez les paramètres dans un **tuple** de tuples (clé, valeur)
- Implémentez une fonction `get_config(cle)` pour lire les valeurs
- Gérez les valeurs par défaut avec `get_config(cle, defaut)`

Exemple de configuration :

In [15]:
```
CONFIG = (
    ("DATABASE_URL", "sqlite:///app.db"),
    ("DEBUG", False),
    ("PORT", 8000),
    ("ALLOWED_HOSTS", ("localhost", "127.0.0.1"))
)
```

Tests à implémenter :

In [16]:
```
print(get_config("PORT"))            # 8000
print(get_config("DEBUG"))           # False
print(get_config("SECRET_KEY", "default"))   # default
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

★ TP 5 – Système de recommandation de films

Créez un système basique de recommandation :

- Base de données : liste de films (dictionnaires avec titre, genre, acteurs)
- Profil utilisateur : ensemble de genres préférés
- Fonction `recommander_films(profil)` qui :
  - Retourne les films dont le genre est dans le profil
  - Trie par popularité (nombre d'acteurs connus)

Données de test :

```
In [17]:
films = [
    {"titre": "Inception", "genre": "SF",
    "acteurs": ["DiCaprio", "Cotillard"]},
    {"titre": "Le Parrain", "genre": "Drame",
    "acteurs": ["Brando", "Pacino"]},
    {"titre": "Interstellar", "genre": "SF",
    "acteurs": ["McConaughey", "Cotillard"]},
    {"titre": "Pulp Fiction", "genre": "Action",
    "acteurs": ["Travolta", "Jackson"]}
]

profil_utilisateur = {"SF", "Action"}
```

Sortie attendue :

```
Out[17]:
Films recommandés pour vous :
• Interstellar (SF) - 2 acteurs connus
• Inception (SF) - 2 acteurs connus
• Pulp Fiction (Action) - 2 acteurs connus
```

Bonus avancé : Ajouter un système de notation utilisateur avec dictionnaire.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

4.7 Quiz du Module 4

1 **[Vrai/Faux]** Une liste peut contenir des éléments de types différents (ex : nombres, chaînes, listes).

☐ Vrai ☐ Faux

2 **[QCM]** Quelle structure est la plus adaptée pour stocker des coordonnées GPS (latitude, longitude) ?

A. Liste

B. Dictionnaire

C. Tuple

D. Ensemble

3 **[Vrai/Faux]** Les ensembles (`set`) conservent l'ordre d'insertion des éléments.

☐ Vrai ☐ Faux

4 **[QCM]** Quelle méthode permet de récupérer une valeur dans un dictionnaire sans risquer d'erreur si la clé n'existe pas ?

A. `dict.find(key)`

B. `dict.get(key, default)`

C. `dict[key]`

D. `dict.search(key)`

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

5 **[QCM]** Que retourne l'expression `[1, 2, 3] * 2` ?

A. `[2, 4, 6]`

B. `[1, 2, 3, 1, 2, 3]`

C. `6`

D. Erreur

6 **[Vrai/Faux]** Un tuple peut être utilisé comme clé dans un dictionnaire.

☐ Vrai ☐ Faux

7 **[QCM]** Quelle opération ensembliste correspond à `a & b` ?

A. Union

B. Intersection

C. Différence

D. Symmetric difference

8 **[Question ouverte]** Donnez un exemple concret où utiliser un `set` est plus efficace qu'une `list`.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

9 **[QCM]** Pourquoi la recherche d'un élément dans un `set` (ensemble) est-elle beaucoup plus rapide que dans une `list` (liste) ?

A. Parce que les sets sont triés automatiquement.

B. Parce que les sets utilisent une table de hachage (accès direct).

C. Parce que les sets sont plus petits en mémoire.

D. C'est faux, les listes sont plus rapides.

10 **[Question ouverte]** Écrivez le code pour créer un dictionnaire à partir de deux listes : clés = `["a", "b", "c"]`, valeurs = `[1, 2, 3]`.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Récapitulatif : Structures de données en Python

Listes [ ]

- Séquence **ordonnée** et **modifiable**
- Types mixtes : `[10, "Python", True]`
- Accès : `lst[0]`, `lst[-1]`
- Opérations : `append`, `pop`, `sort`
- Usage : tâches, notes, historique

Tuples ( )

- Séquence **ordonnée** et **immuable**
- Création : `t = (10, "Python", True)`
- Accès par index : `t[1]`
- Idéal pour : coordonnées, retours de fonction

Dictionnaires {clé:valeur}

- Paires **clé** → **valeur** (clés uniques)
- Accès instantané : `d["nom"]`
- Méthodes : `keys()`, `values()`, `get()`
- Usage : profils, configuration, cache

Ensembles set()

- Collection **non ordonnée** & **sans doublons**
- Création : `{1, 2, 3}` ou `set()`
- Opérations : Union (`|`), Inter (`&`)
- Usage : membres uniques, comparaison

Choix de la structure

- **Liste** : ordre important, modifiable
- **Tuple** : données fixes (coordonnées)
- **Dict** : accès par clé (recherche)
- **Set** : unicité (suppression doublons)

Outils d'exploration

- `dir(objet)` : voir les méthodes
- `help(type)` : voir la doc
- Ex : `dir(list)`, `help(dict)`

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

CHAPITRE V

MANIPULATION DES FICHIERS : STOCKER ET RÉCUPÉRER L'INFORMATION

◎ Objectifs d'apprentissage

À l'issue de ce module, vous serez capable de :

- ✓ **Ouvrir et fermer** des fichiers en toute sécurité
- ✓ **Lire et écrire** des données dans différents formats
- ✓ **Gérer les erreurs** courantes liées aux fichiers
- ✓ **Appliquer** ces concepts à des cas pratiques (logs, configurations, données)

Pourquoi utiliser des fichiers ?

Les fichiers sont essentiels pour :

- **Persister les données** au-delà de l'exécution du programme
- **Partager l'information** entre différents programmes
- **Travailler avec de grandes quantités** de données sans surcharger la RAM

› **Structurer et organiser** les informations (bases de données simples)

5.1 La mémoire volatile vs persistante

Jusqu'à présent, toutes nos données (variables, listes, dictionnaires) vivaient dans la **mémoire vive (RAM)**. C'est comme écrire sur un tableau blanc : c'est rapide, mais dès qu'on éteint l'ordinateur (ou ferme le programme), tout disparaît.

Mémoire Persistante

Un **fichier** est un conteneur d'informations stocké sur un disque dur (SSD/HDD). Contrairement à la RAM, son contenu **survit** à l'extinction de l'ordinateur. C'est l'équivalent d'un carnet de notes papier.

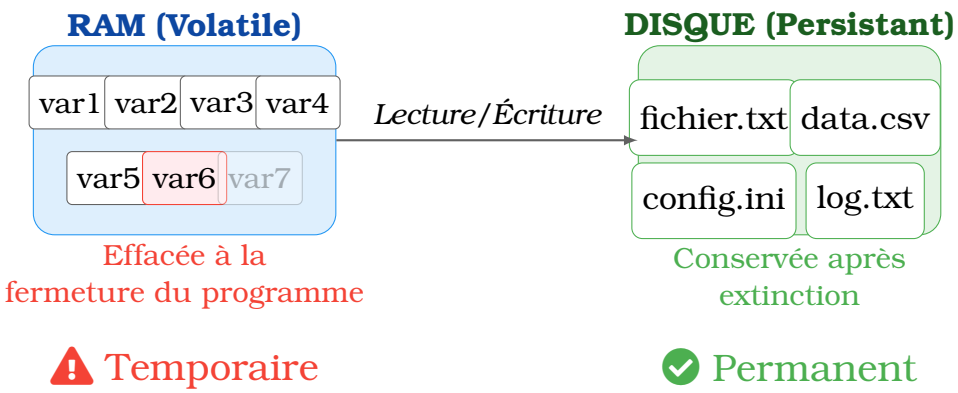


Figure 5.1 – *Comparaison entre mémoire volatile (RAM) et persistante (Disque)*

5.2 La fonction `open()` : La porte d'entrée

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Aspect	RAM (Mémoire volatile)	Disque (Mémoire persistante)
Vitesse	Très rapide (ns)	Plus lente (ms)
Capacité	Limitée (Go)	Immense (To)
Durée de vie des données	Temporaire (s'efface à l'extinction)	Permanente (reste après redémarrage)
Usage Python	Variables, listes, dictionnaires	Fichiers : .txt, .csv, .json, etc.

Table 5.1 – *Comparaison entre mémoire volatile et persistante*

Pour manipuler un fichier, la première étape est de l'ouvrir avec la fonction native `open()`.

In [1]:
```
## Syntaxe de base
fichier = open("nom_du_fichier.txt", "mode")

# Après utilisation, il faut TOUJOURS fermer le fichier
fichier.close()
```

5.2.1 Les modes d'ouverture essentiels

Le "mode" est une chaîne de caractères qui indique à Python ce que vous voulez faire avec le fichier.

5.2.2 Modes d'ouverture additionnels

On peut ajouter ces modificateurs aux modes de base :

- `'t'` (défaut) : Mode texte - pour les fichiers lisibles par l'humain

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

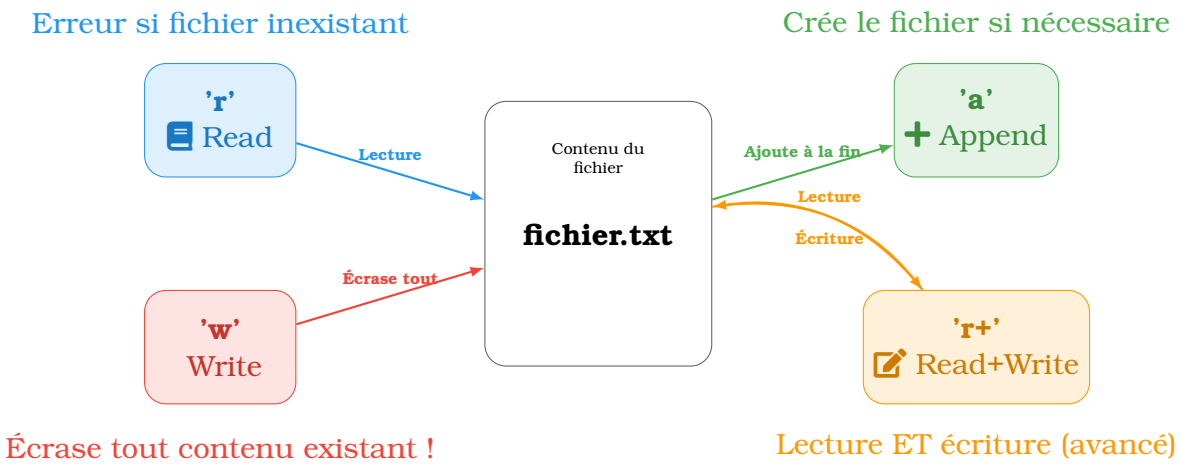


Figure 5.2 – *Modes d'ouverture des fichiers et leurs effets*

Mode	Nom	Description
`'r'`	**Read** (Lecture)	Ouvre le fichier pour lire. **Erreur** si le fichier n'existe pas.
`'w'`	**Write** (Écriture)	Crée un fichier vide. **ATTENTION :** Écrase tout le contenu existant !
`'a'`	**Append** (Ajout)	Ajoute du texte à la fin du fichier sans effacer l'ancien.
`'r+'`	**Read+Write**	Ouvre pour lire ET écrire. **Avancé** : le pointeur commence au début.

Table 5.2 – *Modes d'ouverture des fichiers en Python*

- `'b'` : Mode binaire - pour les fichiers non-textuels (images, PDF, etc.)
  - Exemple : `open('image.jpg', 'rb')` ouvre une image en lecture binaire

5.3 Le mot-clé `with` : La méthode recommandée

En informatique, oublier de fermer un fichier est une erreur classique (fuites de ressources). Python propose une syntaxe élégante qui ferme le fichier **automatiquement**, même en cas d'erreur.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

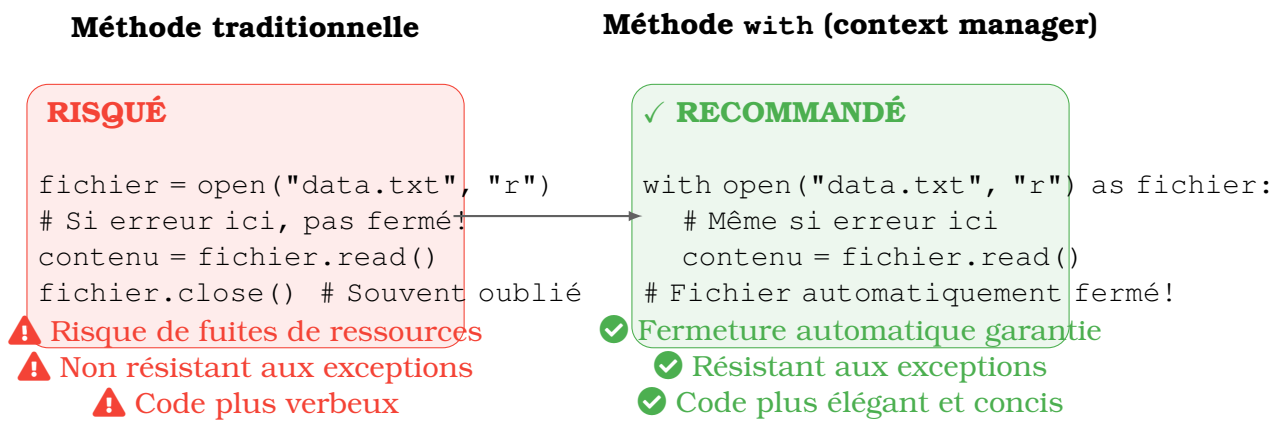


Figure 5.3 – *Comparaison entre méthode traditionnelle et gestionnaire de contexte with*

In [2]:
```
## Syntaxe recommandée - fermeture automatique
with open("mon_fichier.txt", "r") as fichier:
    # On travaille avec le fichier ici
    contenu = fichier.read()

# Ici, le fichier est AUTOMATIQUEMENT fermé. Pas besoin de fichier.close() !
print("Fichier fermé ?", fichier.closed)  # Affiche: True
```

Out [2]:
```
Fichier fermé ? True
```

Bonne pratique

Utilisez **toujours** la syntaxe `with` pour manipuler des fichiers. Elle garantit :

- La libération des ressources, même en cas d'erreur
- Un code plus lisible et plus robuste
- Moins de bugs liés aux fichiers non fermés

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

C'est comme "emprunter un livre à la bibliothèque" : vous l'utilisez, et dès que vous sortez du bloc `with`, vous le rendez automatiquement.

5.4 Lire un fichier

Supposons que nous ayons un fichier `poeme.txt` contenant :

```
Roses are red
Violets are blue
Python is awesome
And so are you
```

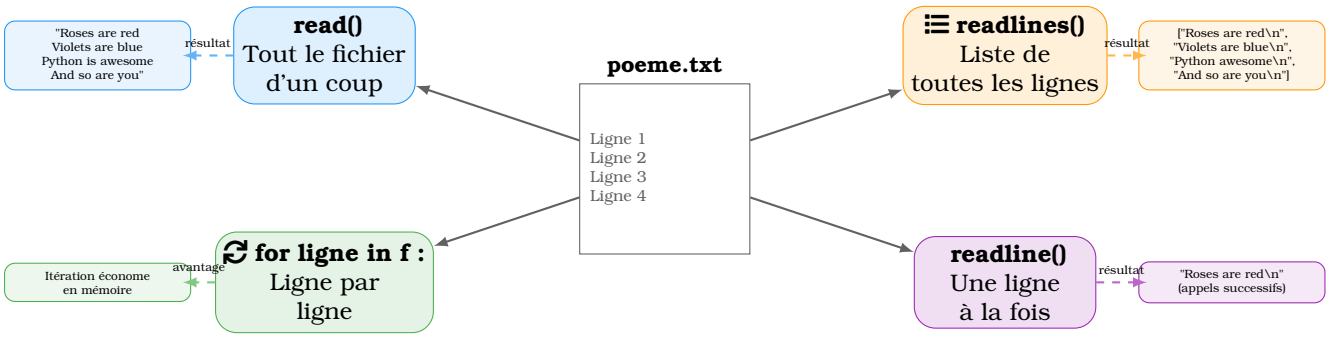


Figure 5.4 – *Différentes méthodes de lecture d'un fichier*

5.4.1 Méthode 1 : Tout lire d'un coup avec `read()`

Charge tout le fichier dans une seule grosse chaîne de caractères.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [3]:
```
with open("poeme.txt", "r") as fichier:
    texte = fichier.read()
    print(texte)
```

Out[3]:
```
Roses are red
Violets are blue
Python is awesome
And so are you
```

5.4.2 Méthode 2 : Lire ligne par ligne avec une boucle

C'est la méthode la plus efficace en mémoire. Idéale pour les gros fichiers.

In [4]:
```
with open("poeme.txt", "r") as fichier:
    for ligne in fichier:
        print(f"Ligne lue : {ligne.strip()}")  # .strip() enlève le saut de ligne
```

Out[4]:
```
Ligne lue : Roses are red
Ligne lue : Violets are blue
Ligne lue : Python is awesome
Ligne lue : And so are you
```

5.4.3 Méthode 3 : Lire toutes les lignes dans une liste

Utile quand vous avez besoin de manipuler l'ensemble des lignes.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
In [5]:
with open("poeme.txt", "r") as fichier:
    lignes = fichier.readlines()

print(f"Nombre de lignes: {len(lignes)}")
print(f"Deuxième ligne: {lignes[1].strip()}")
```

```
Out[5]:
Nombre de lignes: 4
Deuxième ligne: Violets are blue
```

⚠ Attention à la mémoire

Les méthodes `read()` et `readlines()` chargent tout le fichier en mémoire. Pour les fichiers volumineux (>100Mo), préférez la lecture ligne par ligne avec une boucle `for`.

5.5 Écrire et Ajouter

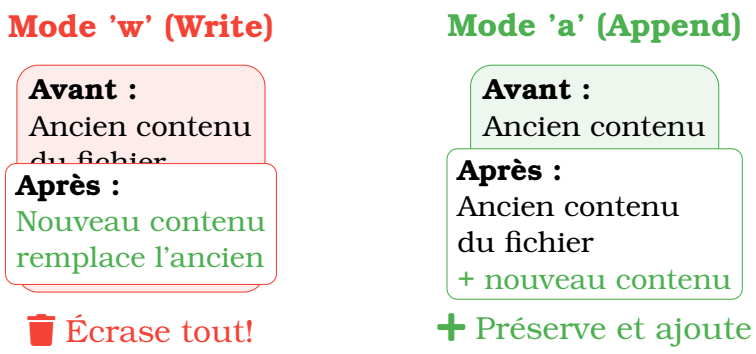


Figure 5.5 – *Différence entre modes d'écriture (w) et d'ajout (a)*

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

5.5.1 Le mode `'w'` (Write) : Écriture avec écrasement

Si le fichier existe, son contenu est **détruit** et remplacé. S'il n'existe pas, il est créé.

```
In [6]:
liste_courses = ["Pommes", "Bananes", "Lait"]

with open("courses.txt", "w") as fichier:
    for item in liste_courses:
        fichier.write(item + "\n")  # N'oubliez pas le \n
    pour sauter la ligne !
```

Important

`fichier.write()` n'ajoute PAS de saut de ligne automatique. Vous devez l'écrire explicitement avec `"\n"`.

5.5.2 Le mode `'a'` (Append) : Ajout à la fin

Ajoute à la fin sans effacer. Parfait pour des logs ou des historiques.

```
In [7]:
## On ajoute du pain à la liste existante
with open("courses.txt", "a") as fichier:
    fichier.write("Pain\n")
```

Contenu de courses.txt après exécution :

```
Pommes
Bananes
Lait
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
Pain
```

5.5.3 Une méthode pratique : `print()` avec redirection

On peut utiliser `print()` pour écrire dans un fichier, ce qui ajoute les sauts de ligne automatiquement.

```
In [8]:
1 with open("hello.txt", "w") as fichier:
2     print("Bonjour le monde !", file=fichier)
3     print("Cette ligne inclut automatiquement un saut de
    ligne.", file=fichier)
4     print("C'est plus pratique que write() !", file=fichier)
```

```
Bonjour le monde !
Cette ligne inclut automatiquement un saut de ligne.
C'est plus pratique que write() !
```

5.6 Gestion des erreurs courantes

De nombreuses erreurs peuvent survenir lors de la manipulation de fichiers :

```
In [9]:
1 ## Erreur 1: Le fichier n'existe pas (mode 'r')
2 try:
3     with open("fichier_inexistant.txt", "r") as fichier:
4         contenu = fichier.read()
5 except FileNotFoundError:
6     print("Erreur: Le fichier n'existe pas")
7
8 # Erreur 2: Problème de permission
9 try:
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

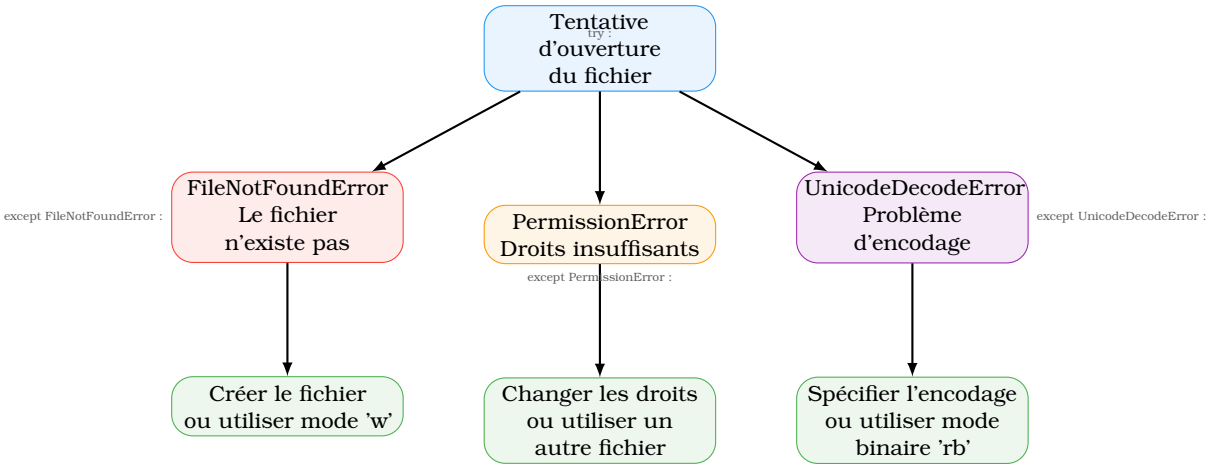


Figure 5.6 – *Arbre de décision pour gérer les erreurs de fichiers courantes*

```
    with open("/etc/shadow", "r") as fichier:  # Fichier système protégé
        contenu = fichier.read()
except PermissionError:
    print("Erreur: Vous n'avez pas les droits pour accéder à ce fichier")

# Erreur 3: Problème d'encodage
try:
    with open("fichier_binaire.jpg", "r") as fichier:  # Mode texte sur fichier binaire
        contenu = fichier.read()
except UnicodeDecodeError:
    print("Erreur: Ce n'est pas un fichier texte (utilisez 'rb')")
```

Out [9] :
```
Erreur: Le fichier n'existe pas Erreur: Vous n'avez pas les droits pour accéder à ce fichier Erreur: Ce n'est pas un fichier texte (utilisez 'rb')
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Solution robuste

```
In [10]:
## Modèle robuste de gestion des erreurs
try:
    with open("mon_fichier.txt", "r") as fichier:
        contenu = fichier.read()
    # Traitement du contenu
except FileNotFoundError:
    print("Le fichier n'existe pas, création d'un
    fichier vide")
    with open("mon_fichier.txt", "w") as fichier:
        fichier.write("")
except Exception as e:
    print(f"Une erreur inattendue s'est produite: {e}")
```

5.7 Travailler avec des formats courants

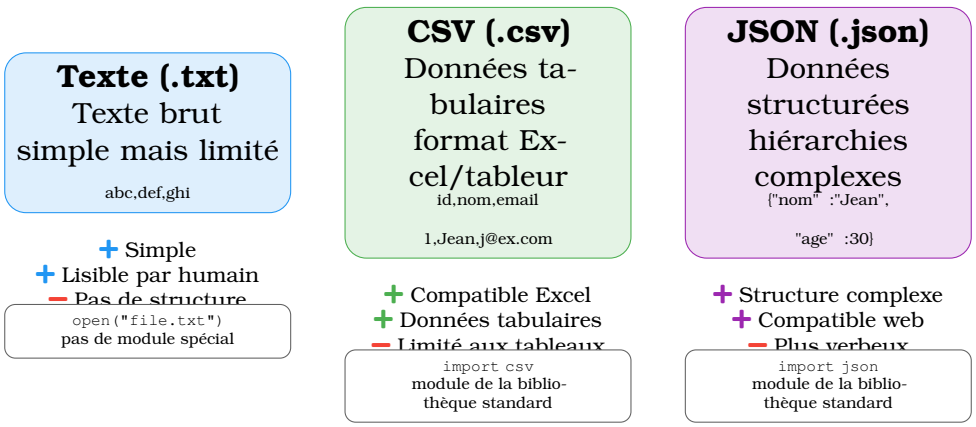


Figure 5.7 – *Comparaison des formats de fichiers courants*

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

5.7 .1 Fichiers CSV (Comma-Separated Values)

Les fichiers CSV sont parfaits pour stocker des données tabulaires (type Excel).

In [11]:
```python
import csv

# Écriture dans un CSV
with open('utilisateurs.csv', 'w', newline='') as csvfile:
    writer = csv.writer(csvfile)
    # Écriture de l'en-tête
    writer.writerow(['Nom', 'ge', 'Email'])
    # Écriture des données
    writer.writerow(['Alice', 28, 'alice@example.com'])
    writer.writerow(['Bob', 32, 'bob@example.com'])
    writer.writerow(['Charlie', 24, 'charlie@example.com'])

# Lecture d'un CSV
with open('utilisateurs.csv', 'r', newline='') as csvfile:
    reader = csv.reader(csvfile)
    print("Contenu du fichier CSV:")
    for i, row in enumerate(reader):
        if i == 0:
            print(f"En-tête: {row}")
        else:
            print(f"Utilisateur {i}: {row}")
```

Out[11]:
```
Contenu du fichier CSV:
En-tête: ['Nom', 'Âge', 'Email']
Utilisateur 1: ['Alice', '28', 'alice@example.com']
Utilisateur 2: ['Bob', '32', 'bob@example.com']
Utilisateur 3: ['Charlie', '24', 'charlie@example.com']
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

5.7.2 Fichiers JSON (JavaScript Object Notation)

Le format JSON est idéal pour stocker des structures de données complexes comme des dictionnaires et des listes imbriquées.

In [12]:

```
import json

# Données à sauvegarder (structures complexes)
donnees = {
    "utilisateurs": [
        {
            "nom": "Alice",
            "age": 28,
            "email": "alice@example.com",
            "actif": True,
            "competences": ["Python", "JavaScript", "SQL"]
        },
        {
            "nom": "Bob",
            "age": 32,
            "email": "bob@example.com",
            "actif": False,
            "competences": ["Java", "C++"]
        }
    ],
    "version": 1.0
}

# Écriture dans un fichier JSON (sérialisation)
with open('donnees.json', 'w') as f:
    json.dump(donnees, f, indent=4)  # indent pour une mise en forme lisible

# Lecture d'un fichier JSON (désérialisation)
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
with open('donnees.json', 'r') as f:
    donnees_chargees = json.load(f)

# Affichage des données chargées
print(f"Version des données: {donnees_chargees['version']}")
print(f"Nombre d'utilisateurs:
    {len(donnees_chargees['utilisateurs'])}")
print(f"Compétences de
    {donnees_chargees['utilisateurs'][0]['nom']}: {',
    '.join(donnees_chargees['utilisateurs'][0]['competences'])}")
```

Out[12]:
```
Version des données: 1.0
Nombre d'utilisateurs: 2 Compétences de Alice: Python,
JavaScript, SQL
```

Points clés à retenir

1. Utilisez **toujours** le gestionnaire de contexte `with` pour ouvrir les fichiers
2. Connaissez les 3 modes principaux : `'r'` (lecture), `'w'` (écriture), `'a'` (ajout)
3. Choisissez la bonne méthode de lecture selon la taille du fichier :
   - Petit fichier : `read()` ou `readlines()`
   - Grand fichier : parcours ligne par ligne avec une boucle `for`
4. N'oubliez pas les `"\n"` quand vous utilisez `write()`
5. Utilisez `try/except` pour gérer les erreurs courantes
6. Pour des structures complexes, préférez CSV ou JSON aux fichiers texte brut

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

5.8 Travaux Pratiques du Module 5

Travaux Pratiques : Manipulation des Fichiers

TP 1 – Journal personnel

Créez un journal personnel simple qui permet à l'utilisateur de :

- Ajouter des entrées datées à un fichier journal
- Lire toutes les entrées précédentes
- Rechercher des entrées contenant un mot-clé

Fonctionnalités requises :

- Utiliser le mode `'a'` pour ajouter des entrées
- Enregistrer automatiquement la date et l'heure de chaque entrée
- Séparer clairement les entrées dans le fichier

Exemple d'interface :

```
Journal personnel
1. Ajouter une entrée
2. Lire toutes les entrées
3. Rechercher dans les entrées
4. Quitter
Votre choix : 1

Saisissez votre entrée : J'ai appris la manipulation de fichiers !
Entrée enregistrée !
```

Bonus : Ajoutez une fonction pour effacer le journal.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

TP 2 – Gestionnaire de contacts

Créez un gestionnaire de contacts qui stocke les informations dans un fichier texte :

- Chaque contact a un nom, téléphone et email
- Format de stockage : une ligne par contact avec des séparateurs
- Fonctions : ajouter, rechercher, supprimer, lister tous les contacts

Exemple de fichier de contacts :

```
Alice Dupont:0601020304:alice@example.com
Bob Martin:0706050403:bob@example.com
Charlie Dubois:0712345678:charlie@example.com
```

Recherche à implémenter :

```
# Exemple d'utilisation de la fonction de recherche
def rechercher_contact(nom):
    with open("contacts.txt", "r") as f:
        for ligne in f:
            if nom.lower() in ligne.lower():
                nom, tel, email = ligne.strip().split(":")
                print(f"Contact trouvé : {nom} - {tel} - {email}")
                return
    print(f"Aucun contact trouvé pour '{nom}'")
```

Bonus : Assurez-vous que les contacts sont uniques (pas de doublons).

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

TP 3 – Analyseur de logs serveur web

Analysez un fichier de logs de serveur web pour extraire des statistiques :

- Téléchargez ou créez un exemple de fichier log (format simplifié)
- Analysez le nombre de visites par page
- Identifiez les adresses IP les plus fréquentes
- Créez un rapport dans un nouveau fichier

Exemple de fichier log :

```
192.168.1.1 - - [10/Jan/2023:13:55:36] "GET /index.html HTTP/1.1" 200
192.168.1.2 - - [10/Jan/2023:13:56:12] "GET /about.html HTTP/1.1" 200
192.168.1.1 - - [10/Jan/2023:13:56:55] "GET /contact.html HTTP/1.1" 2
192.168.1.3 - - [10/Jan/2023:13:57:10] "GET /index.html HTTP/1.1" 200
192.168.1.1 - - [10/Jan/2023:13:58:22] "GET /index.html HTTP/1.1" 200
```

Exemple de rapport attendu :

```
RAPPORT D'ANALYSE DE LOGS - 10/Jan/2023

Pages les plus visitées:
- /index.html : 3 visites
- /about.html : 1 visite
- /contact.html : 1 visite

Visiteurs par IP:
- 192.168.1.1 : 3 visites
- 192.168.1.2 : 1 visite
- 192.168.1.3 : 1 visite
```

Bonus : Ajoutez des statistiques sur les heures de visite.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

TP 4 – Gestion d'inventaire avec CSV

Développez un système de gestion d'inventaire utilisant un fichier CSV :

- Stockez produits avec ID, nom, quantité, prix
- Utilisez le module `csv` pour lire/écrire
- Fonctions : ajouter/supprimer produit, mettre à jour quantité, afficher catalogue

Structure du fichier CSV :

```
id,nom,quantite,prix
P001,Clavier sans fil,15,45.99
P002,Souris Bluetooth,23,29.99
P003,Écran 24 pouces,8,159.99
```

Exemple de fonction pour afficher l'inventaire :

```
import csv

def afficher_inventaire():
    try:
        with open("inventaire.csv", "r", newline="") as fichier:
            lecteur = csv.DictReader(fichier)
            print(f"{'ID':<5} {'Nom':<25} {'Quantité':<10} {'Prix':<1
            print("-" * 55)
            for produit in lecteur:
                print(f"{produit['id']:<5} {produit['nom']:<25} {prod
    except FileNotFoundError:
        print("Le fichier d'inventaire n'existe pas encore.")
```

Bonus : Ajoutez une fonction de recherche par nom de produit.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

TP 5 – Mini système de gestion d'étudiants

Créez un système complet de gestion d'étudiants avec JSON :

- Stockez les données étudiants dans un fichier JSON
- Pour chaque étudiant : ID, nom, cours suivis, notes
- Fonctionnalités : ajouter/supprimer étudiants, ajouter notes
- Créez des fonctions pour sauvegarder/charger les données

Exemple de structure JSON (simplifié) :

```
{
  "etudiants": [
    {
      "id": "E001",
      "nom": "Alice Dupont",
      "cours": {
        "Maths": 18.5,
        "Info": 16.0
      }
    }
  ]
}
```

Fonctions principales à implémenter :

- `charger_donnees()` et `sauvegarder_donnees(donnees)`
- `ajouter_etudiant(id, nom)`
- `ajouter_note(id_etudiant, cours, note)`
- `calculer_moyenne(id_etudiant)`

Bonus : Interface console interactive.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

5.9 Quiz du Module 5

1 **[Vrai/Faux]** La fonction `open()` en mode `'w'` crée un fichier s'il n'existe pas et écrase son contenu s'il existe.

☐ Vrai ☐ Faux

2 **[QCM]** Quel est le mode d'ouverture correct pour ajouter du contenu à la fin d'un fichier sans effacer son contenu existant ?

A. `'r'`

B. `'w'`

C. `'a'`

D. `'x'`

3 **[Vrai/Faux]** La méthode `write()` d'un fichier ajoute automatiquement un saut de ligne à la fin du texte écrit.

☐ Vrai ☐ Faux

4 **[QCM]** Quelle méthode permet de lire toutes les lignes d'un fichier dans une liste ?

A. `file.read()`

B. `file.readall()`

C. `file.readlines()`

D. `file.getlines()`

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

5 **[QCM]** Quelle est la bonne façon d'utiliser le gestionnaire de contexte pour ouvrir un fichier ?

A. `with open("file.txt") as f:`

B. `using open("file.txt") as f:`

C. `with f = open("file.txt"):`

D. `using f = open("file.txt"):`

6 **[Vrai/Faux]** Pour lire un fichier binaire comme une image, vous devez utiliser le mode `'rb'` lors de l'ouverture.

☐ Vrai ☐ Faux

7 **[QCM]** Quelle exception est levée lorsqu'on essaie d'ouvrir en lecture un fichier qui n'existe pas ?

A. `IOError`

B. `FileNotFoundError`

C. `FileOpenError`

D. `ReadError`

8 **[QCM]** Quel module Python doit-on importer pour travailler avec des fichiers CSV ?

A. `csvfile`

B. `csv`

C. `excel`

D. `table`

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

9 **[QCM]** Quelle est la fonction utilisée pour charger des données JSON à partir d'un fichier en Python ?

A. `json.parse()`

B. `json.read()`

C. `json.load()`

D. `json.decode()`

10 **[Question ouverte]** Écrivez le code Python permettant de compter le nombre de lignes dans un fichier texte nommé "donnees.txt".

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Récapitulatif : Manipulation des fichiers en Python

Ouverture

- `with open(fichier, mode) as f:`
- Fermeture automatique du fichier
- Gestion des erreurs intégrée

Modes

- `'r'` : lecture (erreur si absent)
- `'w'` : écriture (écrase tout)
- `'a'` : ajout (à la fin)
- `'rb'`, `'wb'` : binaire

Lecture

- `f.read()` : tout le fichier
- `f.readlines()` : liste de lignes
- `for ligne in f:` : ligne par ligne

Écriture

- `f.write("texte")` : sans \n
- `print("texte", file=f)` : avec \n
- `f.writelines(liste)` : sans \n

Gestion d'erreurs

- `FileNotFoundError` : fichier absent
- `PermissionError` : droits insuffisants
- `UnicodeDecodeError` : encodage

Formats spéciaux

- `csv` : `csv.reader()`, `csv.writer()`
- `json` : `json.load()`, `json.dump()`

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

CHAPITRE VI

INTRODUCTION AU DÉBOGAGE ET BONNES PRATIQUES

Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

- ✔ Identifier et corriger les erreurs dans vos programmes Python
- ✔ Utiliser les structures `try/except` pour gérer les exceptions
- ✔ Importer et utiliser des bibliothèques Python
- ✔ Documenter votre code avec des commentaires et docstrings
- ✔ Appliquer les conventions PEP 8 pour un code professionnel

6.1 Pourquoi le débogage est essentiel ?

Le débogage (debugging)

Le **débogage** est le processus de recherche et de correction des **bugs** (erreurs) dans un programme informatique. Le terme vient de l'anglais « bug » (insecte), car les premiers bugs informatiques étaient parfois causés par de vrais insectes coincés dans les circuits !

Le saviez-vous ?

En 1947, Grace Hopper, pionnière de l'informatique, a trouvé un vrai papillon de nuit coincé dans un relais de l'ordinateur Harvard Mark II. Elle a collé l'insecte dans son journal de bord avec la note : « First actual case of bug being found » (Premier cas réel de bug trouvé). C'est l'origine du terme « debugging » !

6.1 .1 Les erreurs font partie de l'apprentissage

Message important pour les débutants

Faire des erreurs est NORMAL et même SOUHAITABLE ! Chaque erreur est une opportunité d'apprentissage. Les programmeurs expérimentés passent environ 50% de leur temps à déboguer. Ne vous découragez jamais face à une erreur.

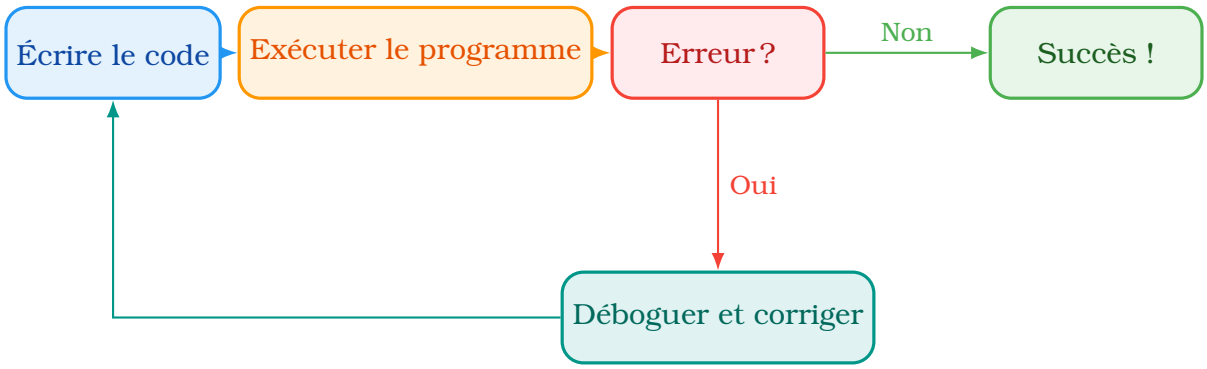


Figure 6.1 – Le cycle de développement : écrire, tester, déboguer, répéter

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

6.1.2 Cas concret : La recette de cuisine

Imaginez que vous suivez une recette de gâteau au chocolat :

Analogie avec la cuisine

1. Vous suivez la recette (= écrire le code)
2. Le gâteau sort du four (= exécuter le programme)
3. Le gâteau est brûlé ! (= erreur/bug)
4. Vous analysez : « Ah, j'ai mis 200°C au lieu de 180°C » (= débogage)
5. Vous corrigez et recommencez (= correction du bug)

En programmation, c'est exactement pareil : on identifie ce qui ne va pas, on corrige, et on réessaye.

6.2 Les trois types d'erreurs en Python

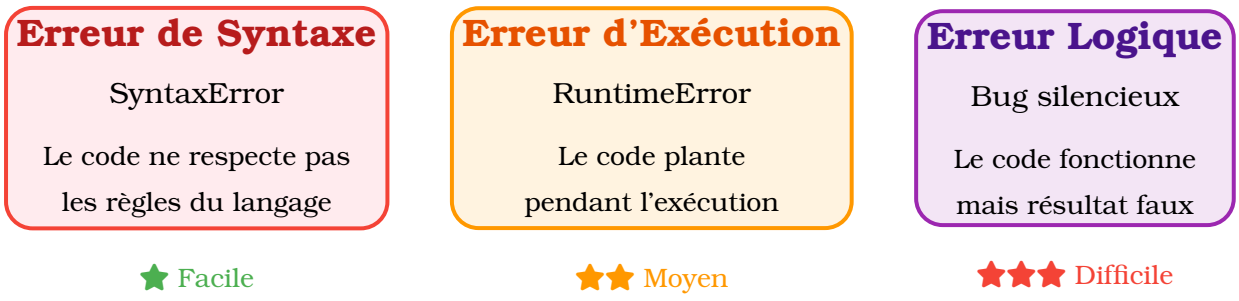


Figure 6.2 – Les trois types d'erreurs en programmation

6.2.1 Erreurs de syntaxe (SyntaxError)

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Erreur de syntaxe

Une **erreur de syntaxe** se produit quand le code ne respecte pas les règles grammaticales de Python. C'est comme écrire une phrase sans respecter la grammaire : Python ne comprend pas ce que vous voulez dire.

6.2.1.1 Exemple 1 : Oublier les deux-points

In [1]:
```
## ERREUR : il manque les deux-points apres la condition
x = 10
if x > 5
    print("x est grand")
```

Out[1]:
```
  File "<stdin>", line 3
    if x > 5
           ^
SyntaxError: expected ':'
```

In [2]:
```
## CORRECTION : ajouter les deux-points
x = 10
if x > 5:
    print("x est grand")
```

Out[2]:
```
x est grand
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

6.2.1.2 Exemple 2 : Parenthèses non fermées

In [3]:
```
## ERREUR : parenthese non fermee
print("Bonjour le monde"
```

Out[3]:
```
  File "<stdin>", line 2
    print("Bonjour le monde"
                            ^
SyntaxError: '(' was never closed
```

In [4]:
```
## CORRECTION : fermer la parenthese
print("Bonjour le monde")
```

Out[4]:
```
Bonjour le monde
```

6.2.1.3 Exemple 3 : Mauvaise indentation

⚠ L'indentation est cruciale en Python !

Contrairement à d'autres langages, Python utilise l'indentation (les espaces en début de ligne) pour délimiter les blocs de code. Une mauvaise indentation provoque une erreur `IndentationError`.

In [5]:
```
## ERREUR : indentation manquante
for i in range(3):
print(i)
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Out [5]:
```
  File "<stdin>", line 3
    print(i)
    ^
IndentationError: expected an indented block
```

In [6]:
```
1 ## CORRECTION : ajouter 4 espaces d'indentation
2 for i in range(3):
3     print(i)
```

Out [6]:
```
0
1
2
```

6.2 .2 Erreurs d'exécution (RuntimeError)

Erreur d'exécution

Une **erreur d'exécution** se produit quand le code est syntaxiquement correct, mais qu'une opération impossible est demandée pendant l'exécution du programme.

6.2.2 .1 Exemple 1 : Division par zéro

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Cas concret : Calcul de moyenne

Vous créez un programme pour calculer la moyenne d'une classe. Mais que se passe-t-il si aucun élève n'a passé l'examen ?

In [7]:
```
## Calcul de moyenne avec 0 eleves
total_notes = 0
nombre_eleves = 0

moyenne = total_notes / nombre_eleves
print(f"La moyenne est : {moyenne}")
```

Out[7]:
```
Traceback (most recent call last):
File "<stdin>", line 5, in <module>
ZeroDivisionError: division by zero
```

6.2.2 .2 Exemple 2 : Variable non définie

In [8]:
```
## ERREUR : utiliser une variable qui n'existe pas
print(message)
```

Out[8]:
```
Traceback (most recent call last):
File "<stdin>", line 2, in <module>
NameError: name 'message' is not defined
```

In [9]:
```
## CORRECTION : definir la variable avant de l'utiliser
message = "Bonjour !"
print(message)
```

Out[9]:
```
Bonjour !
```

6.2.2 .3 Exemple 3 : Index hors limites

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Cas concret : Liste de courses

Vous avez une liste de 3 articles, mais vous essayez d'accéder au 4ème élément.

In [10]:
```
## Liste de courses avec 3 elements
courses = ["pain", "lait", "oeufs"]

# Afficher les elements
print("Premier element :", courses[0])
print("Deuxieme element :", courses[1])
print("Troisieme element :", courses[2])

# ERREUR : l'index 3 n'existe pas
print("Quatrieme element :", courses[3])
```

Out[10]:
```
Premier element : pain
Deuxieme element : lait
Troisieme element : oeufs
Traceback (most recent call last):
File "<stdin>", line 10, in <module>
IndexError: list index out of range
```

6.2.2 .4 Exemple 4 : Erreur de type

In [11]:
```
## ERREUR : soustraire une chaine d'un nombre
age = "25"
annee_naissance = 2024 - age
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
Out[11]:
Traceback (most recent call last):
File "<stdin>", line 3, in <module>
TypeError: unsupported operand type(s) for -: 'int' and 'str'
```

```
In [12]:
## CORRECTION : convertir la chaine en entier
age = "25"
annee_naissance = 2024 - int(age)
print(f"Annee de naissance : {annee_naissance}")
```

```
Out[12]:
Annee de naissance : 1999
```

6.2 .3 Erreurs logiques (bugs silencieux)

Les erreurs les plus dangereuses !

Les **erreurs logiques** sont les plus difficiles à détecter car le programme fonctionne sans planter, mais produit un résultat incorrect. Python ne peut pas vous aider : c'est à vous de vérifier que votre logique est correcte.

6.2.3 .1 Exemple 1 : Erreur dans une formule

Cas concret : Calcul de TVA

Vous créez un programme pour calculer le prix TTC d'un produit avec 20% de TVA.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [13]:
```
## ERREUR LOGIQUE : mauvaise formule pour la TVA
prix_ht = 100

# Formule incorrecte : on ajoute 20 au lieu de 20%
prix_ttc = prix_ht + 20

print(f"Prix HT : {prix_ht} EUR")
print(f"Prix TTC : {prix_ttc} EUR")
```

Out[13]:
```
Prix HT : 100 EUR
Prix TTC : 120 EUR
```

Attention

Le résultat semble correct pour 100 EUR, mais essayons avec 50 EUR...

In [14]:
```
## Le bug devient visible avec un autre prix
prix_ht = 50

# Toujours la meme erreur
prix_ttc = prix_ht + 20

print(f"Prix HT : {prix_ht} EUR")
print(f"Prix TTC : {prix_ttc} EUR")
```

Out[14]:
```
Prix HT : 50 EUR
Prix TTC : 70 EUR
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Information

Le prix TTC de 50 EUR devrait être 60 EUR (50 + 20%), pas 70 EUR!

In [15]:
```
## CORRECTION : utiliser le pourcentage correctement
prix_ht = 50
taux_tva = 0.20

prix_ttc = prix_ht * (1 + taux_tva)

print(f"Prix HT : {prix_ht} EUR")
print(f"TVA (20%) : {prix_ht * taux_tva} EUR")
print(f"Prix TTC : {prix_ttc} EUR")
```

Out[15]:
```
Prix HT : 50 EUR
TVA (20
Prix TTC : 60.0 EUR
```

6.2.3 .2 Exemple 2 : Erreur de condition

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [16]:
```python
## ERREUR LOGIQUE : mauvais operateur de comparaison
def verifier_majorite(age):
    if age > 18:
        return "Majeur"
    else:
        return "Mineur"

print(f"17 ans : {verifier_majorite(17)}")
print(f"18 ans : {verifier_majorite(18)}")
print(f"19 ans : {verifier_majorite(19)}")
```

Out[16]:
```
17 ans : Mineur
18 ans : Mineur
19 ans : Majeur
```

Attention

Une personne de 18 ans est majeure, mais le programme dit « Mineur » ! Le bug vient de `>` au lieu de `>=`.

In [17]:
```python
## CORRECTION : utiliser >= pour inclure 18 ans
def verifier_majorite(age):
    if age >= 18:
        return "Majeur"
    else:
        return "Mineur"

print(f"17 ans : {verifier_majorite(17)}")
print(f"18 ans : {verifier_majorite(18)}")
print(f"19 ans : {verifier_majorite(19)}")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
Out[17]:
    17 ans : Mineur
    18 ans : Majeur
    19 ans : Majeur
```

6.3 Techniques de débogage de base

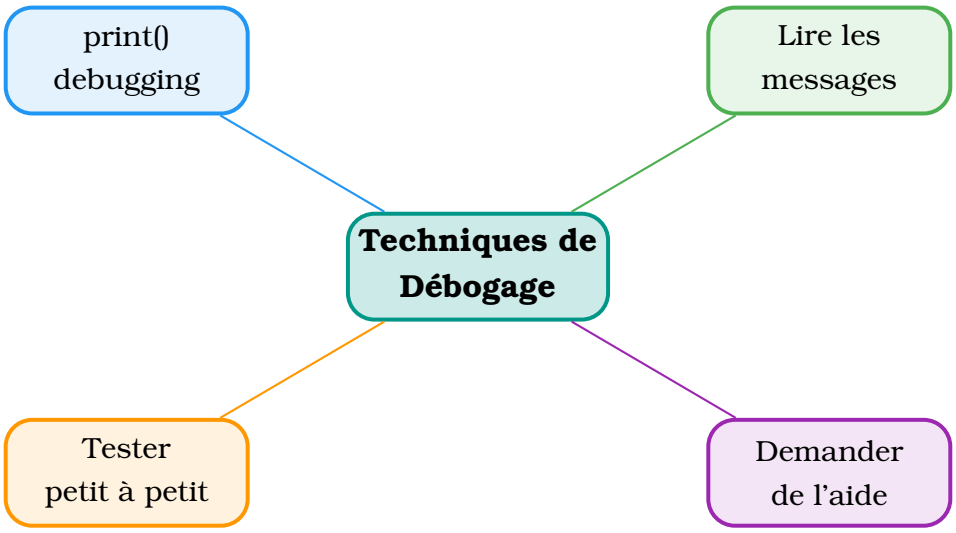


Figure 6.3 – Les quatre techniques de débogage essentielles

6.3.1 Technique 1 : Le débogage par print()

La méthode la plus simple et la plus utilisée

La technique du **print debugging** consiste à ajouter des instructions `print()` dans votre code pour afficher la valeur des variables à différents moments de l'exécution. C'est simple mais très efficace !

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Cas concret : Calculer une facture

Votre programme de facturation donne un total incorrect. Utilisez `print()` pour comprendre ce qui se passe.

In [18]:
```python
## Programme de facturation avec un bug
def calculer_facture(articles, quantites, prix):
    total = 0
    for i in range(len(articles)):
        sous_total = quantites[i] + prix[i]
        total += sous_total
    return total

articles = ["Stylo", "Cahier", "Gomme"]
quantites = [5, 3, 2]
prix = [1.50, 4.00, 0.80]

resultat = calculer_facture(articles, quantites, prix)
print(f"Total : {resultat} EUR")
```

Out[18]:
```
Total : 16.3 EUR
```

Attention

Le résultat devrait être 21.10 EUR! Ajoutons des `print()` pour déboguer :

In [19]:
```python
## Version avec print() de debogage
def calculer_facture(articles, quantites, prix):
    total = 0
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
    print("=== DEBUT DU DEBOGAGE ===")

    for i in range(len(articles)):
        print(f"Article : {articles[i]}")
        print(f"  Quantite : {quantites[i]}")
        print(f"  Prix unitaire : {prix[i]}")

        sous_total = quantites[i] + prix[i]
        print(f"  Sous-total calcule : {sous_total}")
        print(f"  Sous-total attendu : {quantites[i] *
    prix[i]}")

        total += sous_total

    print(f"=== Total final : {total} ===")
    return total

articles = ["Stylo", "Cahier", "Gomme"]
quantites = [5, 3, 2]
prix = [1.50, 4.00, 0.80]

resultat = calculer_facture(articles, quantites, prix)
```

Out[19]:

```
=== DEBUT DU DEBOGAGE ===
Article : Stylo
Quantite : 5
Prix unitaire : 1.5
Sous-total calcule : 6.5
Sous-total attendu : 7.5
Article : Cahier
Quantite : 3
Prix unitaire : 4.0
Sous-total calcule : 7.0
Sous-total attendu : 12.0
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
Article : Gomme
Quantite : 2
Prix unitaire : 0.8
Sous-total calcule : 2.8
Sous-total attendu : 1.6
=== Total final : 16.3 ===
```

Bug identifié !

On voit clairement le problème : on fait `quantites[i] + prix[i]` au lieu de `quantites[i] * prix[i]`. On additionne au lieu de multiplier !

In [20]:
```python
## Version corrigee
def calculer_facture(articles, quantites, prix):
    total = 0
    for i in range(len(articles)):
        sous_total = quantites[i] * prix[i]
        total += sous_total
    return total

articles = ["Stylo", "Cahier", "Gomme"]
quantites = [5, 3, 2]
prix = [1.50, 4.00, 0.80]

resultat = calculer_facture(articles, quantites, prix)
print(f"Total : {resultat} EUR")
```

Out[20]:
```
Total : 21.1 EUR
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

6.3.2 Technique 2 : Lire et comprendre les messages d'erreur

Le Traceback

Quand Python rencontre une erreur, il affiche un **traceback** : un rapport détaillé qui indique exactement où l'erreur s'est produite et de quel type elle est.

Comment lire un traceback

1. **Commencez par la fin** : le type d'erreur et sa description
2. **Regardez le numéro de ligne** : où l'erreur s'est produite
3. **Examinez le code affiché** : la ligne qui pose problème
4. **Remontez si nécessaire** : pour comprendre l'enchaînement des appels

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Type d'erreur	Cause	Solution
`SyntaxError`	Faute de frappe, ponctuation	Vérifier :, parenthèses
`NameError`	Variable non définie	Définir ou corriger le nom
`TypeError`	Opération incompatible	Convertir avec int(), str()
`IndexError`	Index hors limites	Vérifier avec len()
`KeyError`	Clé inexistante (dict)	Utiliser .get()
`ZeroDivisionErr`	Division par zéro	Vérifier le diviseur
`FileNotFoundError`	Fichier introuvable	Vérifier le chemin
`ValueError`	Valeur inappropriée	Valider les entrées

Tableau 6.1 – Les erreurs Python les plus courantes et leurs solutions

6.3 .3 Technique 3 : Tester petit à petit

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Diviser pour mieux régner

Ne testez jamais tout votre programme d'un coup ! Écrivez quelques lignes, testez, puis continuez. C'est plus facile de trouver un bug dans 5 lignes que dans 500.

Cas concret : Construire un programme de notes

Plutôt que d'écrire tout le programme d'un coup, construisez-le étape par étape :

In [21]:
```
## ETAPE 1 : Tester la saisie des notes
print("=== Etape 1 : Saisie ===")
notes = [15, 12, 18, 9, 14]
print(f"Notes saisies : {notes}")
print("Etape 1 OK !")
```

Out[21]:
```
=== Etape 1 : Saisie ===
Notes saisies : [15, 12, 18, 9, 14]
Etape 1 OK !
```

In [22]:
```
## ETAPE 2 : Tester le calcul de la moyenne
print("=== Etape 2 : Moyenne ===")
notes = [15, 12, 18, 9, 14]
moyenne = sum(notes) / len(notes)
print(f"Moyenne : {moyenne}")
print("Etape 2 OK !")
```

Out[22]:
```
=== Etape 2 : Moyenne ===
Moyenne : 13.6
Etape 2 OK !
```

In [23]:
```
## ETAPE 3 : Tester la recherche du max et min
print("=== Etape 3 : Max et Min ===")
notes = [15, 12, 18, 9, 14]
note_max = max(notes)
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
note_min = min(notes)
print(f"Meilleure note : {note_max}")
print(f"Plus basse note : {note_min}")
print("Etape 3 OK !")
```

Out[23]:
```
=== Etape 3 : Max et Min ===
Meilleure note : 18
Plus basse note : 9
Etape 3 OK !
```

In [24]:
```
## ETAPE 4 : Assembler le programme complet
print("=== BULLETIN DE NOTES ===")

notes = [15, 12, 18, 9, 14]
matieres = ["Maths", "Francais", "Anglais", "Histoire",
    "Sciences"]

for i in range(len(notes)):
    print(f"{matieres[i]} : {notes[i]}/20")

print("\n--- Statistiques ---")
print(f"Moyenne generale : {sum(notes)/len(notes):.2f}/20")
print(f"Meilleure note : {max(notes)}/20")
print(f"Plus basse note : {min(notes)}/20")
```

Out[24]:
```
=== BULLETIN DE NOTES ===
Maths : 15/20
Francais : 12/20
Anglais : 18/20
Histoire : 9/20
Sciences : 14/20
-- Statistiques --
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
Moyenne generale : 13.60/20
Meilleure note : 18/20
Plus basse note : 9/20
```

6.4 Gestion des exceptions avec try/except

Exception

Une **exception** est un événement qui interrompt le flux normal d'exécution d'un programme. Plutôt que de laisser le programme planter, Python permet de « capturer » ces exceptions et de les gérer proprement.

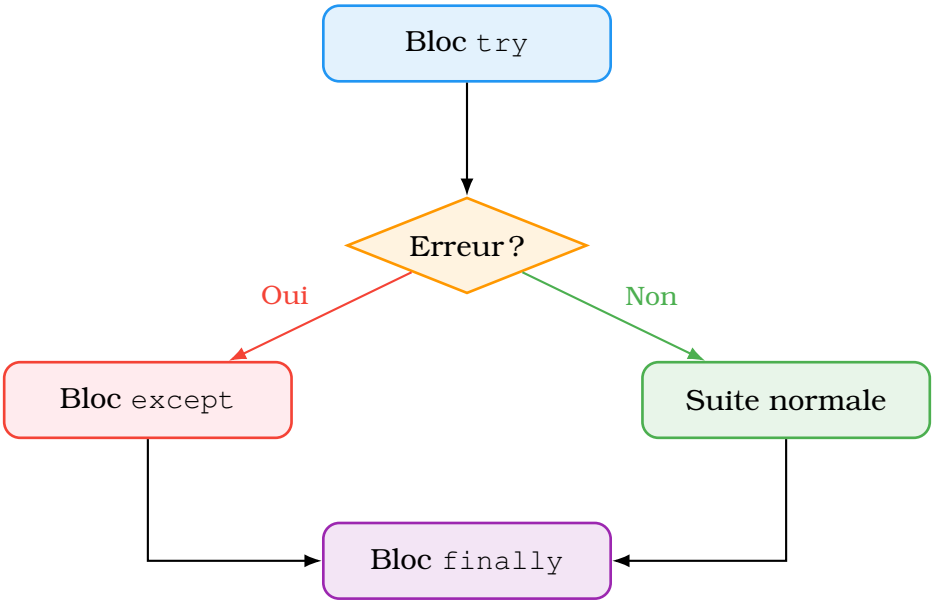


Figure 6.4 – Fonctionnement de try/except/finally

6.4.1 Syntaxe de base : try/except

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [25]:
```
## Syntaxe de base
try:
    resultat = 10 / 0
except:
    print("Une erreur s'est produite !")
```

Out[25]:
```
Une erreur s'est produite !
```

Attention

Utiliser `except` sans préciser le type d'erreur capture TOUTES les erreurs. C'est déconseillé car cela peut masquer des bugs inattendus.

6.4.2 Capturer des types d'erreurs spécifiques

In [26]:
```
## Capturer une erreur specifique
try:
    nombre = int("abc")
except ValueError:
    print("Erreur : vous devez entrer un nombre entier !")
```

Out[26]:
```
Erreur : vous devez entrer un nombre entier !
```

Cas concret : Calculatrice sécurisée

Créons une calculatrice qui gère les erreurs de saisie et la division par zéro.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [27]:
```
## Simulation : 10 / 0
try:
    a = 10
    b = 0
    resultat = a / b
    print(f"Resultat : {resultat}")
except ZeroDivisionError:
    print("Erreur : impossible de diviser par zero !")
```

Out[27]:
```
Erreur : impossible de diviser par zero !
```

In [28]:
```
## Simulation : 10 / 2
try:
    a = 10
    b = 2
    resultat = a / b
    print(f"Resultat : {resultat}")
except ZeroDivisionError:
    print("Erreur : impossible de diviser par zero !")
```

Out[28]:
```
Resultat : 5.0
```

6.4.3 Capturer plusieurs types d'erreurs

In [29]:
```
def acceder_element(liste, index):
    try:
        element = liste[index]
        return element
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
    except IndexError:
        print(f"Erreur : l'index {index} n'existe pas")
        return None
    except TypeError:
        print(f"Erreur : l'index doit etre un entier")
        return None

ma_liste = ["a", "b", "c"]

print("Test 1 :", acceder_element(ma_liste, 1))
print("Test 2 :", acceder_element(ma_liste, 10))
print("Test 3 :", acceder_element(ma_liste, "deux"))
```

Out[29]:
```
Test 1 : b
Erreur : l'index 10 n'existe pas
Test 2 : None
Erreur : l'index doit etre un entier
Test 3 : None
```

6.4 .4 Les blocs else et finally

else et finally

- **else** : exécuté si AUCUNE exception n'a été levée
- **finally** : TOUJOURS exécuté, qu'il y ait une exception ou non

In [30]:
```
def diviser(a, b):
    try:
        resultat = a / b
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
    except ZeroDivisionError:
        print("Erreur : division par zero !")
        return None
    else:
        print("Division reussie !")
        return resultat
    finally:
        print("Operation terminee.")

print("=== Test 1 ===")
r = diviser(10, 2)
print(f"Resultat : {r}")

print("\n=== Test 2 ===")
r = diviser(10, 0)
print(f"Resultat : {r}")
```

Out[30]:
```
=== Test 1 ===
Division reussie !
Operation terminee.
Resultat : 5.0
=== Test 2 ===
Erreur : division par zero !
Operation terminee.
Resultat : None
```

6.4.5 Lever ses propres exceptions avec raise

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [31]:

```python
def verifier_age(age):
    if not isinstance(age, int):
        raise TypeError("L'age doit etre un entier")
    if age < 0:
        raise ValueError("L'age ne peut pas etre negatif")
    if age > 150:
        raise ValueError("L'age semble irrealiste")
    return True

# Tests
try:
    verifier_age(25)
    print("25 ans : valide")
except (TypeError, ValueError) as e:
    print(f"Erreur : {e}")

try:
    verifier_age(-5)
except (TypeError, ValueError) as e:
    print(f"-5 ans : {e}")

try:
    verifier_age("vingt")
except (TypeError, ValueError) as e:
    print(f"'vingt' : {e}")
```

Out[31]:

```
25 ans : valide
-5 ans : L'age ne peut pas etre negatif
'vingt' : L'age doit etre un entier
```

6.4 .6 Cas pratique complet : Validation de formulaire

Cas concret : Inscription à un site web

Créons un système de validation pour un formulaire d'inscription.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [32]:

```python
def valider_email(email):
    if "@" not in email:
        raise ValueError("L'email doit contenir '@'")
    if "." not in email.split("@")[1]:
        raise ValueError("Le domaine doit contenir un '.'")
    return True

def valider_mot_de_passe(mdp):
    if len(mdp) < 8:
        raise ValueError("Minimum 8 caracteres")
    if not any(c.isupper() for c in mdp):
        raise ValueError("Au moins une majuscule")
    if not any(c.isdigit() for c in mdp):
        raise ValueError("Au moins un chiffre")
    return True

def inscrire_utilisateur(email, mot_de_passe):
    erreurs = []

    try:
        valider_email(email)
    except ValueError as e:
        erreurs.append(f"Email : {e}")

    try:
        valider_mot_de_passe(mot_de_passe)
    except ValueError as e:
        erreurs.append(f"Mot de passe : {e}")

    if erreurs:
        print("Inscription echouee :")
        for err in erreurs:
            print(f"  - {err}")
        return False
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
    else:
        print(f"Inscription reussie pour {email} !")
        return True
```

In [33]:

```
## Test 1 : Donnees valides
print("=== Test 1 : Donnees valides ===")
inscrire_utilisateur("jean@email.com", "MonMdp123")

# Test 2 : Email invalide
print("\n=== Test 2 : Email invalide ===")
inscrire_utilisateur("jean-email.com", "MonMdp123")

# Test 3 : Mot de passe faible
print("\n=== Test 3 : Mot de passe faible ===")
inscrire_utilisateur("jean@email.com", "abc")
```

Out[33]:

```
=== Test 1 : Donnees valides ===
Inscription reussie pour jean@email.com !
=== Test 2 : Email invalide ===
Inscription echouee :
- Email : L'email doit contenir '@'
=== Test 3 : Mot de passe faible ===
Inscription echouee :
- Mot de passe : Minimum 8 caracteres
```

6.5 Introduction à l'utilisation des bibliothèques

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Bibliothèque (ou module)

Une **bibliothèque** est un ensemble de fonctions et de classes préécrites que vous pouvez réutiliser dans vos programmes. C'est comme une boîte à outils : plutôt que de fabriquer chaque outil, vous utilisez ceux qui existent déjà.

Types de bibliothèques Python

Bibliothèque standard – Incluse avec Python `math, random, datetime`

Bibliothèques tierces – À installer avec pip `numpy, pandas, requests`

Vos propres modules – Fichiers .py créés `mes_fonctions.py`

Figure 6.5 – Les trois types de bibliothèques Python

6.5.1 Importer une bibliothèque

Il existe plusieurs façons d'importer une bibliothèque :

In [34]:
```
## Methode 1 : Import complet
import math

print(math.pi)
print(math.sqrt(16))
```

Out[34]:
```
3.141592653589793
4.0
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [35]:
```
## Methode 2 : Import avec alias (raccourci)
import math as m

print(m.pi)
print(m.sqrt(25))
```

Out[35]:
```
3.141592653589793
5.0
```

In [36]:
```
## Methode 3 : Import de fonctions specifiques
from math import pi, sqrt

print(pi)
print(sqrt(36))
```

Out[36]:
```
3.141592653589793
6.0
```

6.5.2 La bibliothèque math

Cas concret : Calculs géométriques

Calculons l'aire et le périmètre d'un cercle, et la diagonale d'un rectangle.

In [37]:
```
import math

rayon = 5
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```

aire_cercle = math.pi * rayon ** 2
perimetre_cercle = 2 * math.pi * rayon

print("=== CERCLE (rayon = 5) ===")
print(f"Aire : {aire_cercle:.2f} unites carrees")
print(f"Perimetre : {perimetre_cercle:.2f} unites")

longueur = 4
largeur = 3

diagonale = math.sqrt(longueur**2 + largeur**2)

print("\n=== RECTANGLE (4 x 3) ===")
print(f"Diagonale : {diagonale:.2f} unites")
```

Out[37]:
```
=== CERCLE (rayon = 5) ===
Aire : 78.54 unites carrees
Perimetre : 31.42 unites
=== RECTANGLE (4 x 3) ===
Diagonale : 5.00 unites
```

In [38]:
```
import math

print("=== FONCTIONS MATHEMATIQUES ===")

print(f"ceil(4.3) = {math.ceil(4.3)}")
print(f"floor(4.7) = {math.floor(4.7)}")

print(f"pow(2, 10) = {math.pow(2, 10)}")
print(f"log10(1000) = {math.log10(1000)}")

angle = math.pi / 4
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```python
print(f"sin(45 deg) = {math.sin(angle):.4f}")
print(f"cos(45 deg) = {math.cos(angle):.4f}")
```

Out[38]:
```
=== FONCTIONS MATHEMATIQUES ===
ceil(4.3) = 5
floor(4.7) = 4
pow(2, 10) = 1024.0
log10(1000) = 3.0
sin(45 deg) = 0.7071
cos(45 deg) = 0.7071
```

6.5.3 La bibliothèque random

Cas concret : Jeu de dés et tirage au sort

In [39]:
```python
import random

print("=== LANCER DE DE ===")
for i in range(5):
    de = random.randint(1, 6)
    print(f"Lancer {i+1} : {de}")
```

Out[39]:
```
=== LANCER DE DE ===
Lancer 1 : 4
Lancer 2 : 2
Lancer 3 : 6
Lancer 4 : 1
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
Lancer 5 : 5
```

In [40]:

```python
import random

participants = ["Alice", "Bob", "Charlie", "Diana", "Eve"]

print("=== TIRAGE AU SORT ===")
gagnant = random.choice(participants)
print(f"Le gagnant est : {gagnant} !")

print("\n=== MELANGE ===")
cartes = ["As", "Roi", "Dame", "Valet", "10"]
print(f"Avant : {cartes}")
random.shuffle(cartes)
print(f"Apres : {cartes}")
```

Out[40]:

```
=== TIRAGE AU SORT ===
Le gagnant est : Charlie !
=== MELANGE ===
Avant : ['As', 'Roi', 'Dame', 'Valet', '10']
Apres : ['Dame', '10', 'As', 'Valet', 'Roi']
```

In [41]:

```python
import random

print("=== NOMBRES DECIMAUX ===")
for i in range(3):
    r = random.random()
    print(f"random() : {r:.4f}")

print("\n=== ECHANTILLON LOTO ===")
nombres = list(range(1, 50))
loto = random.sample(nombres, 6)
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
print(f"Numeros du loto : {sorted(loto)}")
```

Out[41]:
```
=== NOMBRES DECIMAUX ===
random() : 0.2847
random() : 0.9123
random() : 0.4521
=== ECHANTILLON LOTO ===
Numeros du loto : [7, 12, 23, 31, 38, 44]
```

6.5.4 La bibliothèque datetime

Cas concret : Gestion des dates et heures

In [42]:
```
from datetime import datetime, date, timedelta

maintenant = datetime.now()
print("=== DATE ET HEURE ACTUELLES ===")
print(f"Date et heure : {maintenant}")
print(f"Annee : {maintenant.year}")
print(f"Mois : {maintenant.month}")
print(f"Jour : {maintenant.day}")
```

Out[42]:
```
=== DATE ET HEURE ACTUELLES ===
Date et heure : 2025-01-21 14:30:45.123456
Annee : 2025
Mois : 1
Jour : 21
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

In [43]:
```
from datetime import date

naissance = date(2000, 5, 15)
print(f"Date de naissance : {naissance}")

aujourdhui = date.today()
age = aujourdhui.year - naissance.year

if (aujourdhui.month, aujourdhui.day) < (naissance.month,
   naissance.day):
    age -= 1

print(f"Age : {age} ans")
```

Out[43]:
```
Date de naissance : 2000-05-15
Age : 24 ans
```

In [44]:
```
from datetime import datetime, timedelta

print("=== CALCUL DE DATES ===")

aujourdhui = datetime.now()
print(f"Aujourd'hui : {aujourdhui.strftime('%d/%m/%Y')}")

dans_une_semaine = aujourdhui + timedelta(days=7)
print(f"Dans 1 semaine :
   {dans_une_semaine.strftime('%d/%m/%Y')}")

il_y_a_30_jours = aujourdhui - timedelta(days=30)
print(f"Il y a 30 jours :
   {il_y_a_30_jours.strftime('%d/%m/%Y')}")
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
Out[44]: === CALCUL DE DATES ===
Aujourd'hui : 21/01/2025
Dans 1 semaine : 28/01/2025
Il y a 30 jours : 22/12/2024
```

6.5 .5 Installer des bibliothèques externes avec pip

pip

pip (Pip Installs Packages) est le gestionnaire de paquets de Python. Il permet d'installer des bibliothèques créées par la communauté.

Terminal

```
## Installer une bibliotheque
pip install requests

# Installer une version specifique
pip install numpy==1.24.0

# Mettre a jour une bibliotheque
pip install --upgrade pandas

# Voir les bibliotheques installees
pip list
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Bibliothèque	Domaine	Description
`requests`	Web	Faire des requêtes HTTP
`numpy`	Calcul	Calcul numérique avec tableaux
`pandas`	Données	Manipulation de données
`matplotlib`	Graphiques	Création de graphiques
`pillow`	Images	Manipulation d'images
`flask`	Web	Création de sites web
`pygame`	Jeux	Développement de jeux

Tableau 6.2 – Bibliothèques Python populaires pour débutants

6.6 Documentation et structuration du code

Citation célèbre

« Le code est lu beaucoup plus souvent qu'il n'est écrit. »
— Guido van Rossum, créateur de Python

6.6.1 Les commentaires

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Commentaire

Un **commentaire** est une note dans le code destinée aux humains. Python l'ignore complètement lors de l'exécution.

In [45]:
```
## Commentaire sur une ligne

# Calculer le prix TTC d'un produit
prix_ht = 100
taux_tva = 0.20
prix_ttc = prix_ht * (1 + taux_tva)

print(f"Prix TTC : {prix_ttc} EUR")
```

Out[45]:
```
Prix TTC : 120.0 EUR
```

Quand commenter ?

- Expliquer **pourquoi** vous faites quelque chose (pas **quoi**)
- Clarifier un algorithme complexe
- Documenter les hypothèses et les limitations
- Marquer les tâches à faire avec `# TODO:`

6.6.2 Les docstrings

Docstring

Une **docstring** (documentation string) est une chaîne de caractères placée au début d'une fonction pour la documenter.

In [46]:
```
def calculer_imc(poids, taille):
    """
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
    Calcule l'Indice de Masse Corporelle (IMC).

    Parametres:
        poids (float): Le poids en kilogrammes
        taille (float): La taille en metres

    Retourne:
        float: L'IMC calcule
    """
    return poids / (taille ** 2)

mon_imc = calculer_imc(70, 1.75)
print(f"Mon IMC : {mon_imc:.2f}")

print("\n--- Documentation ---")
print(calculer_imc.__doc__)
```

```
Out[46]:
Mon IMC : 22.86
-- Documentation --
Calcule l'Indice de Masse Corporelle (IMC).
Parametres: poids (float): Le poids en kilogrammes taille
(float): La taille en metres
Retourne: float: L'IMC calcule
```

6.6.3 Les annotations de type (type hints)

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Type hints

Les **annotations de type** permettent d'indiquer les types attendus des paramètres et de la valeur de retour d'une fonction.

In [47]:
```python
def saluer(nom: str) -> str:
    """Salue une personne par son nom."""
    return f"Bonjour, {nom} !"

print(saluer("Alice"))
print(saluer("Bob"))
```

Out[47]:
```
Bonjour, Alice !
Bonjour, Bob !
```

In [48]:
```python
def calculer_moyenne(notes: list) -> float:
    """Calcule la moyenne d'une liste de notes."""
    if len(notes) == 0:
        return 0.0
    return sum(notes) / len(notes)

def est_admis(moyenne: float, seuil: float = 10.0) -> bool:
    """Verifie si un etudiant est admis."""
    return moyenne >= seuil

mes_notes = [12.5, 14.0, 11.0, 15.5, 13.0]
ma_moyenne = calculer_moyenne(mes_notes)
admis = est_admis(ma_moyenne)

print(f"Notes : {mes_notes}")
print(f"Moyenne : {ma_moyenne:.2f}")
print(f"Admis : {admis}")
```

Out[48]:
```
Notes : [12.5, 14.0, 11.0, 15.5, 13.0]
Moyenne : 13.20
Admis : True
```

6.7 Bonnes pratiques pour un code clair et lisible

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

PEP 8

PEP 8 est le guide de style officiel pour le code Python. Il définit des conventions pour rendre le code plus lisible et cohérent.

6.7.1 Nommage des variables et fonctions

Élément	Convention	Exemple	Règle
Variables	snake_case	`prix_total`	Minuscules + underscores
Fonctions	snake_case	`calculer_moyenne`	Verbe + action
Classes	PascalCase	`CompteBancaire`	Majuscule chaque mot
Constantes	UPPER_CASE	`TVA_TAUX`	Majuscules

Tableau 6.3 – Conventions de nommage en Python (PEP 8)

In [49]:
```
## MAUVAIS : noms peu clairs
x = 100
y = 0.20
z = x * (1 + y)
```

In [50]:
```
## BON : noms clairs et conventions respectees
prix_ht = 100
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
taux_tva = 0.20
prix_ttc = prix_ht * (1 + taux_tva)

TAUX_TVA_STANDARD = 0.20
```

Noms descriptifs

Un bon nom de variable décrit son contenu. Évitez les abréviations obscures.

- `n` → `nombre_eleves`
- `lst` → `liste_notes`
- `d` → `dictionnaire_utilisateurs`

6.7.2 Indentation et espacement

In [51]:

```python
## BON : 4 espaces pour l'indentation
def verifier_parite(nombre):
    if nombre % 2 == 0:
        resultat = "pair"
    else:
        resultat = "impair"
    return resultat

# BON : espaces autour des operateurs
x = 10 + 5
y = x * 2
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
z = (x + y) / 3

# BON : espace apres les virgules
ma_liste = [1, 2, 3, 4, 5]
```

6.7.3 Longueur des lignes

Règle des 79 caractères

PEP 8 recommande de limiter les lignes à **79 caractères maximum**. Cela facilite la lecture et permet d'afficher le code sur différents écrans.

In [52]:
```
## BON : ligne decoupee proprement
resultat = calculer_prix_total(
    prix_unitaire,
    quantite,
    taux_remise,
    taux_tva,
    frais_livraison
)

# BON : utiliser des parentheses pour couper
message = (
    "Bonjour, ceci est un message tres long "
    "qui s'etend sur plusieurs lignes "
    "pour une meilleure lisibilite."
)
print(message)
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Out[52]:
```
Bonjour, ceci est un message tres long qui s'etend sur
plusieurs lignes pour une meilleure lisibilite.
```

6.7.4 Organisation du code

In [53]:
```python
"""
Module de gestion des notes des etudiants.
"""

# 1. Imports
import math
from datetime import datetime

# 2. Constantes
SEUIL_ADMISSION = 10.0
SEUIL_MENTION_BIEN = 14.0
SEUIL_MENTION_TRES_BIEN = 16.0

# 3. Fonctions
def calculer_moyenne(notes):
    """Calcule la moyenne d'une liste de notes."""
    if not notes:
        return 0.0
    return sum(notes) / len(notes)

def determiner_mention(moyenne):
    """Determine la mention selon la moyenne."""
    if moyenne >= SEUIL_MENTION_TRES_BIEN:
        return "Tres Bien"
    elif moyenne >= SEUIL_MENTION_BIEN:
        return "Bien"
    elif moyenne >= SEUIL_ADMISSION:
```

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

```
28          return "Passable"
29      else:
30          return "Ajourne"
31
32  # 4. Code principal
33  mes_notes = [15, 12, 18, 14, 16]
34  moyenne = calculer_moyenne(mes_notes)
35  mention = determiner_mention(moyenne)
36
37  print(f"Moyenne : {moyenne:.2f}")
38  print(f"Mention : {mention}")
```

```
Out[53]:
Moyenne : 15.00
Mention : Bien
```

6.7.5 Récapitulatif des bonnes pratiques

Les 10 commandements du code Python

1. **Noms descriptifs** : `prix_total` plutôt que `pt`
2. **snake_case** pour les variables et fonctions
3. **4 espaces** pour l'indentation (pas de tabulations)
4. **Espaces** autour des opérateurs : `x = 1 + 2`
5. **Lignes courtes** : maximum 79 caractères
6. **Ligne vide** entre les fonctions
7. **Docstrings** pour documenter les fonctions
8. **Commentaires utiles** qui expliquent le « pourquoi »

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

9. **Imports en haut** du fichier
10. **Constantes en MAJUSCULES**

Ce qu'il faut retenir

- **Trois types d'erreurs** : syntaxe, exécution, logique
- **Débogage par print()** : simple et efficace
- **Lire les tracebacks** : commencer par la fin
- **try/except** : capturer et gérer les exceptions
- **Bibliothèques** : `import math`, `import random`
- **Documentation** : commentaires et docstrings
- **PEP 8** : conventions pour un code lisible
- **Nommage** : snake_case, noms descriptifs

6.8 Travaux Pratiques

Travaux Pratiques : Débogage et Bonnes Pratiques

Les exercices suivants sont conçus pour vous faire identifier et corriger des erreurs, et pour vous familiariser avec les outils de débogage.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

TP 1 – Chasse aux bugs de syntaxe

Le code ci-dessous contient 3 erreurs de syntaxe et d'indentation. Trouvez-les et corrigez-les pour que le programme s'exécute correctement.

Code à corriger :

```
def calculer_somme(nombres)
    somme = 0
    for i in nombres
        somme += i
        return somme

resultat = calculer_somme([1, 2, 3, 4, 5])
print("La somme est : " + resultat)
```

Indice : Vérifiez les deux-points, l'indentation et le type de la dernière variable.

TP 2 – Détection d'erreurs logiques

Ce code ne lève aucune erreur mais produit un résultat incorrect. Identifiez le bug logique et corrigez-le.

Code à corriger :

```
def est_pair(n):
    if n / 2 == 0:  # Problème ici ?
        return True
    else:
        return False

print(est_pair(4))  # Affiche False, mais devrait
   afficher True
```

Indice : L'opérateur de division peut poser problème pour vérifier la parité. Pensez au modulo.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

TP 3 – Débogage interactif avec PDB

Insérez un point d'arrêt `pdb` dans le code suivant pour analyser la valeur de la variable `carre` à chaque itération de la boucle.

Code à analyser :

```
def calculer_carrés(n):
    resultats = []
    for i in range(n):
        carre = i * i
        resultats.append(carre)
    return resultats

calculer_carrés(5)
```

Instructions :

1. Ajoutez `import pdb; pdb.set_trace()` à la bonne ligne.
2. Exécutez le script.
3. Utilisez les commandes `n` (next), `p carre` (print), et `c` (continue) pour observer l'exécution.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

TP 4 – Rendre un code robuste

Le programme suivant plantera si l'utilisateur entre 0. Modifiez-le pour gérer cette erreur et afficher un message clair.

Code à corriger :

```
def calculer_division():
    numerateur = int(input("Entrez le numérateur : "))
    denominateur = int(input("Entrez le dénominateur :
    "))
    resultat = numerateur / denominateur
    print(f"Résultat : {resultat}")

calculer_division()
```

Indice : Utilisez un bloc `try...except` pour intercepter `ZeroDivisionError`.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

TP 5 – Débogueur de configuration

Créez un script qui charge des paramètres depuis un fichier JSON. Le script doit gérer proprement les erreurs :

- Si le fichier n'existe pas, affichez "Fichier de configuration introuvable."
- Si le fichier est vide ou a un format JSON invalide, affichez "Erreur de format dans le fichier de configuration."
- Si tout va bien, affichez les paramètres chargés.

Structure du fichier `config.json` :

```
{
    "user": "admin",
    "port": 8080,
    "debug": true
}
```

Instructions :

1. Créez le fichier `config.json`.
2. Écrivez le script `charger_config.py` qui utilise `try...except` pour gérer `FileNotFoundError` et `json.JSONDecodeError`.
3. Testez-le en modifiant/supprimant le fichier JSON pour vérifier la robustesse.

6.9 Quiz du Chapitre

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

1 **[Vrai/Faux]** Une erreur de syntaxe (SyntaxError) est détectée par l'interpréteur Python AVANT l'exécution du programme.

☐ Vrai ☐ Faux

2 **[QCM]** Quelle commande permet d'arrêter l'exécution d'un programme Python à un point donné pour lancer le débogueur ?

A. `import debug`

B. `pdb.stop()`

C. `breakpoint()` ou `import pdb; pdb.set_trace()`

D. `pause()`

3 **[QCM]** Dans un bloc `try...except`, quelle clause est exécutée si une erreur de type `ValueError` se produit ?

A. `except ValueError:`

B. `except Exception as e:`

C. `catch ValueError:`

D. `finally:`

4 **[Vrai/Faux]** La clause `finally` dans un bloc `try...except...finally` est exécutée uniquement si aucune erreur ne se produit.

☐ Vrai ☐ Faux

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

5 **[QCM]** Quelle est la meilleure pratique pour s'assurer qu'un fichier est bien fermé après son utilisation, même en cas d'erreur ?

A. Appeler `fichier.close()` après chaque opération.

B. Utiliser un bloc `with open(...) as f:`.

C. Ne pas s'en soucier, Python gère tout automatiquement.

D. Utiliser une boucle `while` pour garder le fichier ouvert.

6 **[QCM]** Quelle commande du débogueur `pdb` permet d'exécuter la ligne de code suivante sans entrer dans les fonctions ?

A. `s` (step)

B. `n` (next)

C. `c` (continue)

D. `u` (up)

7 **[Vrai/Faux]** La commande `p ma_variable` dans `pdb` permet d'afficher la valeur de `ma_variable`.

☐ Vrai ☐ Faux

8 **[QCM]** Quel est le nom de l'erreur qui se produit lorsqu'on tente d'accéder à un index qui n'existe pas dans une liste ?

A. `TypeError`

B. `ValueError`

C. `IndexError`

D. `KeyError`

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

9 **[QCM]** Quelle est la bonne syntaxe pour un bloc `try...except` qui ne capture qu'une seule erreur spécifique ?

A. `try: ... except: ...`

B. `try: ... except ValueError: ...`

C. `try: ... catch ValueError: ...`

D. `do: ... on error: ...`

10 **[Question ouverte]** Citez deux bonnes pratiques de programmation qui aident à prévenir les bugs.

6.10 Récapitulatif du Chapitre

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Résumé

Types d'erreurs

- **SyntaxError** : Erreur de frappe, manque de `:`, etc.
- **IndentationError** : Problème d'alignement du code.
- **TypeError** : Opération sur des types incompatibles.
- **ValueError** : Valeur incorrecte pour une fonction.
- **IndexError** : Index de liste hors limites.
- **KeyError** : Clé de dictionnaire inexistante.
- **FileNotFoundError** : Fichier introuvable.
- **ZeroDivisionError** : Division par zéro.

Outils de Débogage

- `print()` : Simple pour afficher des valeurs.
- `pdb` : Débogueur interactif (`n`, `s`, `c`, `p`).
- `breakpoint()` : Point d'arrêt moderne (Python 3.7+).
- `assert` : Pour vérifier des conditions à un point donné.

Gestion des Erreurs

- `try:` : Bloc de code à exécuter.
- `except ExceptionType:` : Gère une erreur spécifique.
- `else:` : S'exécute si aucune erreur.
- `finally:` : S'exécute toujours (nettoyage).

Bonnes Pratiques

- **Lisez les messages d'erreur** : La dernière ligne est cruciale.
- **Commentez le code** : Expliquez le *pourquoi*, pas le *comment*.
- **Utilisez des noms clairs** : `calculer_moyenne` au lieu de `calc()`.
- **Testez petit à petit** : Ne pas écrire 100 lignes avant de tester.
- **Versionnez votre code** : Utilisez Git.
- **Gardez le code simple** : Lisibilité avant la complexité.

 © 2024–2025 Dr. AFENYIVEH Dzo M. Serge

Table des matières détaillée